Matter Test-Harness User Manual

v2.16-beta3+winter2026

Test-Harness Links

Matter Version	TH Version	TH image location	Supporting Documentation	SDK commit	Comments
Matter 1.0	v2.6	link	Causeway Link	96c9357	
Matter 1.1	v2.8.1	link	Causeway Link	5a21d17	
Matter 1.2	TH-Fall2023	link	Causeway Link	19771ed	
Matter 1.3	v2.10.4+spring2024	N/A	Causeway Link	50bcad1	To install, use the tag "v2.10.4+spring2024" in the instructions of TH Installation on Raspberry Pi
Matter 1.4	v2.11+fall2024	N/A	Causeway Link	f2e5de7	To install, use the tag "v2.11+fall2024" in the instructions of TH Installation on Raspberry Pi
Matter 1.4.1	v2.12+spring2025	N/A	Causeway Link	91eab26	To install, use the tag "v2.12+spring2025" in the instructions of TH Installation on Raspberry Pi
Matter 1.4.2	v2.13+summer2025	N/A	Causeway Link	1b2b3fd	To install, use the tag "v2.13+summer2025" in the instructions of TH Installation on Raspberry Pi
Matter 1.5	v2.14+fall2025	N/A	Causeway Link	ca9d111	To install, use the tag "v2.14+fall2025" in the instructions of TH Installation on Raspberry Pi
Matter 1.5.1	v2.14.1+winter2026	N/A	Causeway Link	e9ecfc2	To install, use the branch "v2.14.1+winter2026" in the instructions of TH Installation on Raspberry Pi

Matter 1.6	v2.15+spring2026	N/A	Causeway Link	b91b83f	To install, use the branch "v2.15+spring2026" in the instructions of TH Installation on Raspberry Pi
Matter 1.6.1	v2.15.1+spring2026	N/A	TBD	df8bd03	To install, use the branch "{v2.15.1+spring2026}" in the instructions of TH Installation on Raspberry Pi
Matter 1.7	v2.16-beta3+winter2026	N/A	TBD	6ff03a2	To install, use the branch "v2.16-beta3+winter2026" in the instructions of TH Installation on Raspberry Pi

Revision History

Revision	Date	Author	Description
1	11-Jan-2022	[GRL]Suraj Seenivasan	* Initial version of the user manual used for V1.0.
2	22-Aug-2022	[GRL]Suraj Seenivasan	* Updated the steps required to run the Python script inside docker: Input given by Tennessee Carmel-Veilleux.
3	06-Feb-2023	[GRL]Suraj Seenivasan	* Added instructions to update the existing image and personal access token: How to execute simulated test cases.
4	09-Feb-2023	[GRL]Suma	* Structuring the user manual. * Added relevant screenshots.
5	24-Feb-2023	[SiLabs]Yinyi Hu	* Instruction to Flash SiLabs RCP. * Instruction to collect Logs and submit to TEDS.
6	01-Mar-2023	[GRL]Suraj Seenivasan	* Instructions to flash nRF52840 Dongle and nRF52840 DK.
7	16-Mar-2023	[Apple]Fábio Wladimir Monteiro Maia	* Added TH layout explanation.
8	12-Jun-2023	[Apple]Carolina Lopes	* Added instructions to install TH without a Raspberry Pi.
9	06-Sep-2023	[Apple]Antonio Melo Jr.	* Moved the table of contents to a further page. * Added TH links for images and support documentations.
10	25-Oct-2023	[Apple]Hilton Lima	* Added SDK commit column to Test-Harness Links table.
11	26-Oct-2023	[Apple]Carolina Lopes	* Updated some links and images.
12	06-Dec-2023	[Apple]Hilton Lima	* Added Custom Yaml test section. * Updated TH update instructions. * Revision History table style formated.
13	07-Dec-2023	[Apple]Carolina Lopes	* Updated instructions on how to install TH without a Raspberry Pi.
14	14-Dec-2023	[Apple]Hilton Lima	* Updated SDK Python Test section.
15	20-Dec-2023	[Apple]Antonio Melo Jr.	* Updated Test Harness links related to release of Spring 2024.
16	04-Jan-2024	[Apple]Antonio Melo Jr.	* Updated Test Harness Release location link to the new Drive folder.

17	04-Jan-2024	[Apple]Romulo Quidute	* Renamed Custom Yaml test section to Customized Test Scripts and also added Custom Python test information.
18	05-Jan-2024	[Apple]Carolina Lopes	* Added information about the SDK Python Test suites.
19	30-Jan-2024	[Apple]Romulo Quidute	* Updated some referenced links.
20	07-Feb-2024	[Apple]Hilton Lima	* Added 'Common Test Failures' section for SDK tests.
21	07-Feb-2024	[Apple]Carolina Lopes	* Added 'Bringing up the Matter Python REPL' section.
22	15-Feb-2024	[Apple]Hilton Lima	* Updated 'TH Installation on Raspberry Pi' section.
23	26-Feb-2024	[Apple]Carolina Lopes	* Added volume mapping for data model XML files on docker run commands for SDK containter.
24	27-Feb-2024	[Apple]Carolina Lopes	* Updated mapping in docker run command for SDK container.
25	27-Mar-2024	[Apple]Hilton Lima	* Update nRF52840 firmware download link.
26	01-Apr-2024	[Apple]Hilton Lima	* Update download link to Flash SiLabs RCP.
27	01-Apr-2024	[Apple]Romulo Quidute	* Update OTBR scripts location.
28	17-Apr-2024	[Apple]Hilton Lima	* Updated Test Harness links related to final release of Spring 2024.
29	18-Apr-2024	[Apple]Hilton Lima	* Update section 'TH installation without a Raspberry Pi'.
30	30-Apr-2024	[Apple]Hilton Lima	* Update location for update scripts.
31	10-Jun-2024	[Apple]Hilton Lima	* Updated Test Harness links.
32	17-Jun-2024	[Google]Tennessee Carmel-Veilleux	* Added an example of an external operational dataset.
33	01-Aug-2024	[Apple]Carolina Lopes	* Added information about Certification Mode.
34	02-Aug-2024	[Apple]Antonio Melo Jr.	* Update the Ubuntu mentions to the current supported version.

35	06-Aug-2024	[Apple]Antonio Melo Jr.	* Add the TH release version to the front page of the document. * Add a warning to TH update when using a different Ubuntu version. * Update the OTBR start script parameter. * Fix the Thread RCP Firmware link. * Add Nrfconnect Sample APPs Firmwares section with link.
36	09-Aug-2024	[Apple]Antonio Melo Jr.	* Adding the new cleanup procedure to the troubleshooting of TH Installation section.
37	13-Aug-2024	[Apple]Hilton Lima	* Remove old references of TH image (obsolete). * Add a warning to ensure that the username needs to be 'ubuntu'.
38	30-Aug-2024	[Apple]Hilton Lima	* Replace images by text examples in 'Project Configuration' section. * Removed section 'Collect Logs and Submit to TEDS'. * Added informations about 'qr-code' and 'manual-code' parameters.
39	04-Sep-2024	[Apple]Hilton Lima	* Moved PIXIT section. * Added 'Test Parameters for SDK Python Tests' section. * Changed Table of Contents display level.
40	24-Sep-2024	[Apple] Hilton Lima	* Updated nRF util and desktop application links.
41	14-Oct-2024	[Apple]Romulo Quidute	* Added Warning for WIFI_SSID with special chars or empty spaces. * Updated Test-Harness Links for v2.11+fall2024 release.
42	03-Jan-2025	[Apple]Romulo Quidute	* Added Reuse commissioning information section.
43	25-Feb-2025	[Apple]Romulo Quidute	* Updated supported Ubuntu version to 24.04.2.
44	25-Feb-2025	[Apple]Antonio Melo Jr.	* Replacing the patch number with a 'x' in the supported Ubuntu version.
45	13-Mar-2025	[Apple]Romulo Quidute	* Updated Test-Harness Links for v2.12+spring2025 release.
46	31-Mar-2025	[Apple]Romulo Quidute	* Added mapping volume section inside SDK Python Tests.

47	05-May-2025	[Apple]Romulo Quidute, [Apple]Antonio Melo	Changes for v2.13+summer2025 version: * Added the platform certification configuration section. * Updated the docker command in Factory-reset the DUT section * Added the Wi-Fi PAF section supported by Matter 1.4.2.
48	10-Nov-2025	[Apple]Romulo Quidute, [Apple]Antonio Melo	* Changes for v2.14+fall2025 version.
49	14-Nov-2025	[Apple]Antonio Melo Jr.	* Adding the Side Load feature instructions to the custom Test Cases section. * Fixing the Causeway links in the references section. * Fixing the REPL command in the Matter Python REPL section.
50	18-Nov-2025	[Apple]Antonio Melo Jr.	* Using the TH version variable in the Test-Harness Links table.
51	19-Nov-2025	[Apple]Antonio Melo Jr.	* Adding symbolic links to the custom folder paths in the custom Test Cases section.
52	05-Jan-2026	[Apple]Antonio Melo Jr.	* Adding the Test-Harness CLI section. * Fixing some git clone commands to include the recursive submodules flag.
53	19-Jan-2026	[Apple]Romulo Quidute	* Adding Troubleshooting for the hanging with ModuleNotFoundError issue. * Added Container Logging Configuration section.
54	22-Jan-2026	[Apple]Romulo Quidute	* Added OTA Image Build Procedure section with manual workflow for generating OTA requestor images.
55	05-Feb-2026	[Apple]Romulo Quidute	* Added Thread Pairing Mode section.
56	20-Mar-2026	[Apple]Romulo Quidute	* Fixed docker container name to use Docker Compose V2 naming convention (hyphen separator). * Updated Python Test Logging Configuration section.
57	15-Apr-2026	[GRL]Pradeep G	* Added nfc-thread and nfc-wifi pairing mode details under the Project Configuration section. * Added NFC Device Mode (including NFC-Thread Device Mode and NFC-WiFi Device Mode) within the Project Configuration section.
58	15-Apr-2026	[ST]Olivier Lorente	* Added NFC commissioning description.
59	29-Apr-2026	[GRL]Sumith D	* Updated Section 4.5.1 to include CLI usage guidance for side-loaded test cases, including the -custom suffix.

60	08-May-2026	[Apple]Romulo Quidute	* Updated Raspberry Pi prerequisites to specify minimum 8 GB RAM requirement.
61	15-May-2026	[Apple]Romulo Quidute	* Added grouped log download documentation using CLI.
62	20-May-2026	[Apple]Romulo Quidute	* Updated NFC Device Mode section to clarify that discriminator and setup_code are optional for nfc-thread, nfc-wifi and nfc-ethernet pairing modes — onboarding data is read from the NFC tag at runtime. * Added note on qr-code behavior: ignored for commissioning tests under NFC modes, preserved for non-commissioning tests.
63	20-May-2026	[Apple]Romulo Quidute	* Renamed Python Testing Suite categories for clarity: "Python Testing Suite" is now "Python Testing Suite - Auto commissioning" and "Python Testing Suite - No commissioning" is now "Python Testing Suite - No auto commissioning".
64	02-Jun-2026	[GRL]Kishok G	* Updated the all members causeway location url in reference section. * Modified the specific release url to Matter Specifications and Test Plans(home path)
65	15-Jun-2026	[Apple]Romulo Quidute	* Added TC Parameters Mapping File (`--tc-params-file` / `-m`) documentation in the CLI section.
66	07-Jul-2026	[Apple]Romulo Quidute	* Added Download project logs documentation in the CLI section.
67	16-Jul-2026	[GRL]Pradeep G	* Added clarification on hostname usage in Quick-Start Guide of CLI section.
68	21-Jul-2026	[Apple]Romulo Quidute	* Added instructions for running a side loaded Python script directly inside the container when it depends on shared SDK modules.
69	20-Aug-2026	[Apple]Romulo Quidute	* Added `th-cli rescan-tests` command documentation, allowing side-loaded test scripts to be picked up without restarting the backend.
70	21-Aug-2026	[Apple]Antonio Melo Jr.	* Added the `th_config` project configuration object and the User Prompt Timeout Configuration section. * Updated Python Test Logging Configuration section to document the `th_config.enable_realtime_python_test_logs` per-project override. * Documented the `--prompt-timeout` CLI flag for `run-tests`.

71	03-Sep-2026	[Apple]Romulo Quidute	* Updated Section 7.1 to use the current standalone nRF Util executable instead of the deprecated pip-based nrfutil package, and to clarify RCP dongle flashing can be done from a separate Mac/PC.
72	04-Sep-2026	[Apple]Antonio Melo Jr.	* Updated Container Logging Configuration section to document the `th_config.enable_container_logs` per-project override (issue #1091).
73	10-Sep-2026	[Google]Andrey Khodyrev	* Added 'For CNET Tests' section with wpa_supplicant volume mapping instructions for the SDK docker container.
74	10-Sep-2026	[Apple]Romulo Quidute	* Added Repeat, Export, and Import a Test Run Execution sections to the CLI documentation.
75	21-Sep-2026	[Apple]Romulo Quidute	* Added PICS Export, Rename, Delete, and Archive/Unarchive Test Run Execution sections to the CLI documentation.
76	23-Sep-2026	[Apple]Romulo Quidute	* Updated OTBR docker image references in the Thread dataset generation section to match the new default (`openthread/otbr` digest-pinned image), replacing the obsolete `nrfconnect/otbr:9185bda`.
77	28-Sep-2026	[Apple]Romulo Quidute	* Documented `project export`/`project import`, and added missing flag examples for `run-tests --no-streaming`, `test-run-execution` list filters (`--project-id`, `--skip`, `--limit`, `--json`), `available-tests --group-by-cluster`, `test-runner-status --json`, and the `--version` Backend/SDK breakdown.
78	29-Sep-2026	[Apple]Karsten Sperling	* Added trouble shooting guidance for Wi-Fi Fixture interfaces.
79	05-Oct-2026	[Apple]Romulo Quidute	* Added `th_config.sdk_container_apps_dir` documentation for mounting helper app binaries into the SDK container.
80	19-Oct-2026	[Apple]Antonio Melo Jr.	* Clarified that the Test-Harness CLI is installed and updated by `auto-install.sh` and `auto-update.sh` (and not by `start.sh`), in the installation, update and CLI Quick-Start sections (issue #1154). * Added the Project Configuration Reference section (field reference, mandatory fields, validation rules, and editing the project configuration from the UI and with `th-cli project edit`). Added `wifipaf-wifi` to the pairing modes list and fixed the `chip_use_paa_certs` field name (issue #1156).

81	20-Oct-2026	[Apple]Romulo Quidute	* Updated the Python Test Logging Configuration section: real-time logging is the default (issue #1171). * Removed the `ENABLE_REALTIME_PYTHON_TEST_LOGS` and `ENABLE_CONTAINER_LOGS` environment variable fallbacks from the Python Test Logging Configuration and Container Logging Configuration sections; both are now configured only with `th_config` in the project configuration (issue #1178).
82	05-Nov-2026	[Apple]Romulo Quidute	* Documented side loading of helper/base modules in the **custom** Python folder, including the rule that they shadow SDK modules of the same name.
83	05-Nov-2026	[Apple]Romulo Quidute	* Documented that `th-cli rescan-tests` now reports the test cases added, modified and removed by the rescan, lists scripts that could not be parsed, and lists the files added, modified or removed in the custom folder.
84	06-Nov-2026	[Apple]Romulo Quidute	* Added the Collecting Files for ATL Submission section, mapping the web UI download buttons to CLI commands, and documented the per-execution report (`test-run-execution --id <ID> --json`) (issue #1203). * Corrected the `--grouped` log archive description and noted that `test-run-execution export` is not the report to submit to an ATL.
85	10-Nov-2026	[Apple]Romulo Quidute	* Documented `th-cli project applicable-tests`, which lists the test cases that apply to a project based on its PICS configuration (issue #1206).

Table of contents

1. **Introduction** . . . 15
2. **References** . . . 16
3. **Test-Harness (TH) Design** . . . 17
   - 3.1. TH Layout . . . 17
   - 3.2. Data Model . . . 18
   - 3.3. Data Flow . . . 19
4. **Getting Started with Matter Test-Harness (TH)** . . . 20
   - 4.1. TH Installation on Raspberry Pi . . . 20
     - 4.1.1. Prerequisites . . . 20
     - 4.1.2. TH Installation on Raspberry Pi . . . 21
   - 4.2. TH installation without a Raspberry Pi . . . 22
     - 4.2.1. Create an Ubuntu virtual machine . . . 23
     - 4.2.2. Setup TH in Ubuntu . . . 23
     - 4.2.3. Substitute the SDK's docker image and update sample apps . . . 24
   - 4.3. Update Existing TH . . . 24
   - 4.4. Updating Existing Yaml Test Script . . . 25
   - 4.5. Customized Test Scripts (Yaml/Python Tests) . . . 26
     - 4.5.1. Test-Harness Side Load Feature with Custom Test Cases . . . 27
     - 4.5.2. Rescanning Side-Loaded Test Cases . . . 27
     - 4.5.3. Running a Side Loaded Python Script Directly Inside the Container . . . 29
     - 4.5.4. Side Loading Helper/Base Modules (e.g. `TC_SUTestBase.py`) . . . 29
   - 4.6. Troubleshooting . . . 30
     - 4.6.1. Read-Only File System Error . . . 30
     - 4.6.2. Generate Personal Access Token . . . 31
     - 4.6.3. Bringing Up of Docker Containers Manually . . . 31
     - 4.6.4. Cleaning The Environment Manually . . . 32
     - 4.6.5. Test Harness Startup Hanging with ModuleNotFoundError . . . 32
       - 4.6.5.1. Symptoms . . . 32
       - 4.6.5.2. Diagnosis . . . 32
       - 4.6.5.3. Solution . . . 33
     - 4.6.6. No Link-Local IPv6 Address Assigned to the Wi-Fi Fixture Interface . . . 33
       - 4.6.6.1. Symptoms . . . 33
       - 4.6.6.2. Diagnosis . . . 33
       - 4.6.6.3. Solution . . . 33
5. **Bringing Up of Matter Node (DUT) for Certification Testing** . . . 35
   - 5.1. Bringing Up of Reference Matter Node (DUT) on Raspberry Pi . . . 35
     - 5.1.1. To Provision Raspberry Pi Using Wi-Fi Configuration . . . 36
     - 5.1.2. To Provision Raspberry Pi Over Ethernet Configuration . . . 36

5.2. Bringing Up of Reference Matter Node (DUT) on Thread Platform . . . 36
5.2.1. Prerequisites . . . 36
5.2.2. Setting Up Thread Board (nRF52840-DK) . . . 37
5.2.2.1. Instructions to Set Up nRF52840-DK Using nRF Connect Desktop Application Tool . . . 37
5.2.2.2. Instructions to Set Up nRF52840-DK Using Docker Environment . . . 40
5.2.2.3. Building and Flashing Sample Apps for nRF-Connect . . . 41
6. **Bringing up the Matter Python REPL** . . . 43
7. **OT Border Router (OTBR) Setup** . . . 44
7.1. Instructions to Flash the Firmware NRF52840 RCPDongle . . . 44
7.2. Nrfconnect Sample APPs Firmwares to Flash on the NRF52840DK Kit . . . 45
7.3. Instructions to Flash SiLabs RCP . . . 45
7.4. Forming Thread Network and Generating Dataset for Thread Pairing . . . 47
7.5. Troubleshooting: Boarder Router Container failure to initialize . . . 49
8. **Wi-Fi PAF Commissioning** . . . 52
8.1. Components Needed for Tests . . . 52
8.1.1. Hardware . . . 52
8.1.2. Software . . . 52
8.2. Matter SDK . . . 53
8.3. Baseline Test Harness (TH) Configuration for Testing DUT Commissionable Device . . . 53
8.4. Baseline Test Harness (TH) Configuration for Testing DUT Commissioner . . . 53
8.5. Test Procedures and Test Commands . . . 54
8.6. Configure the environment . . . 54
9. **NFC Setup** . . . 56
9.1. Components Needed for Tests . . . 56
9.1.1. Hardware . . . 56
9.1.2. Software . . . 56
9.2. Test Procedures and Test Commands . . . 57
10. **Test Configuration** . . . 58
10.1. Project Configuration . . . 58
10.1.1. Projects Menu . . . 58
10.1.2. Wi-Fi Mode . . . 60
10.1.3. On Network Mode . . . 60
10.1.4. Thread Device Mode . . . 61
10.1.5. NFC Device Mode . . . 62
10.1.6. Thread Pairing Mode (using Border Agent) . . . 63
10.1.7. PAA Certificates . . . 64
10.2. Test Parameters . . . 64
10.2.1. Upload PICS File . . . 65
10.2.2. Test Menu . . . 66
10.2.3. Utility Menu . . . 70
10.2.4. Settings Menu . . . 71

10.3. Project Configuration Reference . . . 71
10.3.1. Editing the Project Configuration . . . 71
10.3.2. Structure and Mandatory Fields . . . 72
10.3.2.1. `dut_config` . . . 72
10.3.2.2. `network.wifi` . . . 73
10.3.2.3. `network.thread` . . . 74
10.3.2.4. `th_config` . . . 75
10.3.3. Validation Rules . . . 75
11. **OTA Image Build Procedure** . . . 76
11.1. Overview . . . 76
11.2. High-Level Workflow Steps . . . 76
11.3. Reference Commands (Raspberry Pi Platform) . . . 76
11.3.1. Build OTA Provider . . . 76
11.3.2. Update Software Version . . . 76
11.3.3. Build OTA Requestor . . . 76
11.3.4. Generate OTA Image . . . 77
11.3.5. Deploy OTA Image to Test Harness . . . 77
11.4. Important Notes . . . 77
12. **Test Case Execution** . . . 78
12.1. Automated and Semi Automated Tests . . . 79
12.1.1. Automated Test Cases . . . 79
12.1.2. Semi Automated Test Cases . . . 79
12.2. Python Tests . . . 81
12.3. Manual Tests . . . 83
12.4. Simulated Tests . . . 84
12.5. SDK Python Tests . . . 85
12.5.1. Run Tests Inside SDK Docker Container . . . 85
12.5.1.1. Prerequisite . . . 85
12.5.1.2. Mapped Volumes . . . 85
12.5.1.3. Placeholders for Steps . . . 86
12.5.1.4. Common Steps . . . 86
12.5.1.5. For NFC Pairing . . . 86
12.5.1.6. For On-Network Pairing . . . 87
12.5.1.7. For BLE+Wi-Fi Pairing . . . 87
12.5.1.8. For BLE+Thread Pairing . . . 87
12.5.1.9. For CNET Tests . . . 87
12.5.1.10. Post-Test Steps . . . 88
12.5.1.11. Possible Issues . . . 88
Common Test Failures . . . 88
12.5.2. Run Tests on the TH User Interface . . . 88
12.5.2.1. Test Parameters for SDK Python Tests . . . 89

PIXIT Support . . . 89
Test Parameters Examples . . . 90
12.5.2.2. Test suites . . . 90
12.5.2.3. Reuse commissioning information . . . 92
12.5.2.4. Python Test Logging Configuration . . . 93
12.5.2.5. Container Logging Configuration . . . 93
12.5.2.6. Helper App Binaries in the SDK Container . . . 94
12.5.2.7. User Prompt Timeout Configuration . . . 95
13. **Matter Test-Harness Command Line Interface (CLI)** . . . 96
13.1. Overview . . . 96
13.2. Quick-Start Guide . . . 96
13.3. Essential Commands . . . 97
13.3.1. Check Version and Status . . . 97
13.3.2. View Available Tests . . . 98
13.3.3. Rescan Test Collections . . . 98
13.3.4. Running Tests . . . 99
13.3.5. TC Parameters Mapping File (`--tc-params-file` / `-m`) . . . 99
13.3.6. Project Management . . . 100
13.3.7. View Test Execution History . . . 103
13.3.8. Repeat a Test Run Execution . . . 103
13.3.9. Export a Test Run Execution . . . 104
13.3.10. Import a Test Run Execution . . . 104
13.3.11. Export PICS Used by a Test Run Execution . . . 104
13.3.12. Rename a Test Run Execution . . . 104
13.3.13. Delete a Test Run Execution . . . 104
13.3.14. Archive / Unarchive a Test Run Execution . . . 105
13.3.15. Download Test Execution Logs . . . 105
13.3.16. Collecting Files for ATL Submission . . . 106
13.3.17. Abort Running Tests . . . 106
13.4. Advanced Features . . . 107
13.4.1. Camera Test Support . . . 107
13.4.2. Auto-Completion . . . 107
13.4.3. Colorized Output . . . 108
13.4.4. Manual Test Cases . . . 108
13.5. Troubleshooting . . . 108
14. **Matter Test Harness Cameras** . . . 110
14.1. Prerequisites . . . 110
14.1.1. Install GStreamer . . . 110
14.1.2. Chrome Browser Configuration . . . 110
14.2. Setup Environment . . . 111
14.3. Running Camera Test Cases . . . 111

14.3.1. Running chip-camera-app with USB Camera . . . 111
15. **Platform Certification Configuration** . . . 112
15.1. Selecting Test Cases Rules . . . 112
15.1.1. Platform Certification . . . 112
15.1.2. Derived Product Certification . . . 112
15.1.3. Full Product Certification . . . 113
15.2. Configuration Input . . . 113

1. Introduction

The Matter Test-Harness is a comprehensive test tool used for certification testing of Matter devices in accordance with the Matter protocol as defined in the Matter specification.

This user guide serves as the primary user documentation to work with the Test-Harness (**TH**) tool, providing high-level architecture of the tool, how to use the tool to execute certification tests and submit the test results to CSA for certification.

The TH tool runs on the Raspberry Pi platform, providing an intuitive Web user interface to create a test project, configure the project/Device Under Test (**DUT**) settings, load the required test cases using the PICS xml file and execute test cases for various devices (commissioner, controller and controlee) as defined in the Matter specification.

The TH tool provides an option to execute the following test scripts— Automated, Semi Automated, Python, Manual and Simulated. Upon completion of the test execution, detailed logs and test results will be available for user analysis. The user will also be able to submit logs to ATL's for review to obtain device certification.

The TH tool can be used by any DUT vendor to run the Matter certification tests, or by any hobby developer to get acquainted with the Matter certification testing tools or technologies.

2. References

1. Matter Specification: Matter Specification (Causeway) / Matter Specification (GitHub)
2. Matter SDK GitHub Repository: Connectedhomeip GitHub Repository
3. Matter Test Plans: Matter Test Plans (Causeway) / Matter Test Plans (GitHub)
4. Matter PICS Tool: PICS Tool - Connectivity Standards Alliance (csa-iot.org)
5. Matter XML Files: XML Files - Connectivity Standards Alliance (csa-iot.org)
6. TEDS Matter Tool: TEDS Matter Tool - Connectivity Standards Alliance (csa-iot.org)

Important: Some links contained in this user manual require a CSA membership and authentication as a CSA authorized user in order to be accessed

3. Test-Harness (TH) Design

This section outlines the TH architecture, data model and data flow on how different components of TH communicate with each other.

3.1. TH Layout

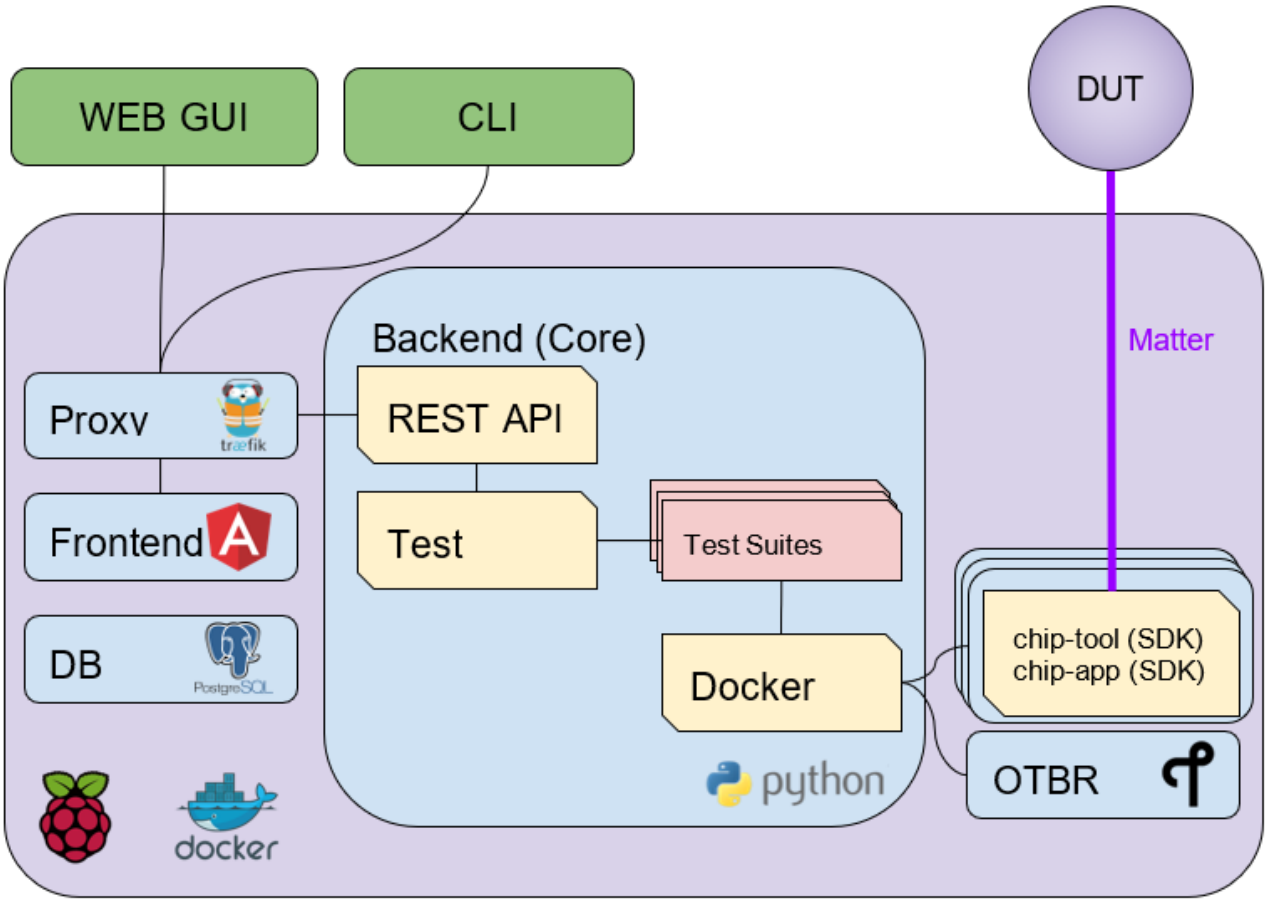


Figure 1. The Test-Harness Layout

Each of the main subsystems of the Test Harness (Proxy, Frontend, Backend and Database) runs on its own docker container deployed to a Ubuntu Raspberry Pi platform. The Proxy container hosts an instance of the traefik application proxy (https://traefik.io/traefik/) which is responsible to route user requests coming from an external (to the Raspberry Pi) web browser to either the Frontend or the Backend as appropriate. The Frontend container serves the dynamic web pages that comprise the Web GUI to be rendered on the user browser including the client-side logic. According to that client-side logic and user input, REST API requests are sent again by the external browser to the Application Proxy and get redirected to the Backend container, where a FastAPI (https://fastapi.tiangolo.com/) Python application implements the server-side logic. Any application information that needs to be persisted gets serialized and written by the server-side logic to the Postgres database running in the Database container.

In addition to the four main containers described above, which get created and destroyed when the Raspberry Pi platform respectively boots up and shuts down, two other containers are created and destroyed dynamically on demand according to the test execution lifecycle: the SDK container and the OTBR container. The SDK container has copies of the Matter SDK tools (binary executables)

which can be used to play the role of clients and servers of the Matter protocol in test interactions, either as Test Harness actuators or DUT simulators. That container gets automatically created and destroyed by the server-side logic at the start and at the end, respectively, of a Test Suite which needs actuators or simulators. The OTBR container, on the other hand, hosts an instance of the Open Thread Border Router and needs to be explicitly started by the TH user when they want to test a real Matter device that runs over a Thread fabric, as described in Section 7, OT Border Router (OTBR) Setup.

3.2. Data Model

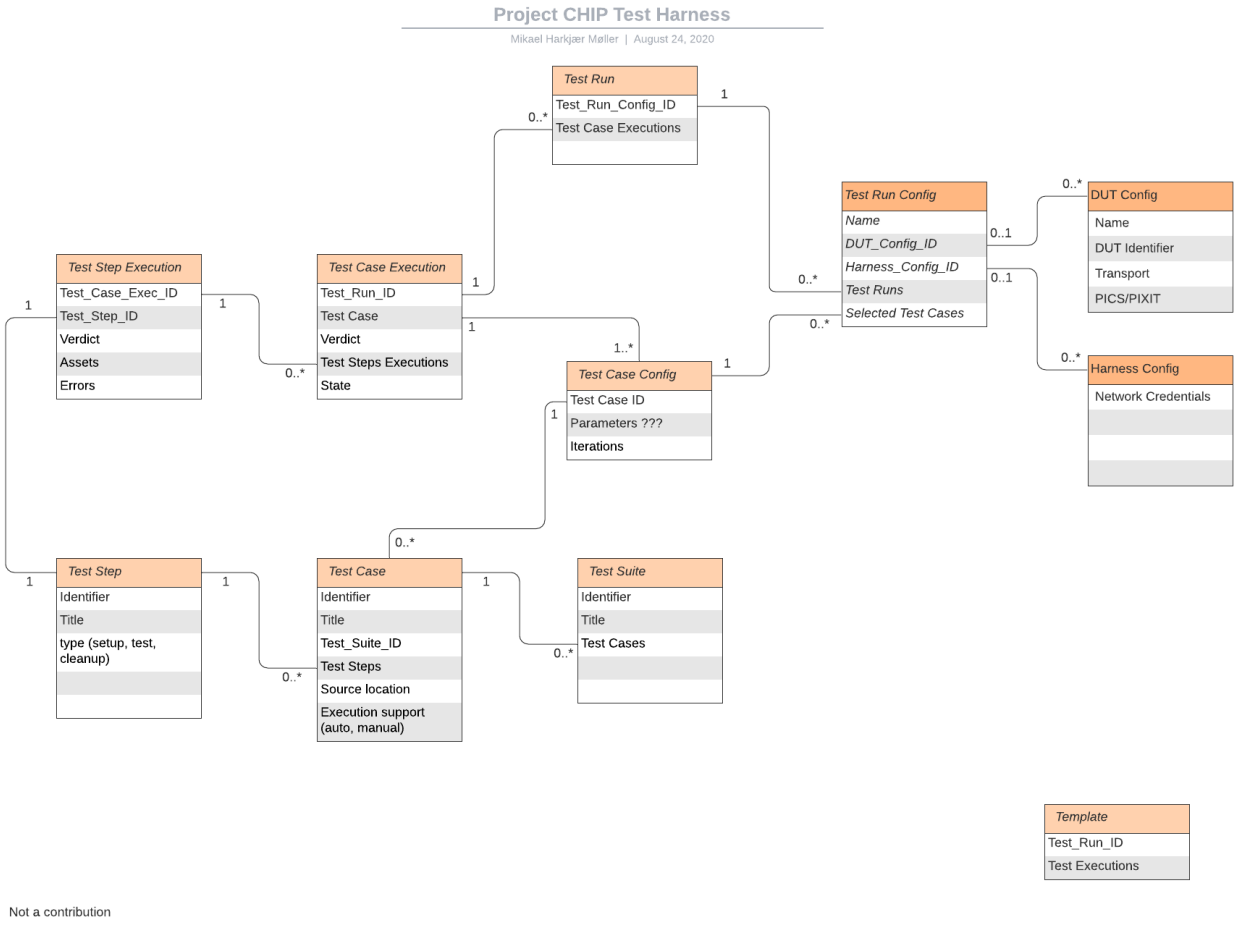


Figure 2. The Data Model

The data model diagram in Figure 2 shows the various data objects that the Test Execution consumes and maintains and the relationship between these data objects.

- Test Run
- Test Run Config
- DUT Config
- Harness Config
- Test Case Execution
- Test Step Execution

- Test Case
- Test Step
- Test Suite
- Test Case Config

3.3. Data Flow

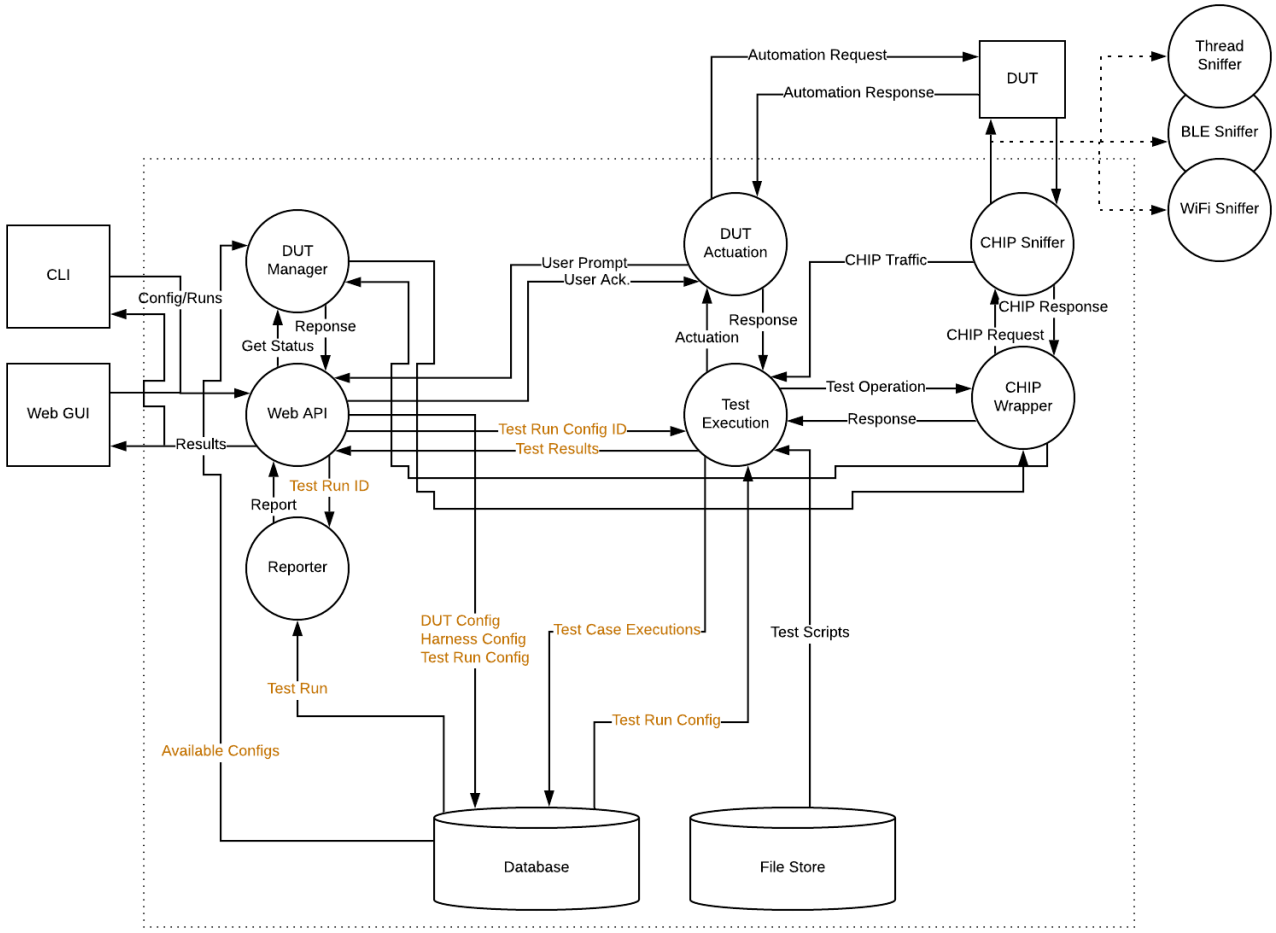


Figure 3. The Data Flow

4. Getting Started with Matter Test-Harness (TH)

The Matter Node (DUT) that is used for certification testing can either be a commissioner, controller or controlee.

If the DUT is a controlee (e.g., light bulb), the TH spins a reference commissioner/controller using chip-tool binary shipped with the SDK. The TH commissioner provisions the DUT and is used to execute the certification tests on the controlee.

If the DUT is a commissioner/controller, the Test TH spins an example accessory that is shipped with the SDK and uses that for the DUT to provision, control and run certification tests.

Refer to Section 5, Bringing Up of Matter Node (DUT) for Certification Testing to bring up the DUT and then proceed with device testing by referring to Section 8, Test Configuration.

For hobby developers who want to get acquainted with certification tools/process/TC's, can spin DUT's using the example apps provided in the SDK. Refer to the instructions to set up one here.

The TH runs on the official **Ubuntu Server 24.04.x LTS (64-bit)** version. If the TH device happens to be using a different Ubuntu release or other OS, we strongly recommend fresh installing version Ubuntu Server 24.04.x LTS (64-bit) for reliable results.

The official installation method uses a Raspberry Pi (TH Installation on Raspberry Pi), but there's an alternative method used in the tool's development that uses a virtual machine instead (TH installation without a Raspberry Pi). Keep in mind that thread networking is not officially supported in VM installations at the moment.

4.1. TH Installation on Raspberry Pi

There are two ways to obtain the latest TH on Raspberry Pi. Follow the instructions in Section 4.1.2, TH Installation on Raspberry Pi to install TH from scratch OR if you already have the TH, follow the instructions in Section 4.4, Update Existing TH to update the TH.

This instruction applies to the latest version of the Test Harness this document refers to. For earlier versions of the TH please follow the user guide of that specific TH version as, for example, Ubuntu versions might differ per installation.

4.1.1. Prerequisites

The following equipment will be required to have a complete TH setup:

- **Raspberry Pi Version (4 or 5) with minimum 8 GB RAM and SD card of minimum 64 GB storage**

The TH will be installed on Raspberry PI. The TH contains couple of docker container(s) with all the required dependencies for certification tests execution.

- **Windows or Linux System (Laptop/Desktop/Mac)**

The Mac/PC will be used to flash the Ubuntu image on the SD card to be used on Raspberry Pi. Download the Raspberry Pi Imager or Balena Etcher tool. The same can be used to set up the required build environment for the Matter SDK or building Matter reference apps for various platforms.

- **RCP dongle**

If the DUT supports thread transport, an RCP dongle provisioned with a recommended RCP firmware for the default OTBR router that comes with the TH will be required to function properly. Currently, the OTBR can work with a Nordic RCP dongle or a SiLabs RCP dongle. Refer to Section 6, OT Border Router (OTBR) Setup on how to install the RCP firmware.

4.1.2. TH Installation on Raspberry Pi

Starting with version v2.10 we have moved from distributing TH as an SD-Card image to publishing the TH Docker containers at Github Container Registry and pulling them at install time. By doing that the release process has been made much faster and less error-prone, while at the same time installation time has gone shorter.

1. Place the blank SD card into the user's system USB slot.
2. Flash the 'Ubuntu Server 24.04.x LTS (64-bit)' image using one of the following tools on the Mac/PC:
   - **Raspberry Pi Imager:** open Raspberry Pi Imager and select 'Ubuntu Server 24.04.x LTS (64-bit)' as the operating system (under *Other general-purpose OS* > *Ubuntu*).
   - **Balena Etcher:** download the Raspberry Pi image of 'Ubuntu Server 24.04.x LTS (64-bit)' from ubuntu.com/download/raspberry-pi, then open Balena Etcher, choose **Flash from file** and select the downloaded image. Skip the OS customization settings below (see the note).
3. If using Raspberry Pi Imager, edit the OS customization settings to:
   - username: ubuntu

The username must be 'ubuntu'. Changing the name may cause problems running TH.

   - password: raspberrypi
   - hostname: ubuntu
   - Make sure you have enabled the SSH service.

The username/password/hostname/SSH customization above is done through Raspberry Pi Imager's OS customization settings, and enabling SSH with password authentication there is enough. Balena Etcher only writes the raw image and has no customization step. An image flashed with Etcher uses the defaults from the `user-data` cloud-init file on the boot partition:

username `ubuntu`, password `ubuntu` (you must change it at first login), and `ssh_pwauth: false`, so SSH password login is disabled. You can still log in on the Pi's own display and keyboard. To allow SSH with a password, set `ssh_pwauth: true` in `user-data` on the boot partition before the first boot.

4. After the SD card has been flashed, remove the SD card and place it in the Raspberry Pi's memory card slot.
5. Power on the Raspberry Pi and ensure that the local area network, display monitor and keyboard are connected.
6. Enter the username and password.
7. Install the TH system:
   - Clone the TH repository:
     - `$git clone -b <Target_Branch/Tag> https://github.com/project-chip/certification-tool.git --recurse-submodules`
   - Goto to TH folder:
     - `$cd certification-tool`
   - Install/configure the TH dependencies:
     - `$./scripts/pi-setup/auto-install.sh`
     - At the end of the script, select option 1 to restart the RaspberryPi.

The auto-install script also installs the Test-Harness CLI (`th-cli`), so no separate CLI installation is required. See Section 11, Matter Test-Harness Command Line Interface (CLI) for how to use it.

8. Wait about 10 minutes.
9. Using the *ifconfig* command, obtain the IP address of the Raspberry Pi. The same IP address will be used to launch the TH user interface on the user's system using the browser.
10. Proceed with test configuration and execution (refer to Section 8, Test Configuration and Section 9, Test Case Execution respectively).

4.2. TH installation without a Raspberry Pi

The official installation method uses a Raspberry Pi (TH Installation on Raspberry Pi). **This alternative installation method is targeted for development purpose and it only supports onnetwork pairing mode.**

To install TH without using a Raspberry Pi you'll need a machine with Ubuntu Server 24.04.x LTS (64-bit). You can create a virtual machine for this purpose, but **be aware that if the host's architecture is not arm64** you'll need to substitute `backend`, `frontend` and the SDK's docker image in order for it to work properly.

Images for linux/amd64 will not always be available in the github registry. So, if

necessary, the images need to be built locally using the following script: `./certification-tool/scripts/build.sh`

4.2.1. Create an Ubuntu virtual machine

Here's an example of how to create a virtual machine for TH using multipass (https://multipass.run/).

Please make sure the docker images are compatible with the host architecture.

- Install multipass

```
brew install multipass
```

- Create new VM with Ubuntu Server 24.04.x LTS (64-bit) (2 cpu cores, 8G mem and a 50G disk)

```
multipass launch 24.04.x -n matter-vm -c 2 -m 8G -d 50G
```

- SSH into VM

```
multipass shell matter-vm
```

About Multipass:
Seems like bridged network is not available, so you will not be able to test with DUT outside the docker container, but you can develop using the sample apps on the platform.

4.2.2. Setup TH in Ubuntu

- Clone git repo

```
git clone -b <Target_Branch/Tag> https://github.com/project-chip/certification-tool.git
--recurse-submodules
```

- Go into the repo directory

```
cd certification-tool
```

- Run TH auto install script

```
./scripts/ubuntu/auto-install.sh
```

This script also installs the Test-Harness CLI (`th-cli`). See Section 11, Matter Test-Harness Command Line Interface (CLI).

- Reboot VM

If using multipass, to find the IP address use the command

```
multipass list
```

4.2.3. Substitute the SDK's docker image and update sample apps

If the platform of the machine that will run the TH is 'linux/arm64' it will not be necessary to build a new SDK docker image.

To run TH on a machine using the 'linux/amd64' platform, you will need to first build a new SDK docker image.

- Get the SDK commit SHA

Value for variable `SDK_DOCKER_TAG` in TH repository path `certification-tool/backend/test_collections/matter/config.py`

- Download the Dockerfile for chip-cert-bins from the commit you need

Substitute <COMMIT_SHA> with the value from `SDK_DOCKER_TAG`:
`github.com/project-chip/connectedhomeip/blob/<COMMIT_SHA>/integrations/docker/images/chip-cert-bins/Dockerfile`

- Copy Docker file to TH's machine
- Make sure that no other SDK image for that commit SHA is loaded in the machine

Run `docker images`
If there's an image with a tag for the commit you're using, delete that image
`docker image rm <IMAGE_ID>`

- Build new SDK image (this could take about 3 hours)

Substitute <COMMIT_SHA> with the value from `SDK_DOCKER_TAG`:
`docker buildx build --load --build-arg COMMITHASH=<COMMIT_SHA> --tag connectedhomeip/chip-cert-bins:<COMMIT_SHA> .`

- Update TH sample apps

To update your sample apps using the new image run this script in the certification-tool repository
`./backend/test_collections/matter/scripts/update-sample-apps.sh`

4.3. Update Existing TH

If the Operating System is not the **Ubuntu Server 24.04.x LTS (64-bit)**, please flash and use a SD card with that Ubuntu release to use this version of Test Harness. Beware that the auto update process below will fail in the case of a different release version.

To update an existing TH environment, follow the instructions below on the terminal.

```
cd ~/certification-tool
./scripts/ubuntu/auto-update.sh <Target_Branch/Tag>
./scripts/start.sh
```

Wait for 10 mins and open the TH application using the browser

The auto-update script also reinstalls the Test-Harness CLI (`th-cli`), so the CLI does not need to be updated separately. The `./scripts/start.sh` script only starts the Test-Harness containers; it does not install or update the CLI.

4.4. Updating Existing Yaml Test Script

It is possible to update yaml test script content by directly editing the file content. It is useful when validating small changes or fixing misspelled commands.

Yaml files are located at:

```
~/certification-
tool/backend/test_collections/matter/sdk_tests/sdk_checkout/yaml_tests/yaml/sdk/
```

To update an existing Yaml test script: (e.g. `Test_TC_ACE_1_1.yaml`)

- Open the script file:

```
~/certification-
tool/backend/test_collections/matter/sdk_tests/sdk_checkout/yaml_tests/yaml/sdk/Test_TC_ACE_1_
1.yaml
```

- Update/change the desired information.
- Save and close the file.
- Restart TH's backend container:

```
$docker restart certification-tool-backend-1
```

- Changes will be available on the next execution of the yaml test.

To create a new Yaml test script:

- Use an existing test script as a starting point.
- Rename the file to a new one: e.g. `Test_TC_ACE_1_1.yaml` to `Test_TC_ACE_9_9.yaml`
- Update the name entry inside the yaml file:

FROM name: 42.1.1. `[TC-ACE-1.1]` Privileges

TO name: 42.1.1. `[TC-ACE-9.9]` Privileges

- Proceed as explained on updating an existent yaml file.

4.5. Customized Test Scripts (Yaml/Python Tests)

To use customized tests, the files must be placed in the specific folder (described below). This way, Test-Harness will load and display the available tests on the interface. These tests will not be affected if the system is restarted or if the SDK Yaml tests are updated.

Custom Yaml files folder are located at:

```
~/certification-tool/backend/test_collections/matter/sdk_tests/sdk_checkout/yaml_tests/yaml/custom/
```

Custom Python files folder are located at:

```
~/certification-tool/backend/test_collections/matter/sdk_tests/sdk_checkout/python_testing/scripts/custom/
```

Use the Custom Folders Symbolic Links:

At `~/certification-tool/backend/`, you may use the following symbolic links as shortcuts:
* `custom_yaml`: to access the Custom Yaml files folder.
* `custom_python`: to access the Custom Python files folder.

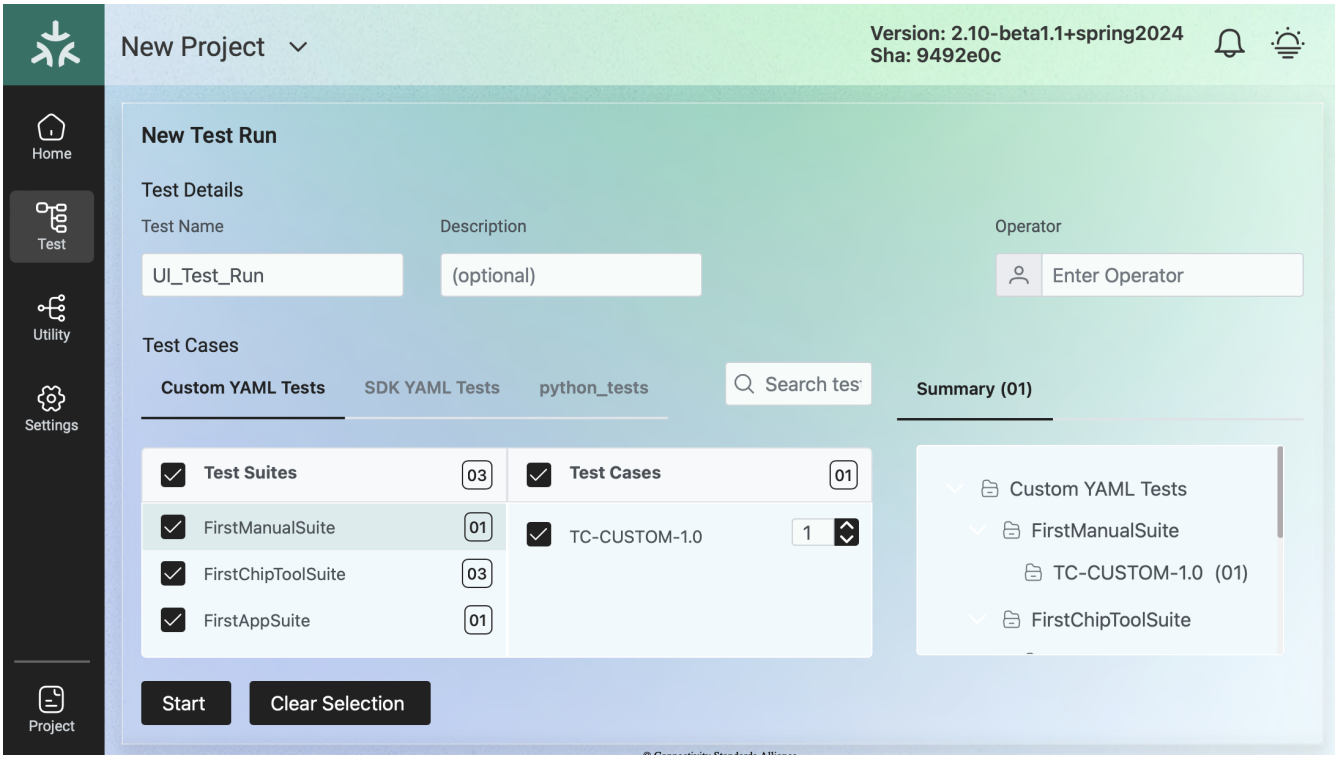


Figure 4. Test-Harness displaying the custom tests.

Experimenting with Test Cases:

You can copy the original SDK Yaml/Python test to Custom Yaml/Python folder and do any changes on it.

4.5.1. Test-Harness Side Load Feature with Custom Test Cases

For the Side Load feature, the user may benefit from the **custom** Test Cases feature mentioned above. To Side Load any desired script, follow the steps below:

1. Download the latest Test Case script from the SDK's master branch (connectedhomeip repository)
2. Place the desired script on the appropriate **custom** folder (depending if the script is Yaml or Python)
3. Rescan the Test-Harness so it picks up the new/updated script, using either the CLI or the API (see Rescanning Side-Loaded Test Cases below) — a full backend restart is **not** required
4. Change the Project's configuration as required to run the Side Loaded script (e.g. updating `test_parameters`)
5. The Side Loaded script will be available on the Test Cases list in the Custom tab (refer to the image above)
6. When executing side-loaded test cases using the CLI, it is mandatory to append the -custom suffix to the Test Case ID. This suffix ensures that the Test-Harness correctly identifies and executes the script from the custom directory. Test cases invoked without this suffix will not be recognized as side-loaded and may fail to execute as expected.

   ```
    Example Command:

     th-cli run-tests --tests-list TC-JFADMIN-1.1-custom --project-id <id> --config
   my_config.json --title <Name of the test run execution>  --pics-config-folder
   ./pics_files/
   ```

   This requirement applies to all test cases placed in the **custom** YAML or Python directories.

4.5.2. Rescanning Side-Loaded Test Cases

After adding a new custom test script or editing an existing one, the Test-Harness needs to re-run test discovery so the change is picked up. This can be done in-process, without restarting the backend container:

```
th-cli rescan-tests
```

This calls the `POST /api/v1/test_collections/rescan` backend endpoint, which re-generates the Python test JSON files and re-discovers all test collections, then returns the refreshed list of test collections. The new or updated script is immediately available on the Test Cases list — no `stop.sh` /`start.sh` cycle is needed, and the backend remains available to other users throughout.

The command reports what the rescan changed, grouped by test collection, so you can tell whether your change was picked up:

```
Rescanned test collections successfully (6 found)
```

```
Custom SDK Python Tests
  + TC-FAKE-1.1-custom           added
  ~ TC-ACE-1.3-custom            modified
  - TC-OLD-1.1-custom            removed
No changes in: SDK Python Tests, SDK YAML Tests
```

When nothing changed, the command says so explicitly:

```
Rescanned test collections successfully (6 found). No changes detected.
```

A test case is reported as **modified** when its discovered definition changes: its title or description, its PICS, or (for Python tests) its steps, class name or path. Edits that do not change what discovery reads, such as changes inside a test step's code, are not reported for the test case, but the file shows up in the list of changed **custom** files below.

Test Engine must be idle:

The rescan cannot run while a test is executing. If a test run is in progress, the command returns an error (`Test Engine is busy.`) instead of rescanning; wait for the test run to finish, or abort it with `th-cli abort-testing`, and try again.

Invalid or malformed scripts:

If a newly added/edited script cannot be parsed, `rescan-tests` skips it and lists it, with the reason, after the changes:

```
Scripts that could not be parsed (skipped):
  ! TC_FAKE_1_1.py: Syntax error: invalid syntax (line 12)
```

The rest of the rescan still completes. Fix the script and run `th-cli rescan-tests` again — the backend does not need to be restarted to recover.

Files changed in the custom folder:

Besides the test cases, `rescan-tests` lists the Python files in the **custom** folder that were added, modified or removed since the previous rescan (or since the backend started). This covers files that are not test cases, such as helper modules, and edits that do not change what discovery reads:

```
Custom files
  + TC_SUTestBase.py             added (replaces sdk/TC_SUTestBase.py)
  ~ TC_ACE_1_3.py                modified
  - TC_OLD_1_1.py                removed
```

`(replaces sdk/<file>)` marks a file that has the same name as a file in the **sdk** folder and is imported by another script, so every test importing it uses the

custom copy (see the **Side Loading Helper/Base Modules** section). A same-name copy of an SDK test is not imported by anything and does not get the note. Unchanged files are not listed, and nothing is printed when no file changed.

Full backend restart still works:

The previous approach of restarting the backend container (e.g. `./scripts/stop.sh` followed by `./scripts/start.sh`) also picks up side-loaded script changes, but is more disruptive since it drops any in-progress state and takes the service down for everyone using it. Prefer `th-cli rescan-tests` unless a full restart is otherwise needed.

4.5.3. Running a Side Loaded Python Script Directly Inside the Container

A side-loaded Python script placed in the **custom** Python folder (`…/sdk_checkout/python_testing/scripts/custom/`) may import shared SDK helper modules (e.g. `TC_GC_common.py`, `TC_TLSCERT_Base.py`) that only exist in the sibling **sdk** folder (`…/sdk_checkout/python_testing/scripts/sdk/`).

When the Test-Harness executes a side-loaded script through the normal `run-tests` flow (CLI or UI), it automatically makes the **sdk** folder's modules available to the script, so these imports resolve without any extra steps.

To run the script directly inside the container (e.g. `docker exec` into the backend container and invoke it manually for debugging), the **sdk** scripts folder must be added to `PYTHONPATH` so those imports can be found:

```
cd /root/python_testing/scripts/custom
PYTHONPATH=/root/python_testing/scripts/sdk python3 <script_name>.py \
  --commissioning-method on-network --discriminator <value> --passcode <value> \
  --storage-path admin_storage.json --endpoint <value>
```

ModuleNotFoundError when running without PYTHONPATH:

Running the script directly (`python3 <script_name>.py`) from inside the **custom** folder without setting `PYTHONPATH` will fail with `ModuleNotFoundError` for any import that only exists in the **sdk** folder. This only affects manual invocation inside the container — it does not affect test execution through the Test-Harness CLI or UI.

4.5.4. Side Loading Helper/Base Modules (e.g. `TC_SUTestBase.py`)

Besides test scripts (`TC_*.py`), the **custom** Python folder may also hold helper or base modules that a side-loaded test imports with a flat import (e.g. `from TC_SUTestBase import …`). When the Test-Harness runs a test, it resolves such imports in this order:

1. the **custom** folder (`…/python_testing/scripts/custom/`)
2. the **sdk** folder (`…/python_testing/scripts/sdk/`)

A helper placed in the **custom** folder therefore **shadows** the SDK module with the same file name. To use a modified helper, copy the SDK file into the **custom** folder, edit the copy and run `th-cli rescan-tests`. Do not edit files in the **sdk** folder.

The shadowing is not limited to side-loaded tests. A file in the **custom** folder is also used by the SDK tests that import a module with that name, so an SDK helper can be patched without side-loading every test that uses it. Test discovery follows the same order.

A copy left in the custom folder keeps applying:

The override is based on the file name alone. A copy of an SDK helper left in the **custom** folder is used by **every** test that imports it, side-loaded or not, and it does not follow SDK changes after a Test-Harness update. Remove the copy as soon as it is no longer needed.

To see what a test run used, check the test log: for each module loaded from the **custom** folder (other than the side-loaded test itself) the Test-Harness prints a line such as:

```
SIDELOAD: TC_SUTestBase loaded from custom/TC_SUTestBase.py, overriding
sdk/TC_SUTestBase.py
SIDELOAD: MyHelper loaded from custom/MyHelper.py (no sdk/ counterpart)
```

No `SIDELOAD:` line means the run used no module from the **custom** folder.

To see what changed in the **custom** folder, `th-cli rescan-tests` lists the added, modified and removed files and marks the ones that replace an SDK module (see Rescanning Side-Loaded Test Cases). You can also compare the two folders:

```
comm -12 <(ls custom | grep '\.py$' | sort) <(ls sdk | grep '\.py$' | sort)
```

(run it from `…/python_testing/scripts/`). A side-loaded copy of an SDK test, for example `custom/TC_ACE_1_3.py`, appears in this list as well. That is expected, since tests are not imported by other modules.

Only plain `import` / `from … import` of modules in these two folders is affected. Packages installed with the SDK (e.g. `matter.testing…`) are not overridden.

4.6. Troubleshooting

4.6.1. Read-Only File System Error

- During the execution of TH installation commands if a read-only file system error or an error showing "Is docker daemon running?" occurs, follow the steps below to fix the issue:

```
$sudo fsck ( Press 'y' for fixing all the errors )
```

- Upon successful completion, try the following commands:

```
$sudo reboot
ssh back into the TH IP address using:
$ssh ubuntu@<IPADDRESS-OF-THE-RASPI>
```

- In case "sudo fsck" fails, use the following commands:

```
sudo fsck -y -f /dev/mmcblk0p2
fsck -y /dev/mmcblk0p2
```

- In case the "remote: Repository not found" fatal error occurs, try the following steps to fix the issue. Clone the certification-tool with personal access token (Refer to Section 4.2.2, Generate Personal Access Token to generate the personal access token) and follow the steps below.

```
cd ~


Take the backup of Test Harness binary using below command:
$mv certification-tool certification-tool-backup
$git clone https://<token>@github.com/project-chip/certification-tool.git

Follow the instructions given in the section below on how to update an existing Test-Harness
```

4.6.2. Generate Personal Access Token

The Personal Access Token may be required during the process of updating an existing TH. Below are the instructions to obtain the personal access token.

1. Connect to the Github account (the one recognized and authorized by Matter).
2. On the upper-right corner of the page, click on the profile photo, then click on **Settings**.
3. On the left sidebar, click on **Developer settings**.
4. On the left sidebar, click on **Personal access tokens** [Personal access tokens (classic)].
5. Click on **Generate new token** .
6. Provide a descriptive name for the token.
7. Enter an expiration date, in days or using the calendar.
8. Select the scopes or permissions to grant this token.
9. Click on **Generate new token** .
10. The generated token will be printed out on the screen. Make sure to save it as a local copy as it will disappear.

Sample token: ghp_hUQExoppLKma**********Urg4P

4.6.3. Bringing Up of Docker Containers Manually

During the initial reboot of the Raspberry Pi, if the docker is not initiated automatically, try the

following command on the Raspberry Pi terminal to bring up the dockers.

Use the command `ssh ubuntu@IP_address` from the PC to log in to Raspberry Pi. Refer to previous sections on how to obtain the IP address of Raspberry Pi.

Once the SSH connection is successful, start the docker container using the command
`$ ./certification-tool/scripts/start.sh`

The above command might take a while to get executed, wait for 5-10 minutes and then proceed with the Test Execution Steps as outlined in the below sections.

4.6.4. Cleaning The Environment Manually

If the Test-Harness environment is facing issues to install, update or start and no other action is working, you may try the cleanup command followed by a install operation.

Please, be advised that this cleanup operation will delete all previous data from the TH database, along with all the docker networks, containers, images used by the application and more.

Follow the bellow procedure to clean and install Test-Harness:

Use the command `ssh ubuntu@IP_address` from the PC to log in to Raspberry Pi. Refer to previous sections on how to obtain the IP address of Raspberry Pi.

Once the SSH connection is successful, clean the environment using the command:
`$ ./certification-tool/scripts/clean-up.sh`

Finally, execute a new installation with the following command:
`$ ./certification-tool/scripts/pi-setup/auto-install.sh`

4.6.5. Test Harness Startup Hanging with ModuleNotFoundError

This issue typically occurs when the `./scripts/start.sh` script takes a long time to complete and appears to hang during startup.

4.6.5.1. Symptoms

- The `start.sh` script does not complete after a reasonable time
- Backend container fails to initialize properly

4.6.5.2. Diagnosis

To check if this is the issue, run the following command:

```
docker logs certification-tool-backend-1
```

Look for an error message similar to:

```
ModuleNotFoundError: No module named 'chipyaml'
```

4.6.5.3. Solution

If you find the `ModuleNotFoundError` in the logs, follow these steps:

```
Stop the Test-Harness:
$ ./scripts/stop.sh

Remove the SDK version file:
$ rm ~/certification-tool/backend/test_collections/matter/sdk_tests/sdk_checkout/.version

Start the Test-Harness again:
$ ./scripts/start.sh
```

This will force the SDK dependencies to be reinstalled during the next startup.

4.6.6. No Link-Local IPv6 Address Assigned to the Wi-Fi Fixture Interface

The network the Wi-Fi Fixture brings up has no DHCP server, so a DUT that joins it is reachable only at an IPv6 link-local address. This requires the host to have a link-local address of its own on the fixture's interface, which the kernel assigns as the interface comes up unless NetworkManager has taken the interface over.

4.6.6.1. Symptoms

A test using the Wi-Fi Fixture fails as it brings up its access point, with a message naming the interface the fixture was given:

```
No link-local IPv6 address assigned to wlx6ccdd6c589f0 after 5 seconds
```

4.6.6.2. Diagnosis

Check whether NetworkManager is running and holding that interface:

```
$ systemctl is-active NetworkManager

$ nmcli device status
```

An interface NetworkManager has claimed but not activated is listed as `disconnected` rather than `unmanaged`, and has IPv6 switched off for as long as it is in that state.

4.6.6.3. Solution

The Test-Harness installation scripts write a udev rule that leaves USB Wi-Fi adapters to the Test-Harness, so an installation set up by `auto-install.sh` or brought up to date by `auto-update.sh` is already configured correctly. An older installation can be configured by running that step on its own:

```
$ ./scripts/ubuntu/2.1-configure-wifi-devices.sh
```

The rule applies to an adapter as it appears, so re-plug an adapter that is already connected, or reboot the host.

Note that NetworkManager is not required on a Test-Harness host and is not installed by the installation scripts, which configure `wpa_supplicant` directly for the host's own Wi-Fi interface.

5. Bringing Up of Matter Node (DUT) for Certification Testing

A Matter node can either be a commissioner, controller, controlee, software component or an application. The Matter SDK comes with a few example apps that can be used by Vendors as a reference to build their products. Refer to the examples folder in the SDK github repo for the same.

DUT vendors need to get the device flashed with the production firmware revision that they want to get their device certified and execute all the applicable TC's for their products using the TH. DUT vendors can skip the below sections as the TH brings up the reference applications automatically during the certification tests execution.

A hobby developer can build Matter reference apps either using a Raspberry Pi or Nordic DK board (if the user wants to use thread transport). Follow the instructions below for the Raspberry Pi and Nordic platforms.

5.1. Bringing Up of Reference Matter Node (DUT) on Raspberry Pi

In the case where a device maker/hobby developer needs to bring up a sample/reference DUT, i.e. light bulb, door lock, etc. using the example apps provided in SDK and verify provisioning of the DUT over the Bluetooth LE, Wi-Fi and Ethernet interfaces, follow the below steps to set up the DUT.

Users can either use the example apps (i.e. light bulb, door lock, etc.) that are shipped with the TH OR build the apps from the latest SDK source.

To use the apps that are shipped with the TH, follow the instructions below:

- Do a fresh install of TH (Installation on Raspberry Pi).
- Go to the apps folder in /home/ubuntu/apps (as shown below) and launch the app that the user is interested in.

To build the example apps from the latest SDK source, follow the instructions below:

- User to acquire Raspberry Pi Version (4 or 5) with SD card of minimum 64 GB memory.
- Do a fresh install of the Ubuntu Server 24.04.x LTS (64-bit) image and install all the required dependencies as outlined in https://github.com/project-chip/connectedhomeip/blob/master/docs/guides/BUILDING.md.
- Clone the connected home SDK repo using the following commands:

```
$ git clone git@github.com:project-chip/connectedhomeip.git --recursive
$ cd connectedhomeip
$ source scripts/bootstrap.sh
$ source scripts/activate.sh
```

- Select the sample app that the user wants to build as available in the examples folder of the SDK repo e.g., lighting-app, all-cluster-app. The user needs to build these apps for the Linux platform using the following command:

Build the app using the below command:

```
./scripts/examples/gn_build_example.sh examples/all-clusters-app/linux/examples/all-clusters-app/linux/out/all-clusters-app chip_inet_config_enable_ipv4=false
```

5.1.1. To Provision Raspberry Pi Using Wi-Fi Configuration

The sample app (lighting-app or lock-app or all-cluster-app) can be provisioned over the Wi-Fi network when the app is launched with the "--wifi" argument.

```
./chip-all-clusters-app --wifi
```

5.1.2. To Provision Raspberry Pi Over Ethernet Configuration

The sample app (lighting-app or lock-app or all-cluster-app) can be provisioned over the Ethernet (using onnetwork configuration) that it is connected when the app is launched with no arguments.

```
./chip-all-clusters-app
```

5.2. Bringing Up of Reference Matter Node (DUT) on Thread Platform

Follow the instructions below to set up the Matter Node on Thread Platform. For additional reference, go to the following link:

https://github.com/project-chip/connectedhomeip/tree/master/examples/all-clusters-app/nrfconnect#matter-nrf-connect-all-clusters-example-application

5.2.1. Prerequisites

The following devices are required for a stable and full Thread Setup:

- **DUT:** nRF52840-DK board and one nRF52840-Dongle

The DUT nRF52840-DK board mentioned in this manual is used for illustration purposes only. If the user has a different DUT, they will need to configure the DUT following the DUT requirements.

5.2.2. Setting Up Thread Board (nRF52840-DK)

To set up the Thread Board, follow the instructions below.

> *The nRF52840-DK setup can be performed in two methods either by flashing the pre-built binary hex of sample apps which is released along with the TH by using the nRF Connect Desktop application tool (refer Section 5.2.2.1) or by building the docker environment to build the sample apps (refer Section 5.2.2.2).*

5.2.2.1. Instructions to Set Up nRF52840-DK Using nRF Connect Desktop Application Tool

a. Requirements:

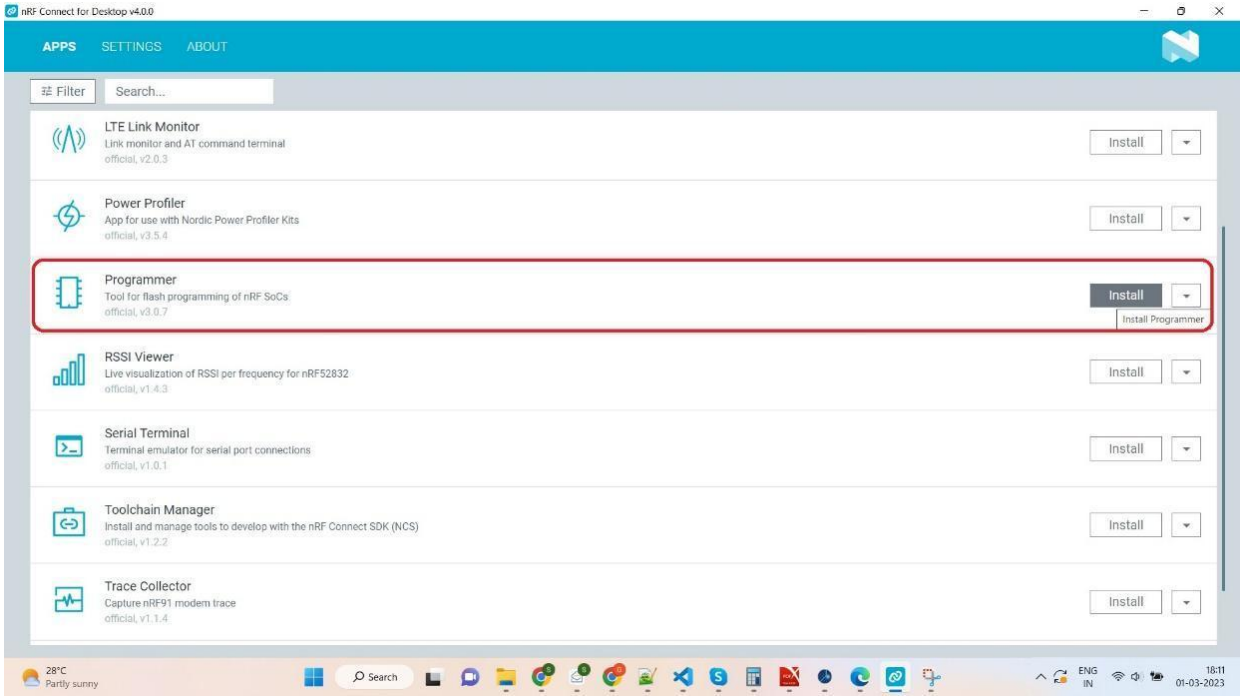
1. nRF Connect for Desktop tool installer: Link

> *The J-Link driver needs to be separately installed on macOS and Linux. Download and install it from SEGGER under the section J-Link Software and Documentation Pack.*

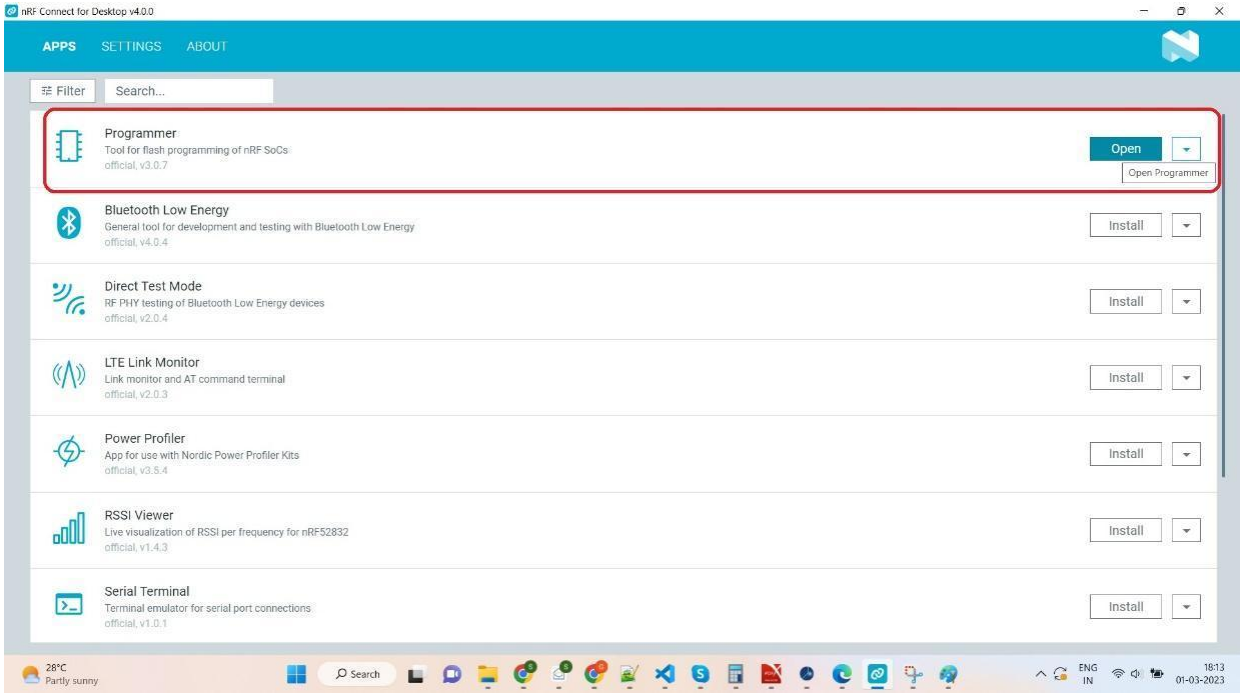
2. Download thread binary files which are released along with the TH.

b. From the User Interface:

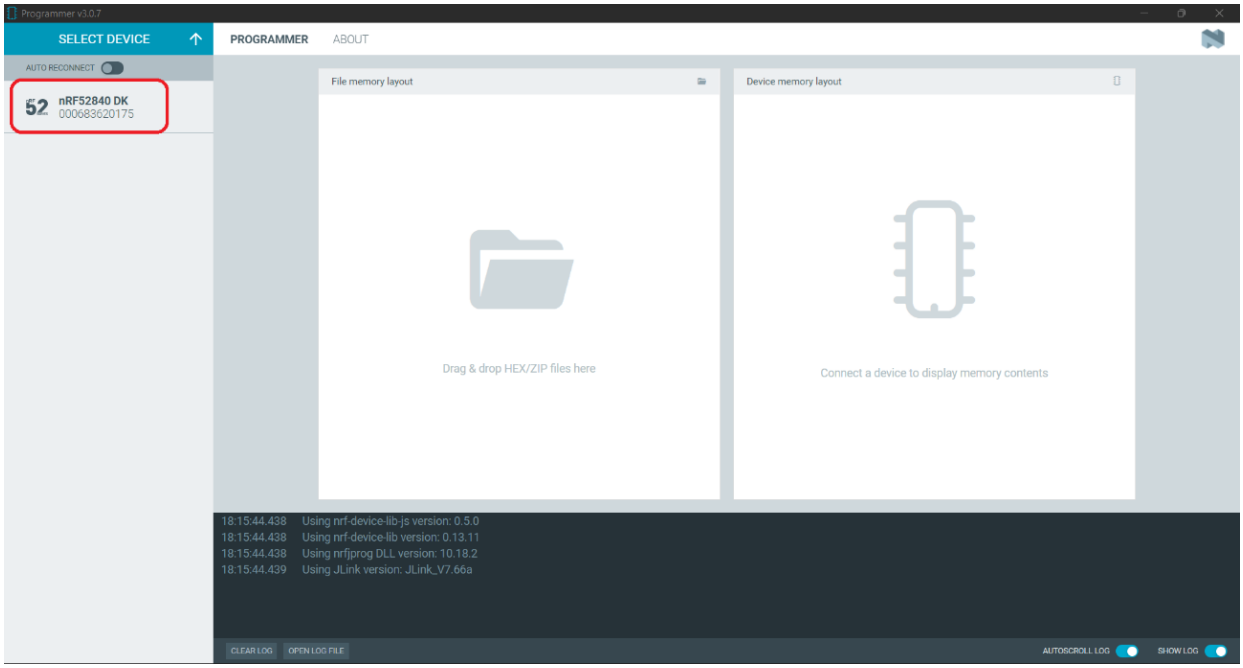
1. Connect nRF52840-DK to the USB port of the user's operating system.
2. From the nRF Connect for Desktop tool, install **Programmer** from the apps tab.



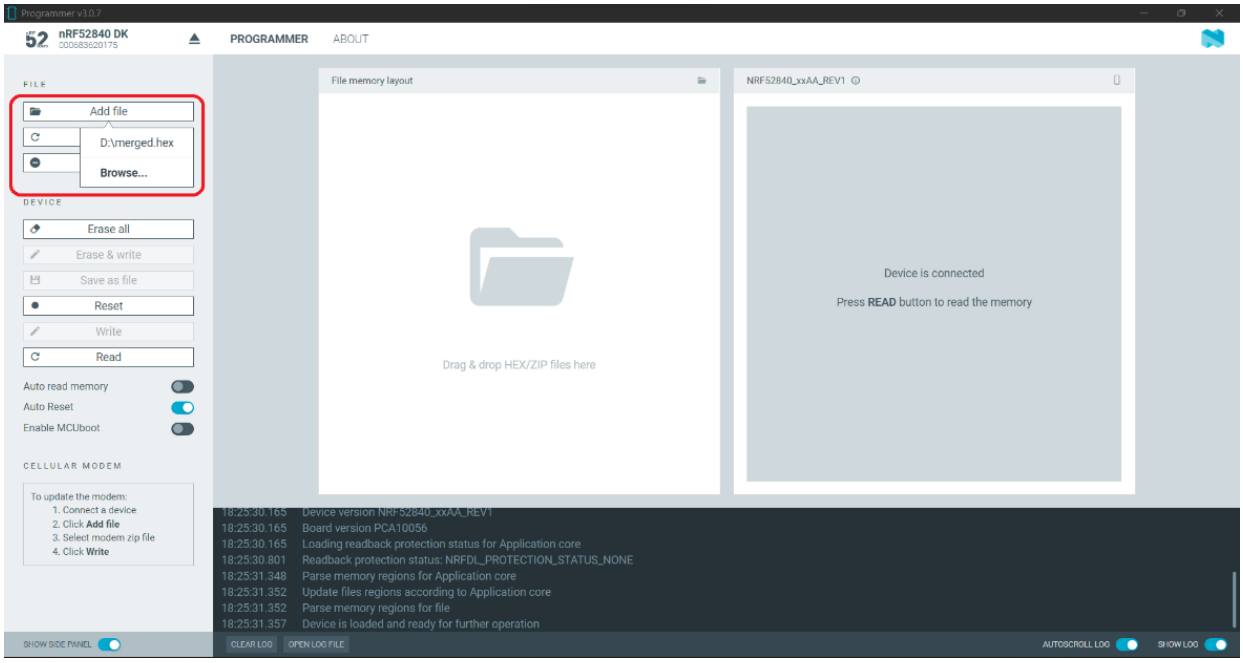
3. Open the Programmer tool to flash the downloaded binary hex file on nRF52840-DK.



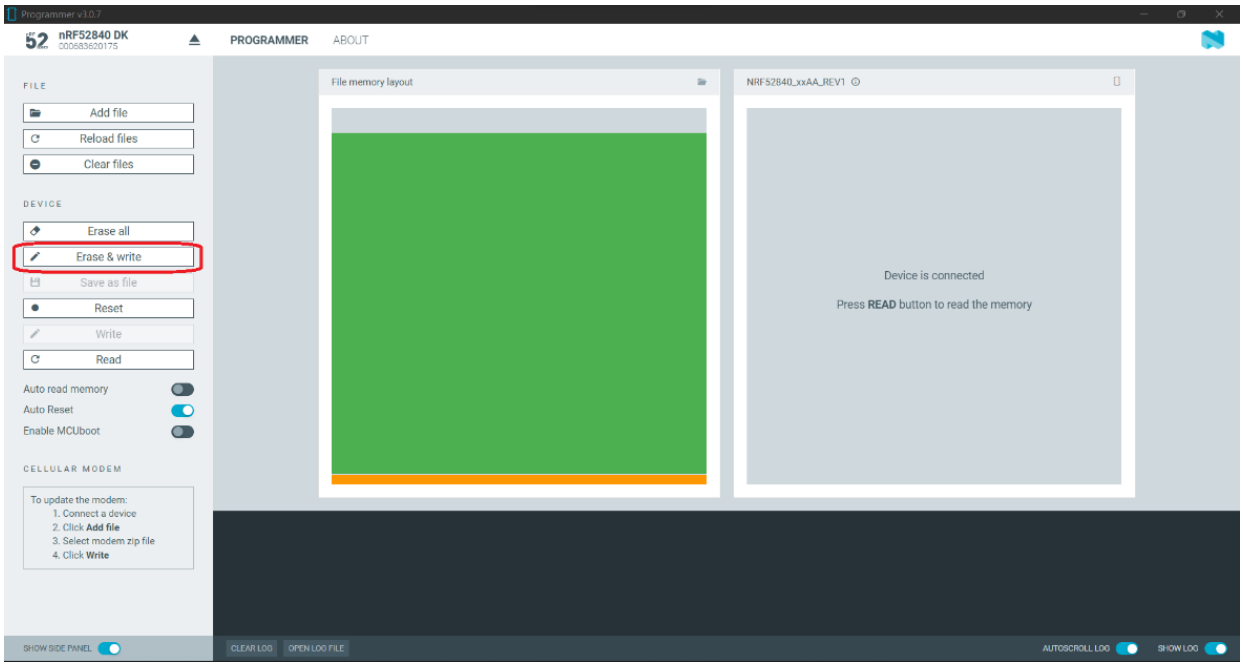
4. In the Programmer tool, select the device name from the **SELECT DEVICE** drop-down list.



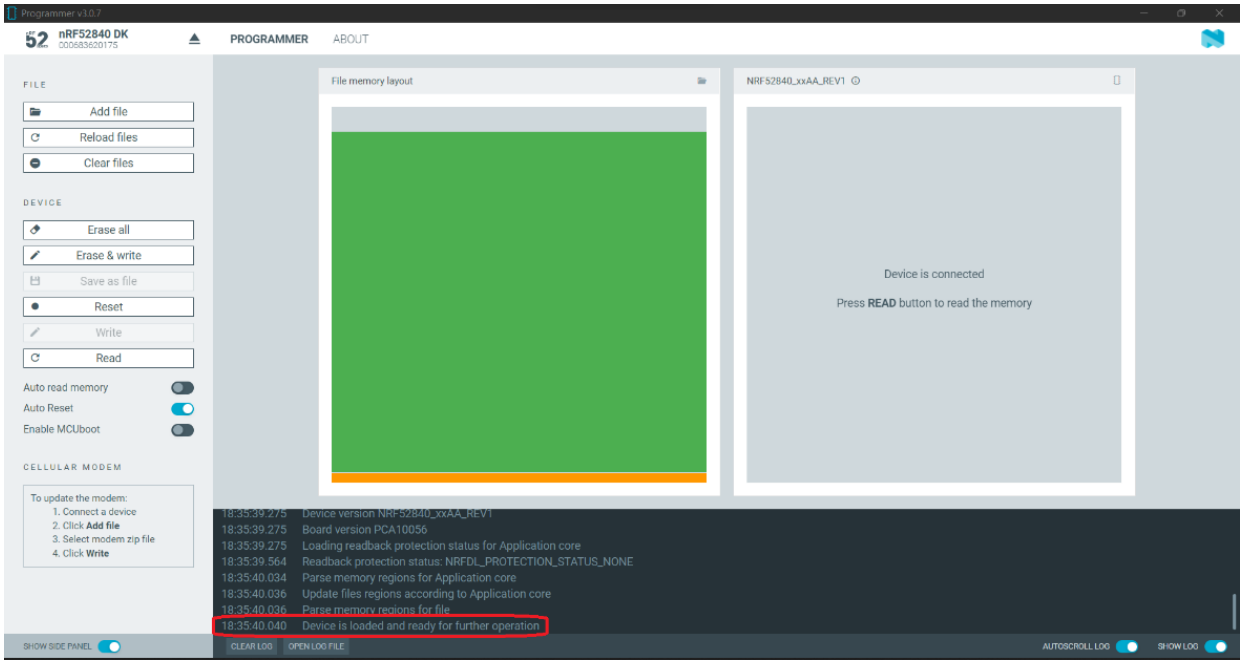
5. Select **Add file** and browse the downloaded file to upload the desired sample app hex file.



6. Select **Erase & write** to flash the hex file on the device.



7. Check the log for successful flash.



8. Connect the nRF52840-Dongle to the USB port of the Raspberry Pi having the latest TH.
9. For the Thread DUT, enable discoverable over Bluetooth LE (e.g., on nRF52840 DK: select Button 4) and start the Thread Setup Test execution by referring to Section 8, Test Configuration .

5.2.2.2. Instructions to Set Up nRF52840-DK Using Docker Environment

1. To build the sample apps for nRF-Connect, check out the Matter repository and bootstrap using following commands:

```
git clone https://github.com/project-chip/connectedhomeip.git
cd ~/connectedhomeip/
source scripts/bootstrap.sh
cd ~/connectedhomeip/
source scripts/activate.sh
```

2. If the nRF-Connect SDK is not installed, create a directory running the following command:

```
$ mkdir ~/nrfconnect
```

3. Download the latest version of the nRF-Connect SDK Docker image by running the following command:

```
$ sudo docker pull nordicsemi/nrfconnect-chip
```

4. Start Docker using the downloaded image by running the following command:

```
sudo docker run --rm -it -e RUNAS=$(id -u) -v ~/nrfconnect:/var/ncs -v
~/connectedhomeip:/var/chip -v /dev/bus/usb:/dev/bus/usb --device-cgroup-rule "c 189:*
rmw" nordicsemi/nrfconnect-chip
```

5. The following commands can be executed to change the settings if required:

~/**nrfconnect** can be replaced with an absolute path to the nRF-Connect SDK source directory.
~/**connectedhomeip** can be replaced with an absolute path to the CHIP source directory.

-v /dev/bus/usb:/dev/bus/usb --device-cgroup-rule "c 189: rmw"*

Parameters can be omitted if flashing the example app onto the hardware is not required. This parameter gives the container access to USB devices connected to your computer such as the nRF52840 DK.

--rm can be omitted if you do not want the container to be auto-removed when you exit the container shell session.
-e RUNAS=$(id -u) is needed to start the container session as the current user instead of root.

6. Update the nRF-Connect SDK to the most recent supported revision, by running the following command:

```
$ cd /var/chip
$ python3 scripts/setup/nrfconnect/update_ncs.py --update
```

5.2.2.3. Building and Flashing Sample Apps for nRF-Connect

Perform the following procedure, regardless of the method used for setting up the environment:

1. Navigate to the example directory:

```
$ cd examples/all-clusters-app/nrfconnect
```

2. Before building, remove all build artifacts by running the following command:

```
$ rm -r build
```

3. Run the following command to build the example, with ***build-target*** replaced with the build target name of the Nordic Semiconductor's kit, for example, nrf52840dk_nrf52840:

```
$ west build -b <build-target> --pristine always -- -DCONFIG_CHIP_LIB_SHELL=y
```

Target Name	Compatible Kit
nRF52840 DK	nrf52840dk_nrf52840
nRF5340 DK	nrf5340dk_nrf5340_cpuapp
nRF52840 Dongle	nrf52840dongle_nrf52840
nRF7002 DK	nrf7002dk_nrf5340_cpuapp

4. To flash the application to the device, use the west tool and run the following command from the example directory:

```
$ west flash --erase
```

5. Connect the nRF52840-Dongle to the USB port of the Raspberry Pi having the latest TH.
6. For the Thread DUT, enable discoverable over Bluetooth LE (e.g., On nRF52840 DK: Press Button 4) and start the Thread Setup Test execution by referring to Section 8, Test Configuration.

6. Bringing up the Matter Python REPL

The Matter Python REPL, also known as `chip-repl`, is a native IPython shell environment loaded with a Python-wrapped version of the C++ Matter stack to permit interacting as a controller to other Matter-compliant devices.

You can use the `chip-cert-bins` SDK image to run `chip-repl` on your Test Harness by follwing these instructions:

- Start container:
  Remember to set `PATH_TO_PAA_ROOTS` and substitute `<SDK SHA RECOMMENDED>`

```
docker run -v $PATH_TO_PAA_ROOTS:/paa_roots -v
/var/run/dbus/system_bus_socket:/var/run/dbus/system_bus_socket -v /home/ubuntu/certification-
tool/backend/test_collections/matter/sdk_tests/sdk_checkout/python_testing:/root/python_testin
g -v $(pwd):/launch_dir --privileged --network host -it connectedhomeip/chip-cert-bins:<SDK
SHA RECOMMENDED>
```

- Activate python environment:

```
source python_env/bin/activate
```

- Run matter-repl:

```
python3 python_env/bin/matter-repl
```

7. OT Border Router (OTBR) Setup

If the DUT supports Thread Transport, DUT vendors need to use the OTBR that is shipped with the TH for certification testing. Here are the instructions to set up OTBR that comes with the TH. Users need to get the RCP programmed with the recommended version and connect it to the Raspberry Pi running the TH. The OTBR will be started when the TH runs the thread transport related TC's.

Currently the OTBR in the TH works with either the Nordic RCP dongle or SiLabs RCP dongle. Refer to Section 7.1 to flash the NRF52840 firmware or Section 7.3 to flash the SiLabs firmware and get the RCP's ready. Once the RCP's are programmed, the user needs to insert the RCP dongle on to the Raspberry Pi running the TH and reboot the Raspberry Pi.

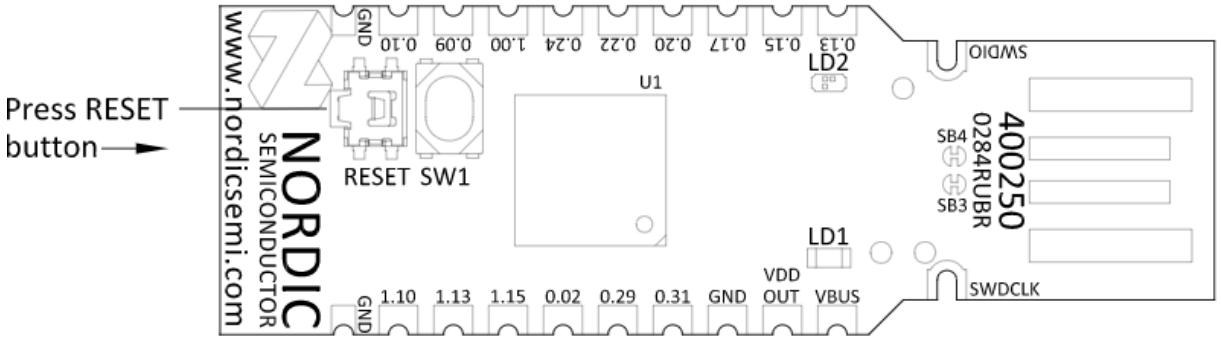
7.1. Instructions to Flash the Firmware NRF52840 RCPDongle

The RCP dongle does not need to be flashed on the Raspberry Pi running the TH. It can be flashed from any Mac or PC with a free USB port; once flashing completes, move the dongle over to the Raspberry Pi.

1. Download RCP firmware package from the following link on the user's system — Thread RCP Firmware Package
2. nRF Util is Nordic's unified command line utility for its products. It is distributed as a standalone, self-updating executable and is no longer installed via `pip`. For more details and the download for your OS, refer to the following link— https://www.nordicsemi.com/Products/Development-tools/nrf-util
3. Download and install nRF Util on the user's system. On Linux, this can be done using the following commands (on macOS or Windows, download the executable for your OS from the link above instead of the `curl` command below):

```
curl --location
https://files.nordicsemi.com/artifactory/swtools/external/nrfutil/executables/x86_6
4-unknown-linux-gnu/nrfutil -o nrfutil
chmod +x nrfutil
./nrfutil self-upgrade
./nrfutil install device
```

4. Connect the nRF52840 Dongle to the USB port of the user's system.
5. Press the Reset button on the dongle to enter the DFU mode (the red LED on the dongle starts blinking).



6. Run the following command to confirm the dongle is detected and to get its serial number:

```
nrfutil device list
```

7. To install the RCP firmware package on to the dongle, run the following command from the path where the firmware package was downloaded, using the serial number obtained in the previous step:

```
nrfutil device program --firmware <FILE NAME> --serial-number <SERIAL NUMBER>
```

Example:

```
nrfutil device program --firmware nrf52840dongle_rcp_c084c62.zip --serial-number
001050231451
```

8. Once the flash is successful, the red LED turns off slowly.
9. Remove the Dongle from the user's system and connect it to the Raspberry Pi running TH.
10. In case any permission issue occurs during flashing, launch the terminal and retry in sudo mode.

7.2. Nrfconnect Sample APPs Firmwares to Flash on the NRF52840DK Kit

The Nrfconnect Sample apps binary Package is available for download and should be flashed in the development kit NRF52840DK to use it as DUT in the Test-Harness tests.

7.3. Instructions to Flash SiLabs RCP

Download the latest version of ot-rcp-binaries from the assets list of the latest release: Silicon Labs Matter GitHub

For detailed RCP firmware usage, refer to: https://www.silabs.com/documents/public/application-notes/an1256-using-sl-rcp-with-openthread-border-router.pdf

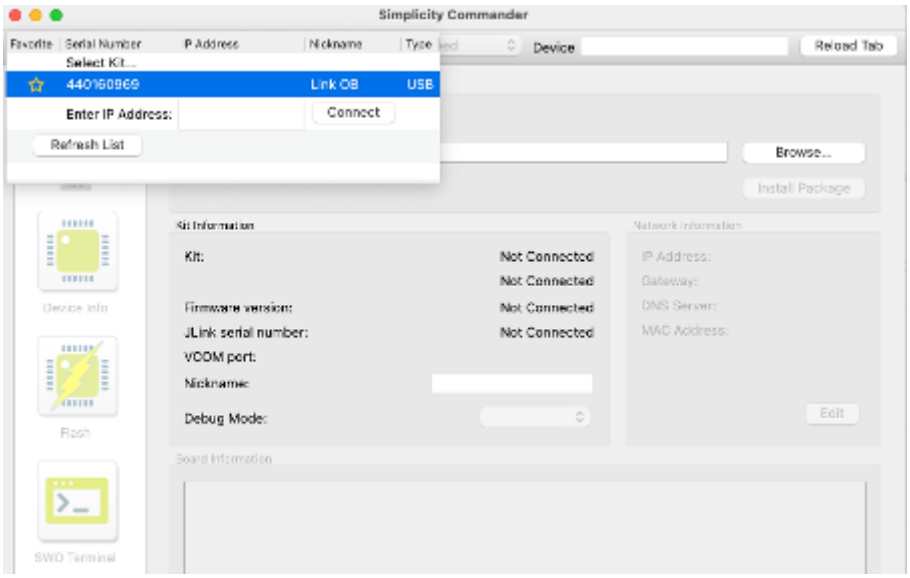
Requirements:

- SiLabs RCP: Thunderboard Sense 2 Sensor-to-Cloud Advanced IoT Kit or EFR32MG Wireless Starter Kit
- SiLabs RCP Firmware: see the ot-rcp-binaries download above

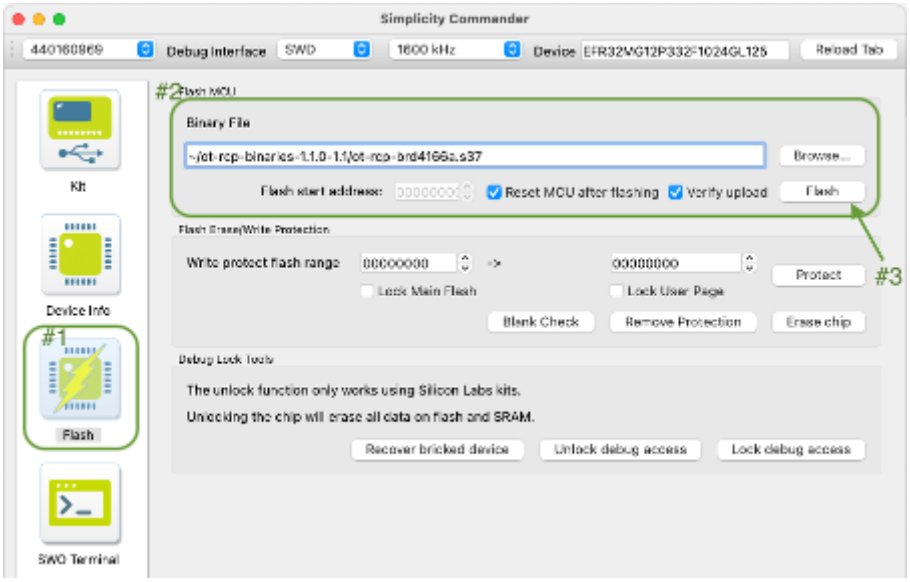
- Simplicity Commander: Installer for Windows, MAC or Linux

From UI:

- Connect the RCP dongle to the USB port of the user's operating system or via Ethernet.
- From the Simplicity Commander app, select and connect to RCP:
  - For USB connection, select the corresponding Serial Number from the drop-down list.
  - For Ethernet connection, enter the IP address of the RCP and click on **Connect** .



- To flash an image, go to "Flash", select the RCP binary file, and click on **Flash** .



From CLI:

- In case RCP is connected via Ethernet and the Simplicity Commander UI is not an option, the RCP image can be flashed using CLI.
- From path to Simplicity Commander:
  `commander flash <rcp-image-path> --ip <rcp-ip-address>`

7.4. Forming Thread Network and Generating Dataset for Thread Pairing

TH spins the OTBR docker image automatically when executing the thread related test cases. Follow the steps below if the user wants to start OTBR with custom parameters. The user needs to generate a dataset for the custom OTBR. To generate hexadecimal code required for manual Thread pairing procedure, use the instructions below.
ssh the Raspberry-Pi in the User System using the command "**ssh ubuntu@IP_address**"
Example output for the above command to generate the dataset value:

```
ubuntu@ubuntu:~$ ./certification-tool/backend/test_collections/matter/scripts/OTBR/otbr_start.sh
openthread/otbr@sha256:aa02bd1534984be704c84a575f97412c71fab504e6e1f094d0de4e06bfe46cf3 083c8472bc52 10 months ago 1.21GB
otbr image
openthread/otbr@sha256:aa02bd1534984be704c84a575f97412c71fab504e6e1f094d0de4e06bfe46cf3 already installed 54d868724cbb0c05c155983d5df5e9a3c1b61cbdafdf38eef2d8d1928f305a

waiting 10 seconds to give the docker container enough time to start up...
Param: 'dataset init new'
Done
Param: 'dataset channel 25'
Done
Param: 'dataset panid 0x5b35'
Done
Param: 'dataset extpanid 5b35dead5b35beef'
Done
Param: 'dataset networkname 5b35'
Done
Param: 'dataset networkkey 00112233445566778899aabbccddeeff'
Done
Param: 'dataset commit active'
Done
Param: 'prefix add fd11:35::/64 pasor'
Done
Param: 'ifconfig up'
Done
Param: 'thread start'
Done
Param: 'netdata register'
Done
Param: 'dataset active -x
0e080000000000010000000300001935060004001fffe002085b35dead5b35beef0708fd902fb12bca8af9051000112233445566778899aabbccddeeff0304356233501025b350410cdfe3b9ac95afd445e659161b03b3c4a0c0402a0f7f8
Done
Simple Dataset:
00030000l902085b35dead5b35beef05100011223344556677889 9aabbccddeeff01025b35
```

If any issue occurs while using **otbr_start.sh**, follow the steps below to generate the dataset value manually:

On Terminal 1:

1. Follow the steps below to build the OTBR docker:
   a. Create the docker network by executing the following commands:

```
sudo docker network create --ipv6 --subnet fd11:db8:1::/64 -o
com.docker.network.bridge.name=otbr0 otbr
sudo sysctl net.ipv6.conf.otbr0.accept_ra_rt_info_max_plen=128
sudo sysctl net.ipv6.conf.otbr0.accept_ra=2
```

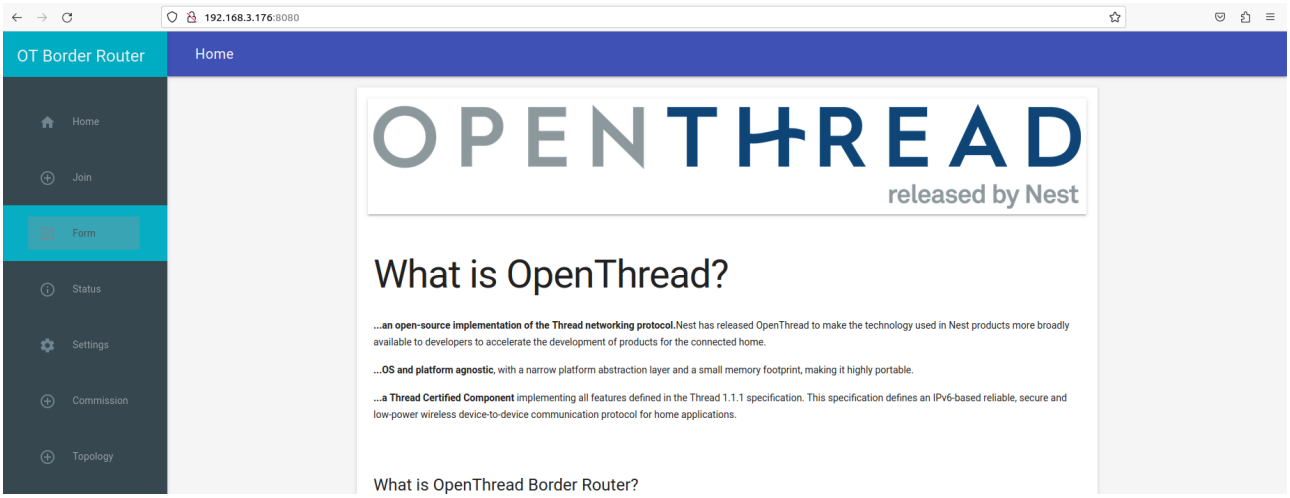
   b. Run the dependency:

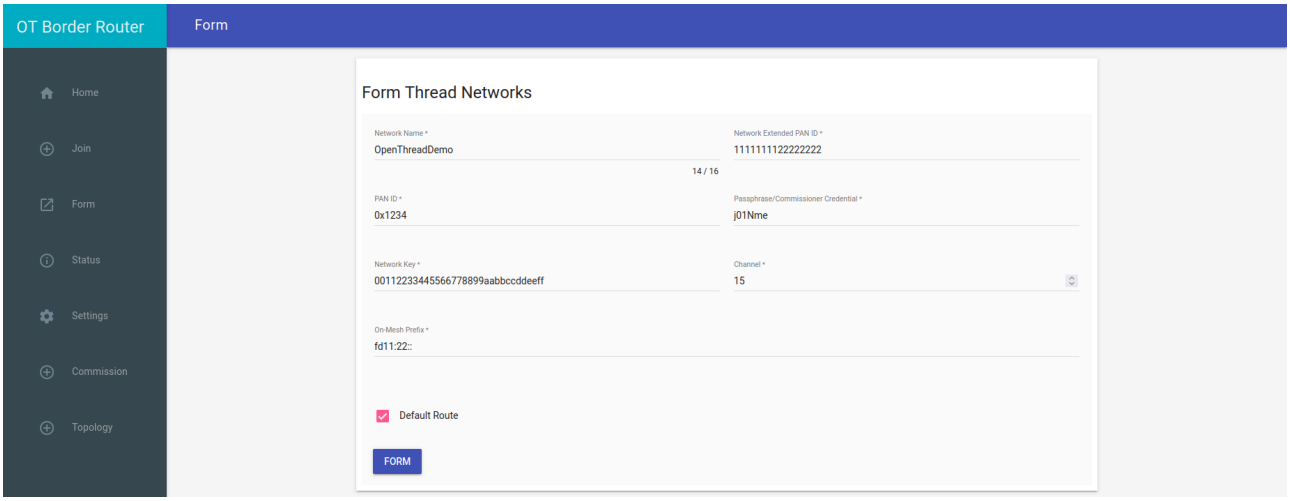
```
sudo modprobe ip6table_filter
```

   c. Run the docker:

```
sudo docker run -it --rm --privileged --network otbr -p 8080:80 --sysctl
"net.ipv6.conf.all.disable_ipv6=0 net.ipv6.conf.all.forwarding=1" --name otbr -e
NAT64=0 --volume /dev/ttyACM0:/dev/ttyACM0
openthread/otbr@sha256:aa02bd1534984be704c84a575f97412c71fab504e6e1f094d0de4e06bfe46cf3
--radio-url spinel+hdlc+uart:///dev/ttyACM0
```

2. Generate the Thread form for dataset by entering '<Raspberry-Pi IP>:8080' on the user's system browser. The OTBR form will be generated as shown below.
3. Click on the **Form** option and follow the sequence to generate the OTBR form.





On Terminal 2:

1. Generation of Hex Code:
   Obtain the dataset hex value by running the following command:

```
sudo docker exec -ti otbr ot-ctl dataset active -x
```

Example hex code :

```
0e080000000000010000000300000f35060004001fffe00208111111112222222220708fdabd97fc1941f290510
0011223344556778899aabbccddeeff030e4f70656e5468726561644656d6f0102123404104 45f2b5ca6f2a9
3a55ce570a70efeecb0c0402a0f7f8
```

2. The above generated sample pairing code can be used during the manual Thread pairing procedure with the following command:

```
./chip-tool pairing ble-thread <node-id> hex:<dataset hex value> <setup-pin>
<discriminator>
./chip-tool pairing ble-thread 97
hex:0e080000000000010000000300001035060004001fffe00208111111112222222020708fd882e3d3a7373dc
05100011223344556778899aabbccddeeff030f4f70656e5468726561644656d707901021234041015 70fcfd
6de18b3d78d6d39881a8a5710c0402a0f7f8 20202021 3840
```

7.5. Troubleshooting: Boarder Router Container failure to initialize

1. Error message: (Example)

```
Error occurred during setup of test suite.FirstChipToolSuite. 409 Client Error for
http+docker://localhost/v1.42/containers/10ad48500522af3d5a23c181a6018053248250b958a353
ed88d5a5f538dcbf33/exec: Conflict ("Container
10ad48500522af3d5a23c181a6018053248250b958a353ed88d5a5f538dcbf33 is not running")
```

   Solution:

   a. Check for the presence of rogue executions of the otbr-chip container. Using command:

```
$docker ps
```

Stop any running otbr-chip containers from the result.

```
$docker container stop <container_id>
```

b. Check host (**raspberry**) network configuration interface's ip address does not conflict with **otbr-chip** default interface ip address.

Conflicting network configuration could be pointed out by checking container's initialization log.

```
$docker logs <container_id>
```

Example Log Output:

```
...
+ service tayga start
* Starting userspace NAT64 tayga
RTNETLINK answers: File exists
RTNETLINK answers: File exists
RTNETLINK answers: File exists
RTNETLINK answers: File exists
...fail!
+ die 'Failed to start tayga'
+ echo ' * ERROR: Failed to start tayga'
* ERROR: Failed to start tayga
+ exit 1
tail: cannot open '/var/log/syslog' for reading: No such file or directory
tail: no files remaining
```

Default Tayga interface address:

```
ipv4-addr 192.168.255.1 # This address could be checked on /etc/tayga.conf on otbr-chip container
```

Use command below on host (**raspberrypi**) to check interface's ip addresses

```
$ifconfig
...
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.2.2 netmask 255.255.255.0 broadcast 192.168.2.255 inet6
fdcb:377:2b62:f8fd:dea6:32ff:fe94:c54c prefixlen 64 scopeid 0x0<global> inet6
fe80::dea6:32ff:fe94:c54c prefixlen 64 scopeid 0x20<link> ether dc:a6:32:94:c5:4c
txqueuelen 1000 (Ethernet) RX packets 250969 bytes 184790487 (184.7 MB) RX errors 0
dropped 0 overruns 0 frame 0 TX packets 125202 bytes 85904550 (85.9 MB) TX errors 0
dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0 inet6 ::1 prefixlen 128 scopeid 0x10<host> loop txqueuelen
1000 (Local Loopback) RX packets 520 bytes 48570 (48.5 KB) RX errors 0 dropped 0
overruns 0 frame 0 TX packets 520 bytes 48570 (48.5 KB) TX errors 0 dropped 0 overruns 0
carrier 0 collisions 0
```

If any interface matches tayga ip address, change the conflicting IP on host.

8. Wi-Fi PAF Commissioning

This section provides a guide to enable Wi-Fi PAF commissioning tests.

8.1. Components Needed for Tests

To enable the Wi-Fi PAF Commissioning tests, the following hardware and software components are required.

8.1.1. Hardware

- **Raspberry Pi**: version permissible for CSA Matter Test Events, used as a platform for Test Harness(Pi 4 or higher).
- **WLAN USB dongle**: compatible with the latest hostapd and wpa_supplicant, used as the Wi-Fi subsystem for transmitting and receiving Wi-Fi Un-Synchronized Discovery (USD) Public Action Frames (PAF), including one of the following kinds:
  - NETGEAR A6210: https://www.amazon.com/NETGEAR-Dual-Band-Adapter-A6210-10000S-Refurbished/dp/B00NSB0G66
  - NETGEAR WN111 802.11n Wireless LAN USB 2.0 Adapter: https://a.co/d/4I7YMez
  - Linksys AE6000 Dual-Band Wireless Mini USB Adapter: https://a.co/d/iyXXpIs
  - SANOXY USB Mini Wifi Wireless LAN Internet Adapter: https://www.amazon.com/SANOXY-150Mbps-Wireless-Network-802-11n/dp/B01HFRCUVM

8.1.2. Software

- **hostapd and wpa_supplicant**: download from the latest master branch. Make sure this exists: CONFIG_NAN_USD=y
  - Used to enable transmitting and receiving Wi-Fi Un-Synchronized Discovery (USD) Public Action Frames (PAF) on platforms for both Test Harness and DUT.
  - **Here is the procedure to configure the above**:
    - git clone https://w1.fi/hostap.git
    - cd hostap/wpa_supplicant
    - git checkout master (Run this if you are in different branch)
    - vi defconfig or nano defconfig
    - Make sure "CONFIG_NAN_USD=y" is set (It should be at the last line)
    - cp defconfig .config
    - sudo apt update
    - sudo apt install libnl-3-dev libnl-genl-3-dev
    - make all

8.2. Matter SDK

- Matter 1.5 SDK is capable of Wi-Fi PAF commissioning
- Regular build command with “chip_device_config_enable_wifipaf=true” (**This step is required only when building from the Master branch. If you’re using the SDK provided in the apps folder of the TH, it’s not needed**)
- **Test commands on SDK as a Commissionee**:
  - `$ ./chip-all-clusters-app --wifi --wifipaf freq_list=2437` // for 2.4GHz.
  - `$ ./chip-all-clusters-app --wifi --wifipaf freq_list=2437,2412,5745,5220` // for default 2.4GHz CH6 + a list of channels in 2.4GHz + a list of 5GHz channels
- **Test commands on SDK as a Commissioner**:
  Use "wifipaf-wifi" to pair:
  - `./chip-tool-paf pairing wifipaf-wifi [node_id] [ssid] [ap_pwd] [passcode] [discriminator]`
  - **Example**: `./chip-tool-paf pairing wifipaf-wifi 1 n_m_2g nxp12345 20202021 3840` // on default 2.4GHz
  - **Example**: `sudo ./chip-tool pairing wifipaf-wifi 1 n_m_2g nxp12345 20202021 3840 --freq 5220` // on 5GHz

8.3. Baseline Test Harness (TH) Configuration for Testing DUT Commissionable Device

1. If DUT commissionable device is 2.4 GHz-only, configure TH commissioner to create an active subscriber on Default Publish Channel (2.4GHz Channel 6) for a test.
2. If DUT commissionable device is 2.4 + 5 GHz, to test 2.4GHz commissioning, configure TH commissioner to create an active subscriber on Default Publish Channel (2.4GHz Channel 6) for a test.
3. If DUT commissionable device is 2.4 + 5 GHz, to test 5GHz commissioning, configure TH commissioner to create an active subscriber on CH 44 non-ETSI regulatory domains or CH 149 in ETSI regulatory for a test.

8.4. Baseline Test Harness (TH) Configuration for Testing DUT Commissioner

For non-ETSI regulatory domain, configure the TH Commissionee with the Default Publish Channel to be Channel 6 in 2.4 GHz and a Publish Channel List that includes all 20 MHz channels in 2.4 GHz band, Channel 44 and Channel 149 in 5 GHz band, and the Commissioner’s network operating channel if the operating channel is not a DFS channel.

8.5. Test Procedures and Test Commands

The test procedures and test commands for Wi-Fi PAF commissioning are documented in the Test Plan Verification spreadsheet under the Matter Specifications and Test Plans folder: https://groups.csa-iot.org/wg/members-all/document/folder/2269 **or** https://docs.google.com/spreadsheets/d/19ZAbIRObi1HcvbesI4tSVmQn7cxz-ZNyFH80onmae7Y/edit?gid=311763523#gid=311763523

The test steps are specifically in the following places:

- 3.2.1. [TC-DD-2.1] Announcement by Device Verification [DUT - Commissionee]: Steps: 8, 9, 10, and 11 with “MCORE.DD.DISCOVERY_PAF” enabled in PICS.
  - TH does not need any special setting, just need to connect to AP. It’s ok even using ethernet. But Wi-Fi should be on that I always test by using Wi-Fi connecting to AP. Test wpa_supplicant configure file script is below:
    - ctrl_interface=DIR=/run/wpa_supplicant
    - update_config=1
    - network={
      ssid="n_m_2g"
      key_mgmt=WPA-PSK
      psk="nxp12345"
      }
  - **Before executing TC-DD-2.1 please read the “readme” file available in the TC-DD-2.1 folder and complete the configuration.**
- 3.2.2. [TC-DD-2.2] Discovery by Commissioner Verification [DUT - Commissioner]: Steps: 3.a and 3.b with “MCORE.DD.DISCOVERY_PAF” enabled in PICS.
- 3.3.11. [TC-DD-3.11] Commissioning Flow = 0 (Standard Flow) - QR Code [DUT - Commissioner]: Steps: 2.a, 2.b and 2.c with “MCORE.DD.DISCOVERY_PAF” enabled in PICS
- 3.3.12. [TC-DD-3.12] Commissioning Flow = 1 (User-Intent Flow) - QR Code [DUT - Commissioner]: Steps: 2.a, 2.b, 2.c and 2.d with “MCORE.DD.DISCOVERY_PAF” enabled in PICS
- 3.3.13. [TC-DD-3.13] Commissioning Flow = 2 (Custom Flow) - QR Code [DUT - Commissioner]: Steps: 2.a, 2.b, 2.c and 2.d with “MCORE.DD.DISCOVERY_PAF” enabled in PICS
- 3.3.14. [TC-DD-3.14] Commissioning Flow - QR Code - Negative Scenario [DUT - Commissioner]: Steps: 4.a and 4.b with “MCORE.DD.DISCOVERY_PAF” enabled in PICS

8.6. Configure the environment

- **Configure it as commissionee**:
  - Configure the wpa_supplicant on the commissionee side using the provided example file named wpa_supplicant-def_comm.conf.
  - Run `./config_paf_env.sh comee` (**Make sure the “config_paf_env.sh” file from the “scripts” folder inside the provided ZIP archive is copied to the working directory before executing this command**).

- **Configure it as commissioner**:
  - Configure the wpa_supplicant on the commissionee side using the provided example file named wpa_supplicant-def_comer.conf.
  - Set the ssid/password of AP to wpa_supplicant-def_comer.conf
  - Run `./config_paf_env.sh comer` (**Make sure the "config_paf_env.sh" file from the "scripts" folder inside the provided ZIP archive is copied to the working directory before executing this command**).
- Renew the commissionee environment before running the test:
  - Run `./renew_paf_comee.sh` (**Make sure the "renew_paf_comee.sh file from the "scripts" folder inside the provided ZIP archive is copied to the working directory before executing this command**)
  - The above script is just clearing the tmp files and removing the existing networks.

9. NFC Setup

This section provides a guide to enable NFC commissioning tests. The NFC interface can be used to commission many kinds of Matter devices, including Thread, Wi-Fi, and Ethernet devices.

9.1. Components Needed for Tests

To enable the NFC Commissioning tests, the following hardware and software components are required.

9.1.1. Hardware

Component	Note
Raspberry Pi	Version permissible for CSA Matter Test Events, used as a platform for Test Harness (Pi 4 or higher).
STM32WBA65I-DK1	STMicroelectronics's STM32WBA65I Discovery Kit (mother board)
X-NUCLEO-NFC11B2	STMicroelectronics's NFC expansion board. Note: This board is not yet available on the market. Please contact tania.guidet@st.com to obtain it.
HID OMNIKEY 5022 CL	NFC/RFID contactless reader. Connected to the Raspberry Pi via USB. Host uses the PC/SC driver to communicate with it. This NFC Reader shall be used when testing NFC commissioning on a **DUT as Commissionee**.

In case of **'nfc-thread'** commissioning, an nRF52840 Dongle, from Nordic, should also be set up.

The STM32WBA65I-DK1 and X-NUCLEO-NFC11A1-S boards form the reference design for **'nfc-thread'** tests for a **DUT as Commissioner**.

9.1.2. Software

STM32WBA65I-DK1 firmware will be available in Matter CSG release directory(matter-csg)

This firmware can be flashed with the tool STM32 Cube Programmer.

A virtual COM port connection can be setup either to check the Software version or to see the debug traces:

- Speed: 921600
- Data: 8 bit
- Parity: None
- Stop bits: 1 bit
- Flow control: None

9.2. Test Procedures and Test Commands

Refer to "*Python Script Validation Procedure*" sheet of TestPlanVerification XLS document (e.g.: TC_DD_1_5, TC_DD_3_22, TC_DD_3_23, TC_DD_3_24).

When running NFC Commissioning tests in Docker, refer to For NFC Pairing for instructions about PC/SC smartcard daemon initialization.

10. Test Configuration

10.1. Project Configuration

When the DUT is a client, refer to Simulated Tests. The TH brings up the example accessory using chip-app1 binary. The user will be prompted to commission the device. Once the commissioning process is completed, proceed with the test execution.

In the case where the DUT is a server, the TH spins up the controller, the DUT bring-up procedure should be completed and has to be paired with the controller.

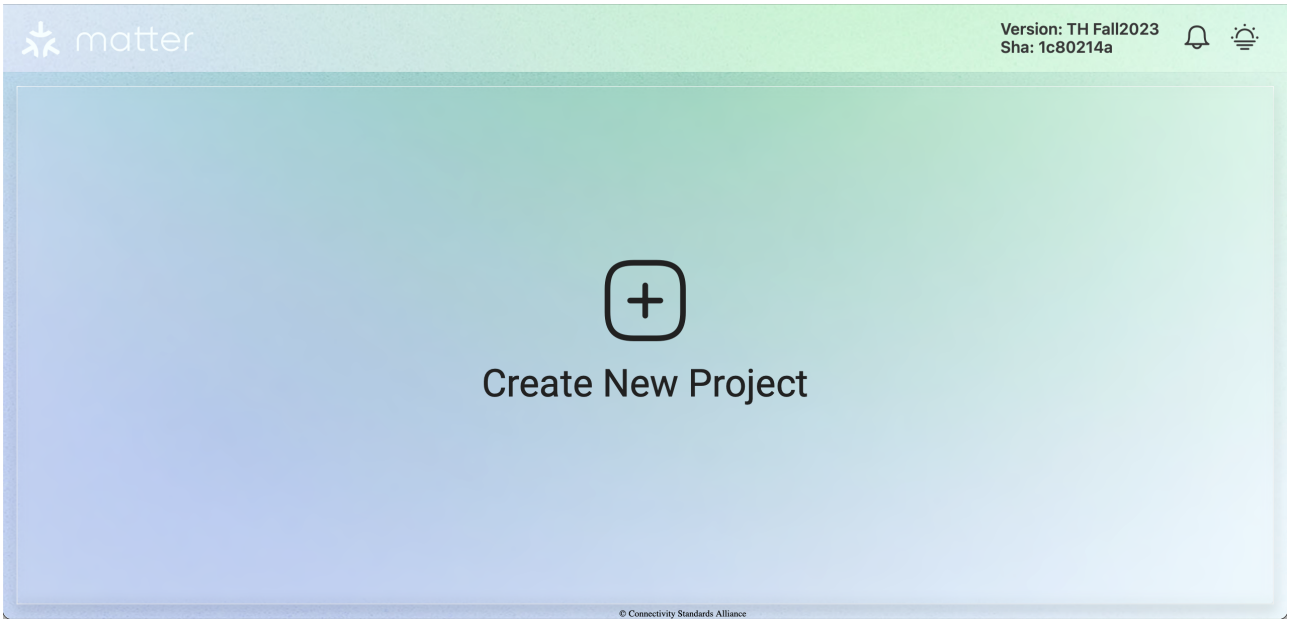
Depending on the DUT's network transport, any one of the appropriate pairing modes can be opted:

- **'ble-wifi'** to complete the pairing for the DUT using BLE Wi-Fi
- **'onnetwork'** to complete the pairing for the DUT that is already on the operational network (e.g., the device is already present on the same Ethernet network of the TH) connection
- **'ble-thread'** to complete the pairing for the Thread Device using BLE
- **'thread-meshcop'** to complete the pairing for the Thread Device using Border Agent (without BLE)
- **'nfc-thread'** to complete the pairing for the Thread Device using NFC Transport Layer (NTL).
- **'nfc-wifi'** to complete the pairing for the Wi-Fi Device using NFC Transport Layer (NTL).
- **'nfc-ethernet'** to complete the pairing for the Ethernet Device using NFC Transport Layer (NTL).
- **'wifipaf-wifi'** to complete the pairing for the Wi-Fi Device using Wi-Fi PAF (refer to Wi-Fi PAF Commissioning).

Follow the sections below for the project configuration and test execution. The complete list of configuration fields is described in Section 8.3, Project Configuration Reference.

10.1.1. Projects Menu

1. Open a Web browser from the user's system and enter the IP address of the Raspberry Pi as given in Section 4.1.2, TH Installation on Raspberry Pi.
2. In case the TH user interface does not launch, refer to Section 4.2.3, Bringing Up of Docker Containers Manually.



3. A new window will be opened as "Matter Test Harness".
4. Click on the **Create New Project** button. Enter the project name as "Test Project" and edit the Project Config settings to provide additional details (every field is described in Section 8.3, Project Configuration Reference).

```
{
  "test_parameters": null,
  "network": {
    "wifi": {
      "ssid": "testharness",
      "password": "wifi-password"
    },
    "thread": {
      "rcp_serial_path": "/dev/ttyACM0",
      "rcp_baudrate": 115200,
      "on_mesh_prefix": "fd11:22::/64",
      "network_interface": "eth0",
      "dataset": {
        "channel": "15",
        "panid": "0x1234",
        "extpanid": "1111111122222222",
        "networkkey": "00112233445566778899aabbccddeeff",
        "networkname": "DEMO"
      },
      "otbr_docker_image": null
    }
  },
  "dut_config": {
    "discriminator": "3840",
    "setup_code": "20202021",
    "pairing_mode": "onnetwork",
    "chip_timeout": null,
    "chip_use_paa_certs": false,
```

```
    "trace_log": true
  },
  "th_config": {
    "prompt_timeout_seconds": 60,
    "enable_realtime_python_test_logs": null,
    "enable_container_logs": null,
    "sdk_container_apps_dir": null
  }
}
```

10.1.2. Wi-Fi Mode

a. Configure the DUT by providing details like discriminator, setup_code and set the **pairing_mode as "ble-wifi"**.

```
"dut_config": {
  "discriminator": "3840",
  "setup_code": "20202021",
  "pairing_mode": "ble-wifi",
  "chip_timeout": null,
  "chip_use_paa_certs": false,
  "trace_log": true
}
```

b. To pair in the BLE Wi-Fi mode, configure the Network settings by providing the ssid and password.

```
"network": {
  "wifi": {
    "ssid": "testharness",
    "password": "wifi-password"
  },
...
}
```

10.1.3. On Network Mode

a. If the DUT is already present on the operational network (e.g., connected to the same network as the controller via Ethernet) then the user can select this mode.

b. Configure the DUT by providing details like discriminator, setup_code and set the **pairing_mode as "onnetwork"**.

```
"dut_config": {
  "discriminator": "3840",
  "setup_code": "20202021",
  "pairing_mode": "onnetwork",
```

```
  "chip_timeout": null,
  "chip_use_paa_certs": false,
  "trace_log": true
}
```

10.1.4. Thread Device Mode

a. Input the DUT configuration details like discriminator: "3840", setup_code:"20202021", and **pairing_mode as "ble-thread"**.

```
"dut_config": {
  "discriminator": "3840",
  "setup_code": "20202021",
  "pairing_mode": "ble-thread",
  "chip_timeout": null,
  "chip_use_paa_certs": false,
  "trace_log": true
}
```

b. The TH loads the default thread configuration values that match the OTBR built on the TH. The following configuration can be customized as per the user's need.

```
"thread": {
  "rcp_serial_path": "/dev/ttyACM0",
  "rcp_baudrate": 115200,
  "on_mesh_prefix": "fd11:22::/64",
  "network_interface": "eth0",
  "dataset": {
    "channel": "15",
    "panid": "0x1234",
    "extpanid": "1111111122222222",
    "networkkey": "00112233445566778899aabbccddeeff",
    "networkname": "DEMO"
  },
  "otbr_docker_image": null
}
```

The OTBR docker is contained in the TH and runs automatically upon the start of the TH tool.

c. If using an already configured Thread network with a Thread Border router present on the same network as the TH, it is possible to provide an explicit operational data configuration so that it is used instead of locally configuring a new Thread PAN/

```
"thread": {
  "operational_dataset_hex":
```

```
"0e080000000000010000350600040001fffe00708fd5270f26ee4c02c041064dc641d7195508d7cd17c
e22db711420c0402a0f7f8000300000f010212340208111111112222222203044445 4d4f05100011223
3445566778899aabbccddeeff"
}
```

OTBR needs to be configured and running. TH will not start any OTBR docker containers.

10.1.5. NFC Device Mode

a. NFC-Thread Device Mode: Set **pairing_mode as "nfc-thread"**. The `discriminator` and `setup_code` fields are not required — onboarding data is read directly from the NFC tag at runtime and these values are ignored if present.

```
"dut_config": {
  "pairing_mode": "nfc-thread",
  "chip_timeout": null,
  "chip_use_paa_certs": false,
  "trace_log": true
}
```

b. NFC-WiFi Device Mode: Set **pairing_mode as "nfc-wifi"**. The `discriminator` and `setup_code` fields are not required — onboarding data is read directly from the NFC tag at runtime and these values are ignored if present.

```
"dut_config": {
  "pairing_mode": "nfc-wifi",
  "chip_timeout": null,
  "chip_use_paa_certs": false,
  "trace_log": true
}
```

c. NFC-Ethernet Device Mode: Set **pairing_mode as "nfc-ethernet"**. The `discriminator` and `setup_code` fields are not required — onboarding data is read directly from the NFC tag at runtime and these values are ignored if present.

```
"dut_config": {
  "pairing_mode": "nfc-ethernet",
  "chip_timeout": null,
  "chip_use_paa_certs": false,
  "trace_log": true
}
```

d. To validate test cases using **nfc-thread**, **nfc-wifi**, or **nfc-ethernet**, the following parameter must be included in the test parameters, irrespective of the test case-specific arguments:

```
- "int-arg": "NFC_Reader_index:<value>"
```

e.g.,

```
"test_parameters": {
  "int-arg": "NFC_Reader_index:0"
}
```

NFC_Reader_index tells the test which NFC reader hardware to use (in case there are multiple NFC Readers), and must be set correctly based on the ATL setup. If there is only one NFC reader, its index is 0.

e. `qr-code` in test parameters:

For **commissioning** test cases under NFC pairing modes, the `qr-code` field in `test_parameters` is automatically ignored — onboarding data comes from the NFC tag, not from a QR code. Including it will not cause an error, but it will be ignored.

For **non-commissioning** test cases (e.g., `TC_DD_1_5`), `qr-code` is a legitimate test input and must be preserved. It is passed to the test as-is and is not stripped.

10.1.6. Thread Pairing Mode (using Border Agent)

a. Input the DUT configuration details like discriminator: "3840", setup_code:"20202021", and **pairing_mode as "thread-meshcop"**.

```
"dut_config": {
  "discriminator": "3840",
  "setup_code": "20202021",
  "pairing_mode": "thread-meshcop",
  "chip_timeout": null,
  "chip_use_paa_certs": false,
  "trace_log": true
}
```

b. For Thread pairing mode, Border Agent parameters `operational_dataset_hex`, `ba_host` and `ba_port` are mandatory.

```
"thread": {
  "operational_dataset_hex":
"0e080000000000010000035060004001fffe00708fd5270f26ee4c02c041064dc641d7195508d7cd17c
e22db711420c0402a0f7f8000300000f010212340208111111112222222203044445 4d4f05100011223
3445566778899aabbccddeeff",
  "ba_host": "192.168.1.100",
  "ba_port": 5684
}
```

10.1.7. PAA Certificates

For the case that the DUT requires a PAA certificate to perform a pairing operation, input "true" for the flag "chip_use_paa_certs" to configure the Test-Harness to use them.

```
"dut_config": {
  "discriminator": "3840",
  "setup_code": "20202021",
  "pairing_mode": "onnetwork",
  "chip_timeout": null,
  "chip_use_paa_certs": true,
  "trace_log": true
}
```

Make sure to include the desired PAA certificates in the default path "**/var/paa-root-certs/**", in the Raspberry-Pi.

10.2. Test Parameters

a. Input the test parameters like endpoint on the DUT where the cluster to be tested is implemented.

```
"test_parameters": {
  "endpoint": 5
}
```

b. "qr-code" and "manual-code" parameters:
Only one of the following parameter is allowed, also when one of them is configured, the TH will not send "passcode" and "discriminator" (from "dut_config") arguments to DUT.

i. "qr-code" parameter example:

```
"test_parameters": {
  "qr-code": "MT:-24J042C00KA0648G00"
}
```

ii. "manual-code" parameter example:

```
"test_parameters": {
  "manual-code": "34970112332"
}
```

iii. **Invalid** configuration: "manual-code" and "qr-code" together:

```
"test_parameters": {
```

```
  "qr-code": "MT:-24J042C00KA0648G00"
  "manual-code": "34970112332"
}
```

This is an invalid configuration. TH will not accept both parameters set at the same time.

c. Overwrite the default timeout. Value in [s]:

```
"test_parameters": {
  "timeout": 300
}
```

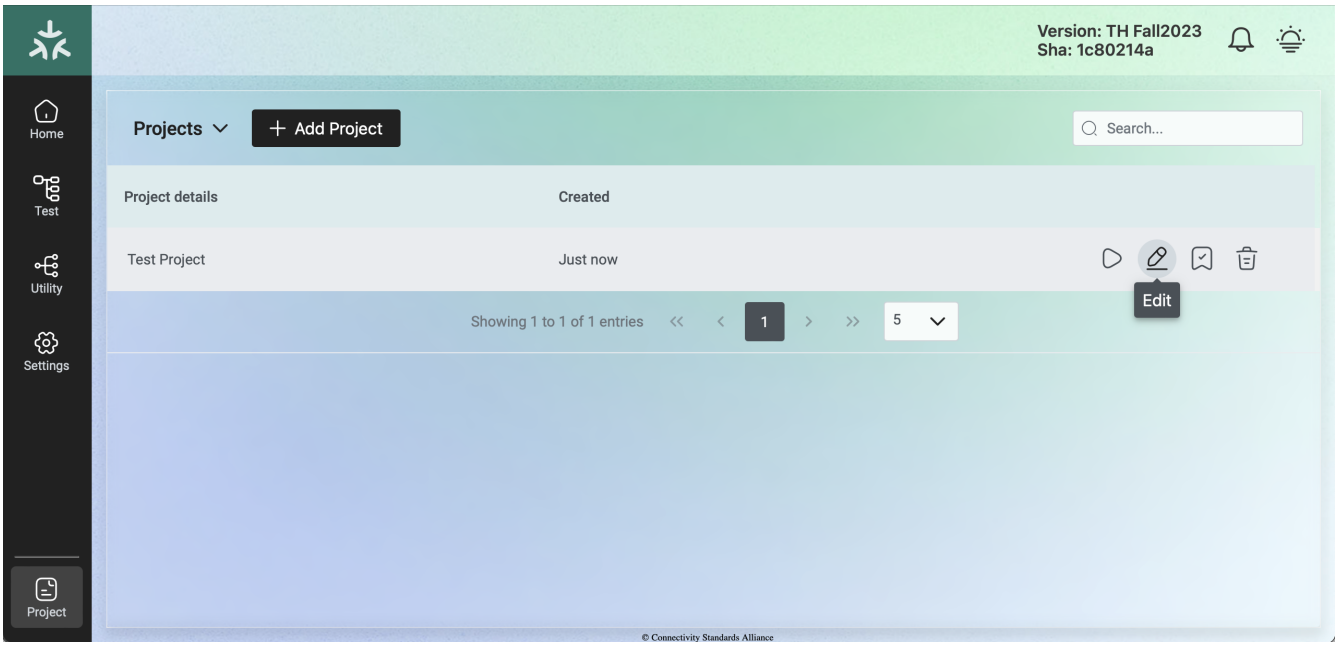
This `test_parameters.timeout` controls how long the chip-tool/Python test **execution** is allowed to run. It is unrelated to the interactive user-prompt timeout (`th_config.prompt_timeout_seconds`) described in User Prompt Timeout Configuration.

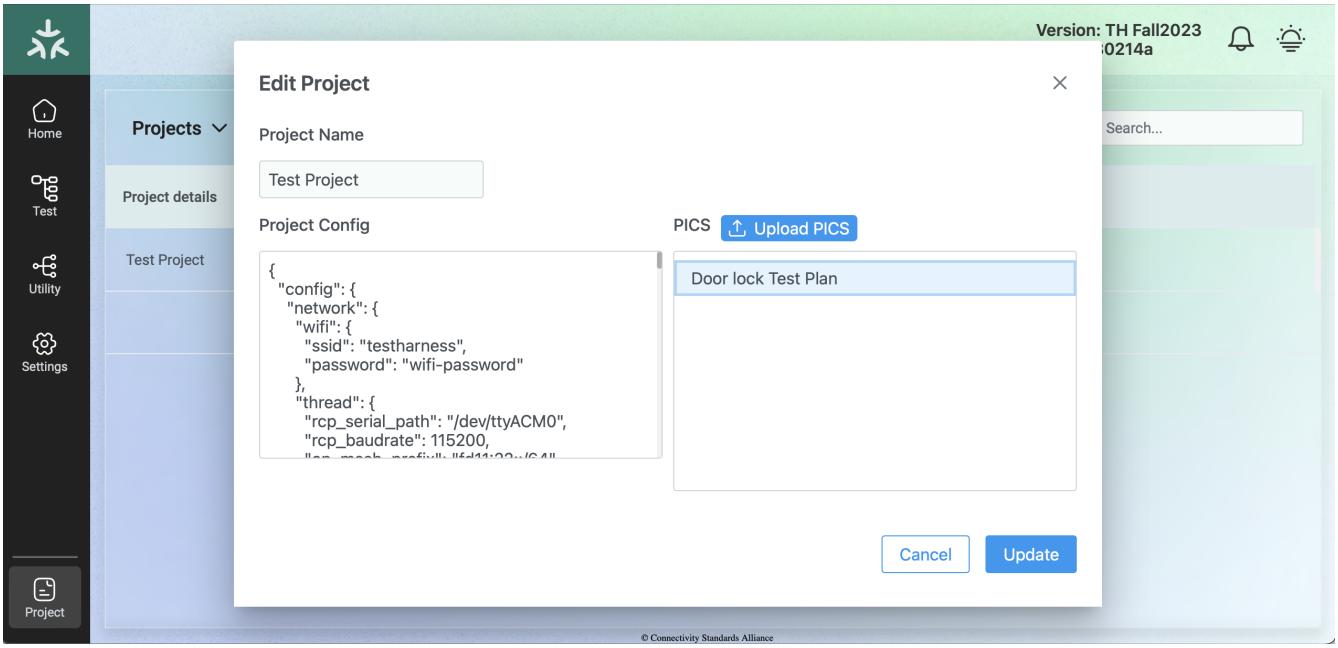
On completion of the "network" and the "dut_config" configuration, select the **Update** and then **Create** button to create the Test Project.

10.2.1. Upload PICS File

The newly created project will be listed under the Project details column.

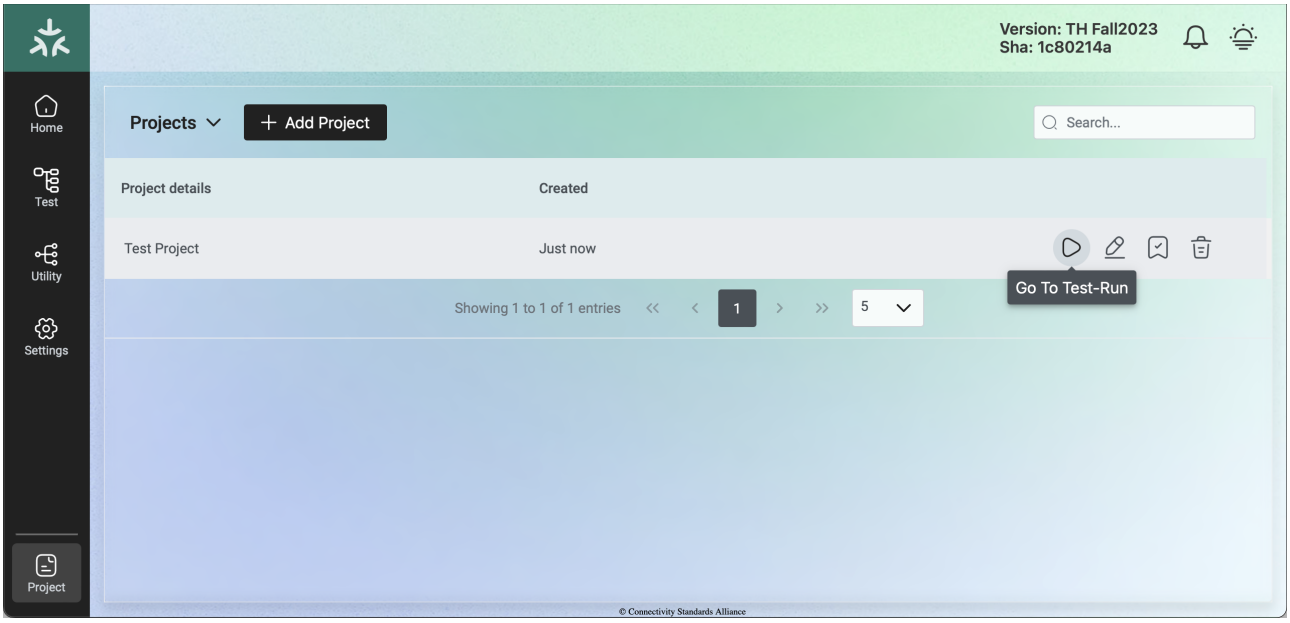
Click on the Edit option to configure the project to load the required PICS file for the cluster to be tested and select the **Update** button. Refer to Section 9, Test Case Execution.

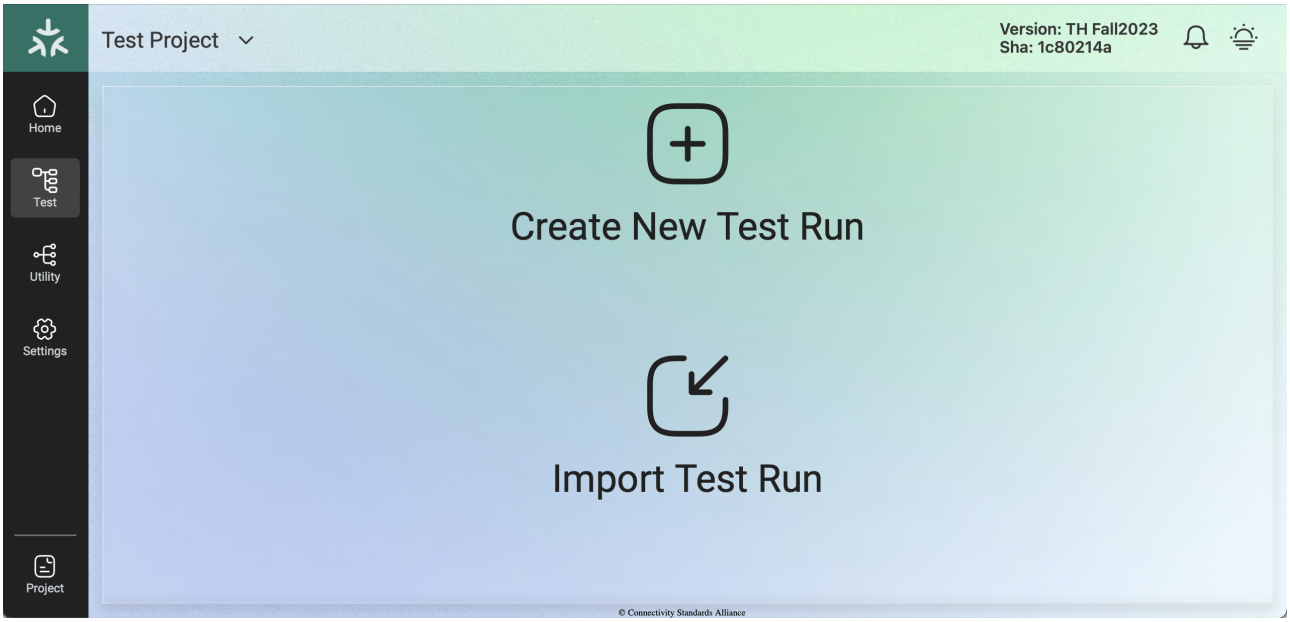




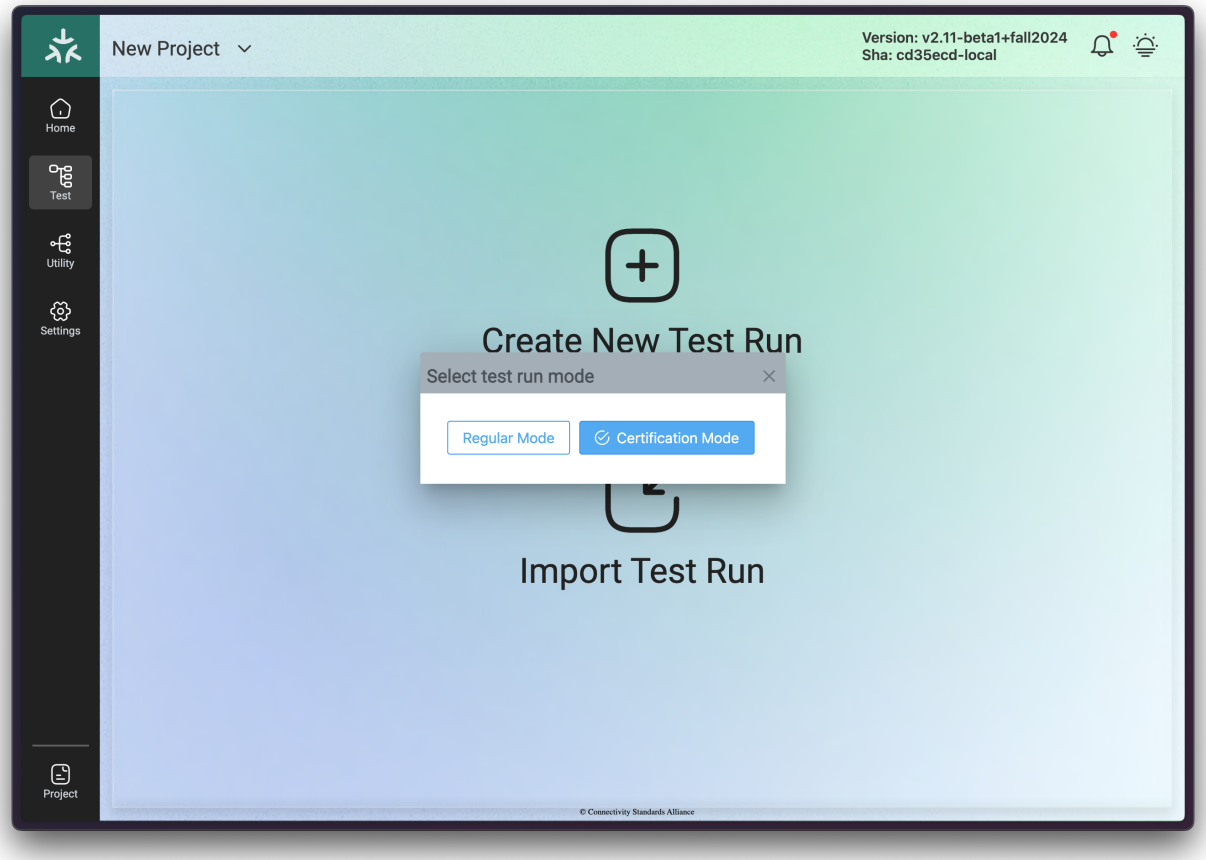
10.2.2. Test Menu

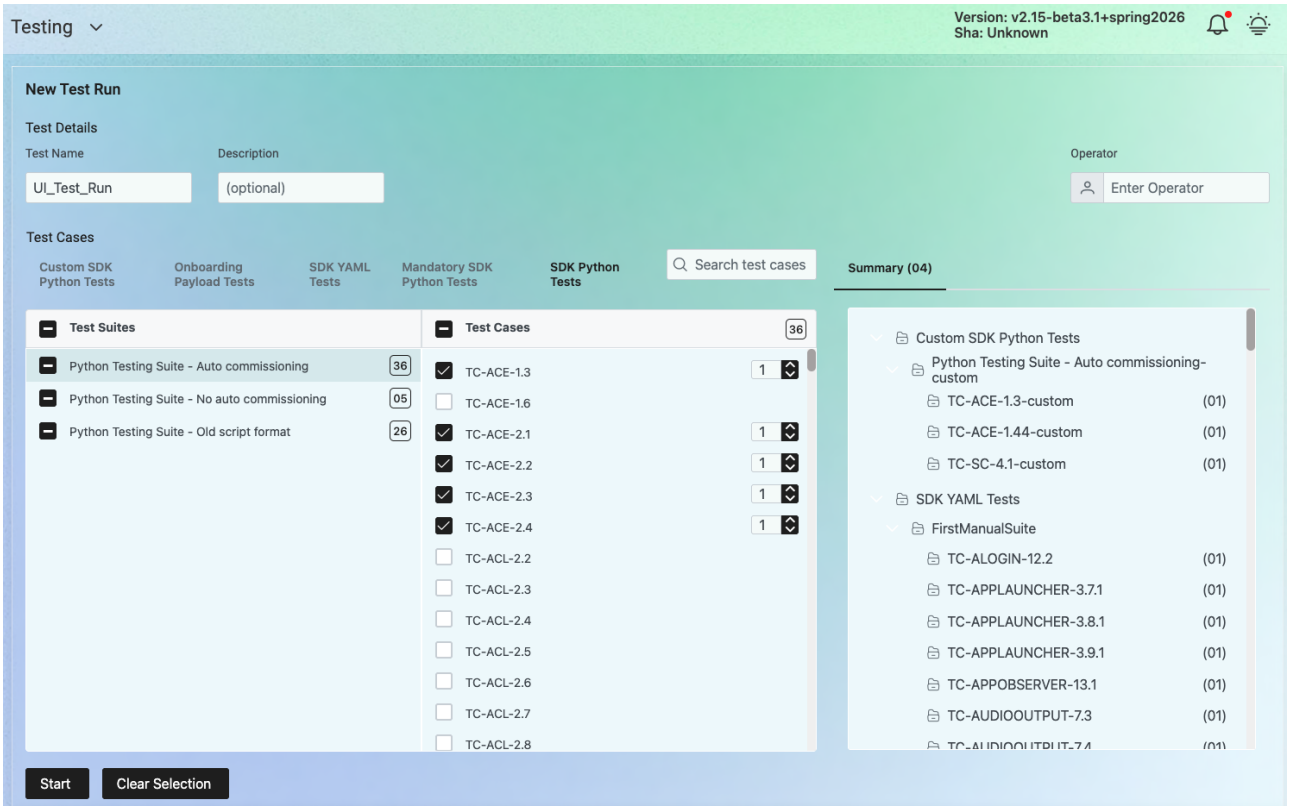
1. Now the Test Project is ready for execution. Click on the **Go To Test-Run** icon and create a new Test Run batch.





2. A Test Run can be created in Regular Mode or Certification Mode. The test cases are automatically selected based on the PICS files provided in the Project Configuration. For a Test Run in Regular Mode, it is possible to change this selection, but in Certification Mode that selection is unchangable — a test case must be executed if and only if the PICS files indicate that it is applicable.



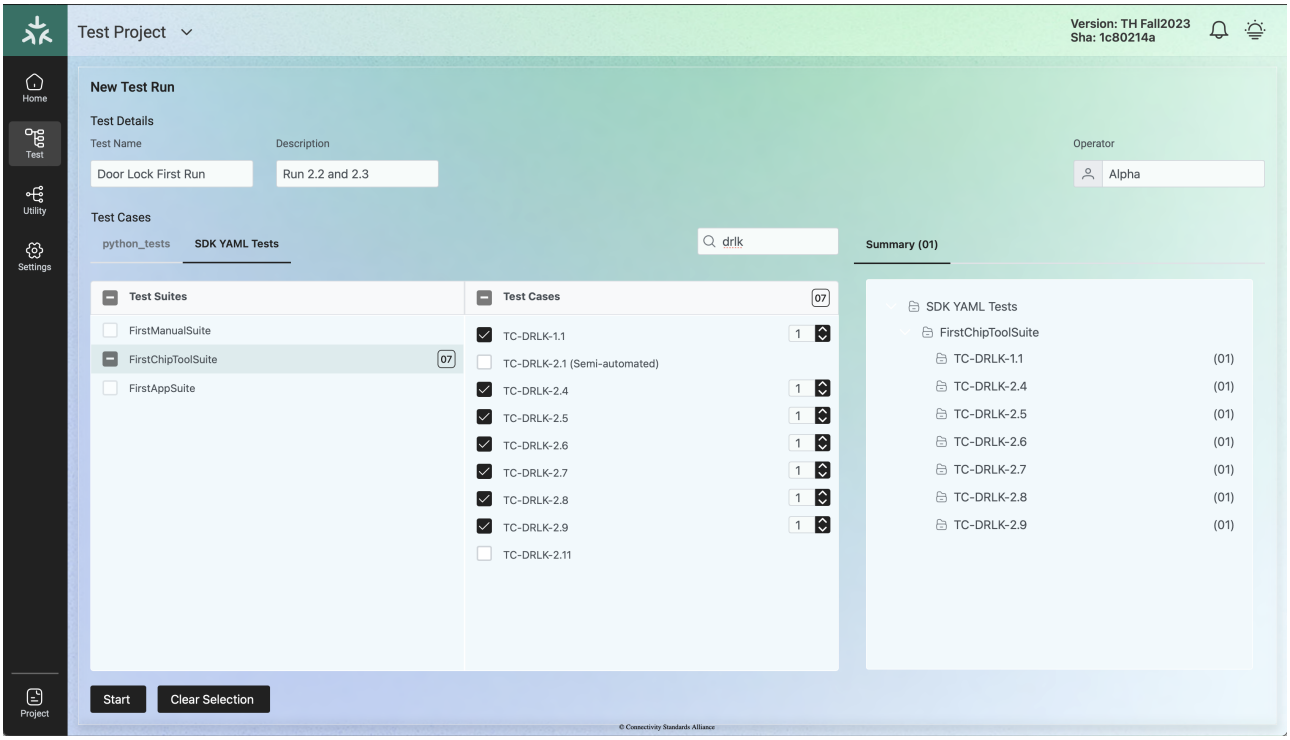


3. Provide a Test name for this run such as Door Lock First Run. Input any additional description about the run. Enter the Test Engineers Name under Operator. Select only the test cases that are to be executed and deselect other test cases. There is a search option available to search for a particular test case. The number of times the test is to be executed can be given by clicking on the number spin control.
   Ensure that DUT is in the discoverable mode before clicking on the Start button.
   Example command to be used to launch the sample apps (e.g., all-cluster-app):

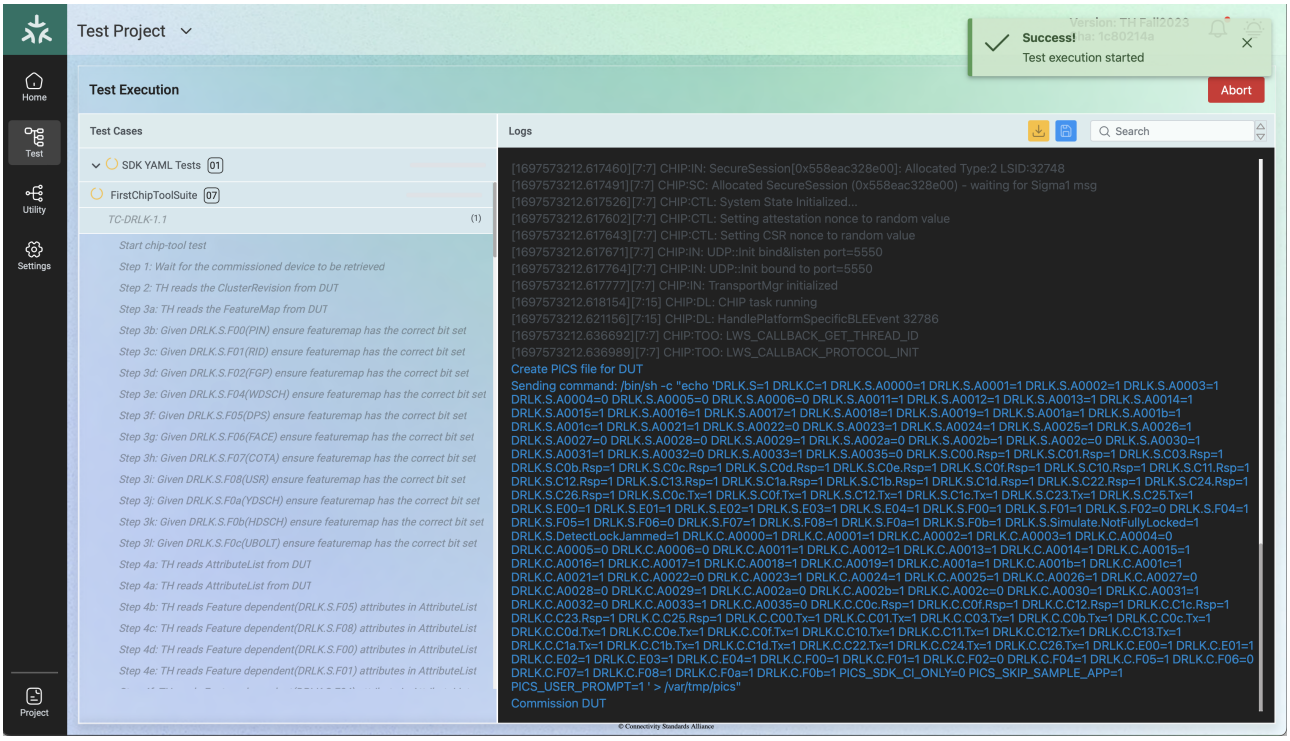
   Ble-wifi: ./chip-all-clusters-app --wifi

   Onnetwork: ./chip-all-clusters-app

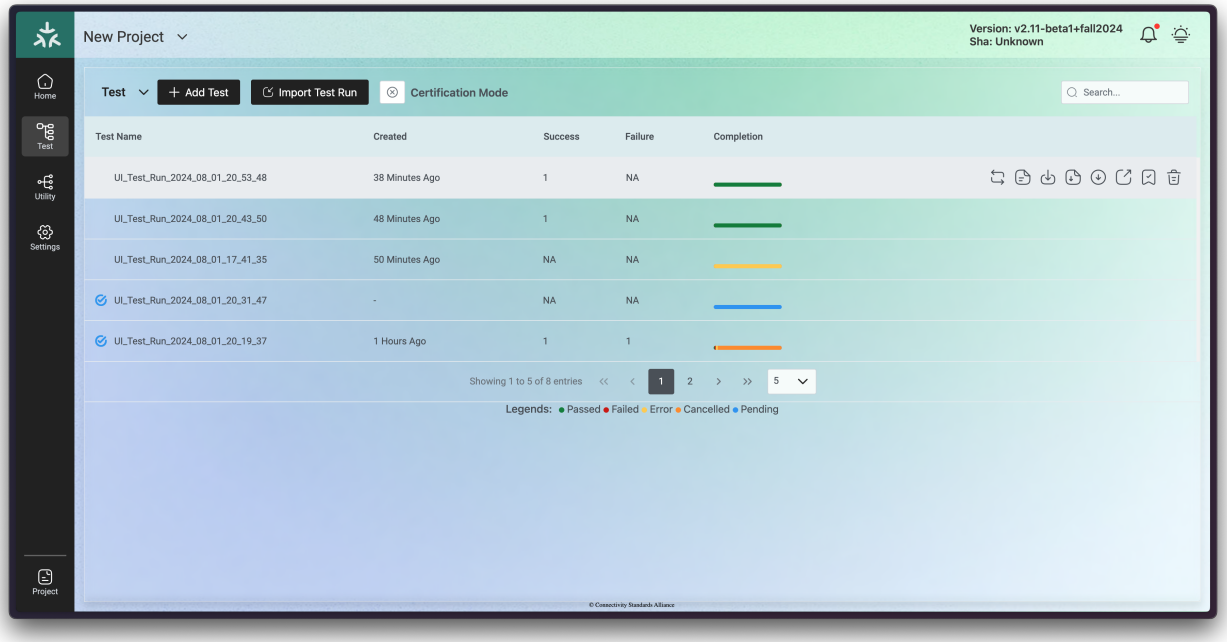
   Thread: Enable discoverable over Bluetooth LE (ex: On nRF52840 DK: Press Button 4 to start BLE advertisements)



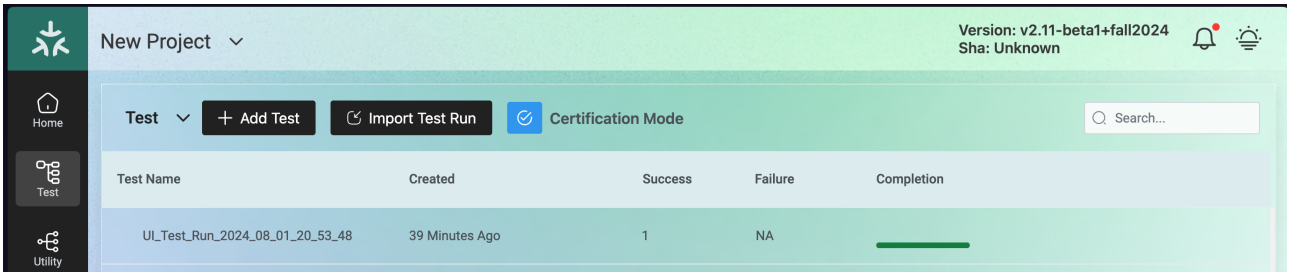
4. Click on the **Start** button for the test execution. Note that the test execution gets started and the log window appears. Click on the **Abort** button to stop the test execution.



5. Once the test execution is completed, click on
   - The Yellow icon to download the test logs
   - The Blue icon to save the test reports
6. Click on the **Result** button and select the test that was executed and click on **Show Report** to view the reports. The user can also select previously executed tests and view the reports and logs. There is an option provided to re-run the test cases. Refer to Section 10, Collect Logs and Submit to TEDS to collect the logs and submit the reports to TEDS.

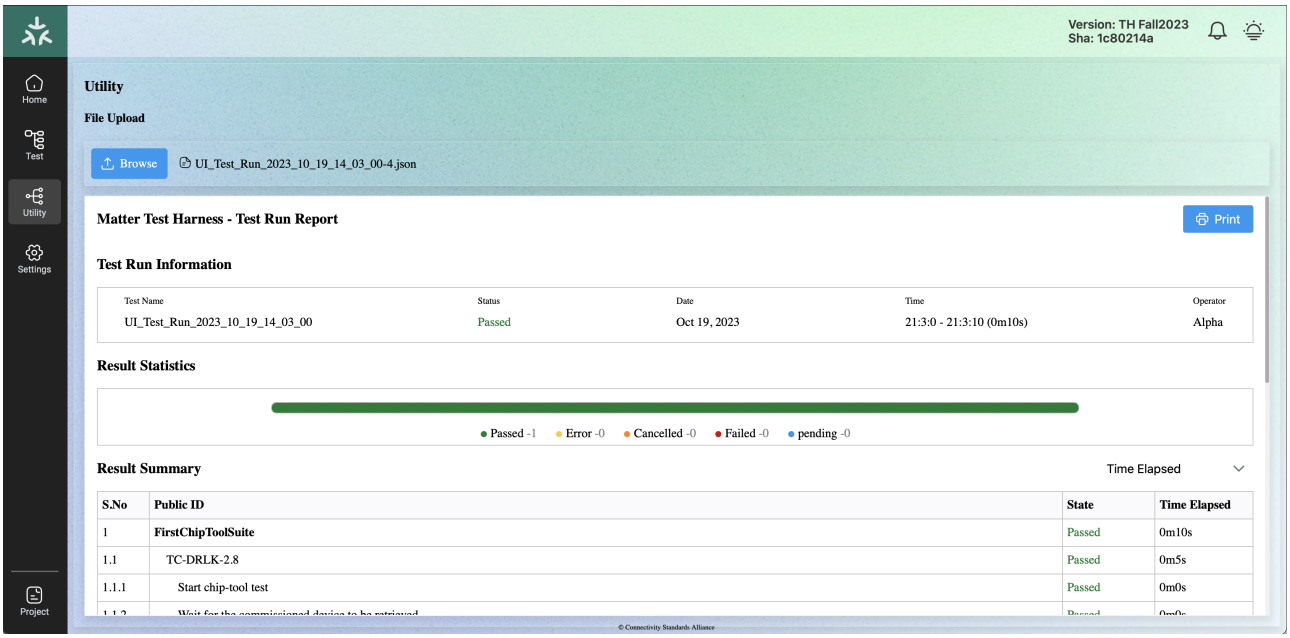


7. To start a new Test Run in Certification Mode, first select the Certification Mode button and then click on + **Add Test**.



10.2.3. Utility Menu

1. Click on **Utility Menu** to review the previous test report.



2. Click on the **Browse** button to upload the previous report and select the desired log filter

options. The console logger contains a filter drop-down list to select the different categories of logs to display. Use the **Print** button to print the test report.

10.2.4. Settings Menu

Click on the "**Select theme**" option drop-down to select the different theme for the user interface.

10.3. Project Configuration Reference

Each project stores its test environment as a single JSON document, the **project configuration**. The same document is shown in the **Project Config** field of the web UI and is read and written by the CLI commands `th-cli project create`, `project update`, `project edit`, `project list --json` and `run-tests --config`. This section describes how the configuration can be edited, its structure, and which fields are mandatory.

10.3.1. Editing the Project Configuration

The configuration can be edited from the web UI or from the CLI. Both save the configuration through the same Test-Harness API, so the same validation applies to either.

Task	Web UI	CLI
Create a project	**Create New Project**: the **Project Config** field is pre-filled with the default configuration. Edit it, select **Update** and then **Create** (see Projects Menu).	`th-cli project create --name "<name>" --config <file>`
View the configuration of a project	Select **Edit** on the project in the project list.	`th-cli project list --id <ID> --json`
Edit the configuration of an existing project	Select **Edit** on the project in the project list. The text area holds a JSON document with two entries, `config` and `pics`. Edit it and apply the change.	`th-cli project edit --id <ID>` opens only the `config` portion in your editor (`$VISUAL` or `$EDITOR`).
Replace the configuration with the content of a file	Paste the new JSON into the **Edit** text area.	`th-cli project update --id <ID> --config <file>`
Copy a project, with its configuration, to another Test-Harness	Not available in the UI.	`th-cli project export` and `th-cli project import`

Notes:

- `project update` and the UI replace the whole `config` object, not individual fields. Use `th-cli project edit` to change one or two values without re-creating the full document.
- When `th-cli project edit` is used, the CLI checks the JSON before saving. If the JSON is invalid, or if the Test-Harness rejects the configuration, the editor reopens with your edit preserved and an explanation of the problem. If you added a key that was not in the original configuration, the CLI lists it and asks for confirmation (see Validation Rules for why this matters).
- The UI and the CLI do not update each other live. If a project is edited in both, the last save wins. Reload the project list in the UI after editing with the CLI.
- The file passed to `--config` can be either the configuration alone (`{"network": {…}, "dut_config": {…}}`) or a full project document (`{"name": "…", "config": {…}}`), so the output of `project export` can be reused. JSON does not allow comments.
- `th-cli run-tests --config <file>` applies the file to that test run only. It does not change the configuration stored in the project.

The simplest way to start a new configuration is the default one pre-filled by the UI, or the configuration of an existing project printed by `th-cli project list --id <ID> --json`.

10.3.2. Structure and Mandatory Fields

Key	Type	Required	Description
`network`	object	Yes	Network settings. It must contain both `wifi` and `thread`, even when the DUT uses neither of them (see below).
`dut_config`	object	Yes	DUT pairing settings (see below).
`test_parameters`	object or `null`	No	Test parameters passed to the test cases (see Test Parameters). `qr-code` and `manual-code` cannot be used together.
`th_config`	object	No	Test-Harness settings. When omitted, the defaults listed below apply.

10.3.2.1. `dut_config`

Key	Type	Required	Description
`pairing_mode`	string	Yes	One of `onnetwork`, `ble-wifi`, `ble-thread`, `thread-meshcop`, `nfc-wifi`, `nfc-thread` or `wifipaf-wifi`. See Project Configuration for when to use each mode.

Key	Type	Required	Description
`discriminator`	string	Yes, except for NFC modes	DUT discriminator, for example `"3840"`. For `nfc-wifi` and `nfc-thread` it is optional, as the onboarding data is read from the NFC tag.
`setup_code`	string	Yes, except for NFC modes	DUT setup code (passcode), for example `"20202021"`. Optional for the NFC modes, as above.
`chip_use_paa_certs`	boolean	Yes	The key must be present, with either `true` or `false`. Omitting it is an error: it does not fall back to a default value. `true` makes the Test-Harness use the PAA certificates installed in `/var/paa-root-certs/` (see PAA Certificates).
`trace_log`	boolean	Yes	The key must be present, with either `true` or `false`. Omitting it is an error: it does not fall back to a default value. When `true`, SDK Python tests run with trace logging enabled (`--trace-to json:log`).
`chip_timeout`	string or `null`	No	Accepted for compatibility, but not currently used by the Test-Harness. Leave it as `null`.
`enhanced_setup_flow`	object	No	Used to run SDK Python tests against a DUT that implements the Enhanced Setup Flow (Terms and Conditions). When present, both entries are mandatory: `tc_user_response` (integer), passed to the test as `--tc-acknowledgements`, and `tc_version` (integer), passed as `--tc-acknowledgements-version`. The test is also started with `--require-tc-acknowledgements 1`.

10.3.2.2. `network.wifi`

Key	Type	Required	Description
`ssid`	string	Yes	Wi-Fi network name used for Wi-Fi pairing.
`password`	string	Yes	Wi-Fi network password.

Key	Type	Required	Description
`interfaces`	list of strings	No (default: empty list)	Names of the host Wi-Fi interfaces dedicated to the Wi-Fi Fixture, for example `["wlx6ccdd6c589f0"]`. Naming at least one interface is what makes the Test-Harness start the Wi-Fi Fixture container. The Test-Harness needs exclusive use of these interfaces; the test fails to start if a host service manages one of them. See No Link-Local IPv6 Address Assigned to the Wi-Fi Fixture Interface.
`docker_image`	string or `null`	No	Docker image used for the Wi-Fi Fixture container. When omitted or `null`, the default image is used.

10.3.2.3. `network.thread`

`thread` takes one of two forms.

Test-Harness border router (the Test-Harness starts the OTBR container using an RCP dongle):

Key	Type	Required	Description
`rcp_serial_path`	string	Yes	Serial device of the RCP dongle, for example `/dev/ttyACM0`.
`rcp_baudrate`	integer	Yes	Baud rate of the RCP serial connection, for example `115200`.
`on_mesh_prefix`	string	Yes	On-mesh prefix, for example `fd11:22::/64`.
`network_interface`	string	Yes	Host network interface used by the border router, for example `eth0`.
`dataset`	object	Yes	Thread dataset. All of `channel`, `panid`, `extpanid`, `networkkey` and `networkname` are mandatory strings.
`otbr_docker_image`	string or `null`	No	OTBR Docker image. When `null`, the default image is used.

External border router (the Thread network already exists; used with `thread-meshcop`):

Key	Type	Required	Description
`operational_dataset_hex`	string	Yes	Active operational dataset as a hex string.
`ba_host`	string	Only for `thread-meshcop`	Host or IP address of the Thread Border Agent.

Key	Type	Required	Description
`ba_port`	integer	Only for `thread-meshcop`	Port of the Thread Border Agent, for example `5684`.

10.3.2.4. `th_config`

Key	Type	Default	Description
`prompt_timeout_seconds`	integer	`60`	How long the Test-Harness waits for an answer to a user prompt. See User Prompt Timeout Configuration.
`enable_realtime_python_test_logs`	boolean or `null`	`null`	`null` defers to the `ENABLE_REALTIME_PYTHON_TEST_LOGS` environment variable. See Python Test Logging Configuration.
`enable_container_logs`	boolean or `null`	`null`	`null` defers to the `ENABLE_CONTAINER_LOGS` environment variable. See Container Logging Configuration.
`sdk_container_apps_dir`	string or `null`	`null`	`null` defers to the `SDK_CONTAINER_APPS_DIR` environment variable. See SDK Container Apps Directory.

10.3.3. Validation Rules

When a configuration is saved, the Test-Harness checks it as follows:

- `network` and `dut_config` are mandatory, and `network` must contain `wifi` and `thread`.
- In `dut_config`, `pairing_mode`, `chip_use_paa_certs` and `trace_log` must always be present. A missing key is rejected; the Test-Harness does not substitute a default value for it.
- Any key in `dut_config` other than the ones listed above is **rejected**, and the error message lists the valid keys.
- An unknown key anywhere **else** in the configuration is **silently ignored**, not rejected. The configuration is saved exactly as typed, but the Test-Harness never reads the unknown key. A misspelled key, such as `prompt_timeout_second` in `th_config`, therefore causes no error and the default value is used instead, so check key names against the tables above. The default configuration also contains a `fabric_id` entry in `network`, which the Test-Harness ignores.
- `qr-code` and `manual-code` in `test_parameters` are mutually exclusive.
- With `thread-meshcop`, `ba_host` and `ba_port` are mandatory in `network.thread`.

11. OTA Image Build Procedure

11.1. Overview

For validating OTA-related test cases using the Test Harness (TH), the `ota-requestor-app.ota` image is required. Currently, the Test Harness does not natively generate this image, and ATLs (Authorized Test Laboratories) must follow a manual OTA image build workflow.

11.2. High-Level Workflow Steps

ATLs should follow these steps when preparing for OTA-related test cases:

1. Identify that OTA-related test cases require an OTA requestor image (`ota-requestor-app.ota`).
2. Build the OTA Provider applications on the DUT/Reference platform.
3. Update the Software Version on the DUT (for example, from the default value 1 to a higher version such as 2) to enable OTA upgrade validation.
4. Generate the OTA image using the Matter `ota_image_tool` with the updated Software Version.
5. Provide/Copy the generated `ota-requestor-app.ota` image to the Test Harness `apps` directory.
6. Execute OTA-related test cases using the Test Harness.

11.3. Reference Commands (Raspberry Pi Platform)

The commands below are provided as a reference implementation for RPi-based setups. Other platforms may require different build commands.

11.3.1. Build OTA Provider

```
scripts/examples/gn_build_example.sh examples/ota-provider-app/linux out/debug
chip_config_network_layer_ble=false
```

11.3.2. Update Software Version

1. Edit the configuration file to update the software version:

   ```
   vi config/standalone/CHIPProjectConfig.h
   ```

   Update the software version value from the default (e.g., 1) to a higher version (e.g., 2).

11.3.3. Build OTA Requestor

```
scripts/examples/gn_build_example.sh examples/ota-requestor-app/linux out/debug
```

```
chip_config_network_layer_ble=false
```

11.3.4. Generate OTA Image

1. Navigate to the output directory:

   ```
   cd out/debug
   ```

2. Run the OTA image tool to create the OTA image:

   ```
   ../../src/app/ota_image_tool.py create -v 0xDEAD -p 0xBEEF -vn 2 -vs "2.0" -da
   sha256 chip-ota-requestor-app ota-requestor-app.ota
   ```

   Where: * `-v 0xDEAD`: Vendor ID * `-p 0xBEEF`: Product ID * `-vn 2`: Version Number * `-vs "2.0"`: Version String * `-da sha256`: Digest Algorithm * `chip-ota-requestor-app`: Input binary file * `ota-requestor-app.ota`: Output OTA image file

11.3.5. Deploy OTA Image to Test Harness

Copy the generated `ota-requestor-app.ota` image to the Test Harness apps/images directory:

```
cp ota-requestor-app.ota ~/apps
```

11.4. Important Notes

- This procedure is **not documented** in earlier versions of the Test Harness User Guide, which may lead to confusion or inconsistent validation results across ATLs.
- The software version specified in the OTA image must be higher than the version running on the DUT to trigger the OTA upgrade process.
- Ensure that the Vendor ID and Product ID match your DUT configuration.

12. Test Case Execution

Refer to Section 2, References for PICS tool documentation to generate the PICS XML files.

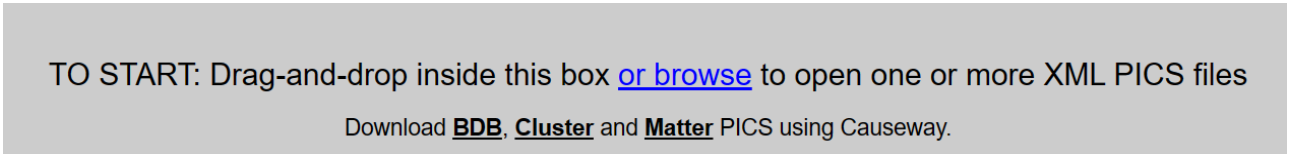
PICS (*Protocol Implementation Conformance Statement*) is a list of features supported by a device as defined by a technology *protocol* , standard or specification. Each feature is known as a *PICS Item* , and device *implementation* is either mandatory or optional. PICS is used by the device manufacturer as a *statement* of *conformance to* a technology standard and a requirement for all CSA Product Certification programs.

PICS codes are generated from the Test Plans. The Base.xml file lists all the Core feature PICS from the Matter Base Specifications and the application cluster PICS are listed in the respective TestPlan.xml files. Follow the steps below to generate and upload the PICS files.

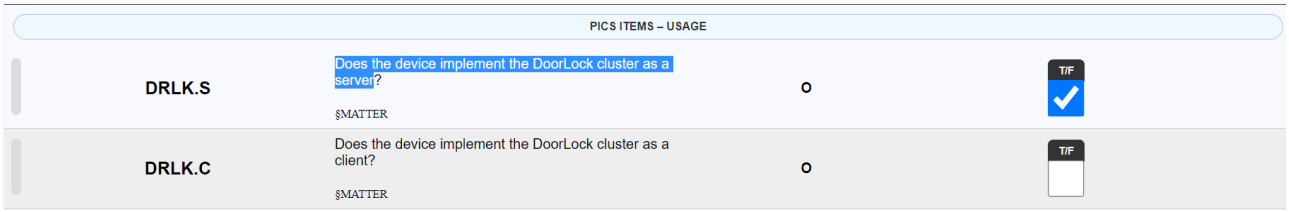
1. Click on the following link to download the PICS XML files— https://groups.csa-iot.org/wg/members-all/document/folder/4120
2. Click on the following link to use the PICS tool— PICS Tool v1.6.4 matter 1.0 - Connectivity Standards Alliance (csa-iot.org)
3. Load the Base.xml file by clicking on the **Browse** option. In case the following error is observed:

   Base.xml: This XML PICS template is unapproved and has not been tested with this tool. To test new or updated PICS documents, please enable *author mode* and try again.

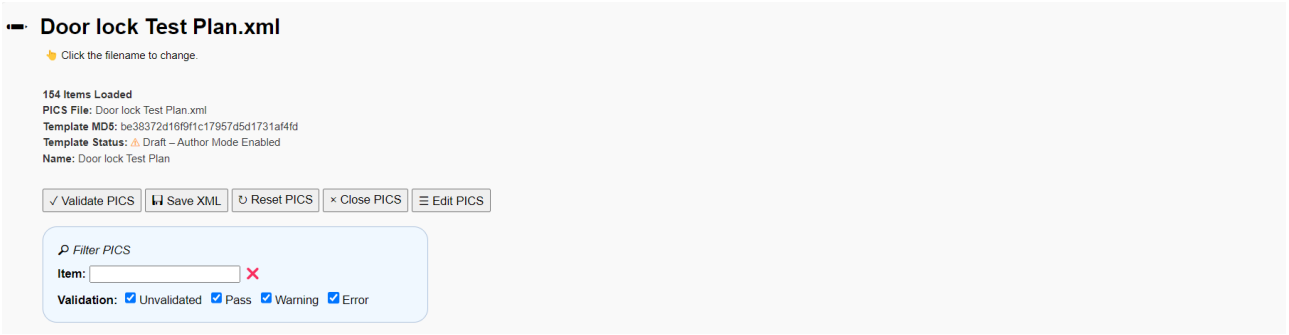
   Enable author mode and retry uploading the XML file.

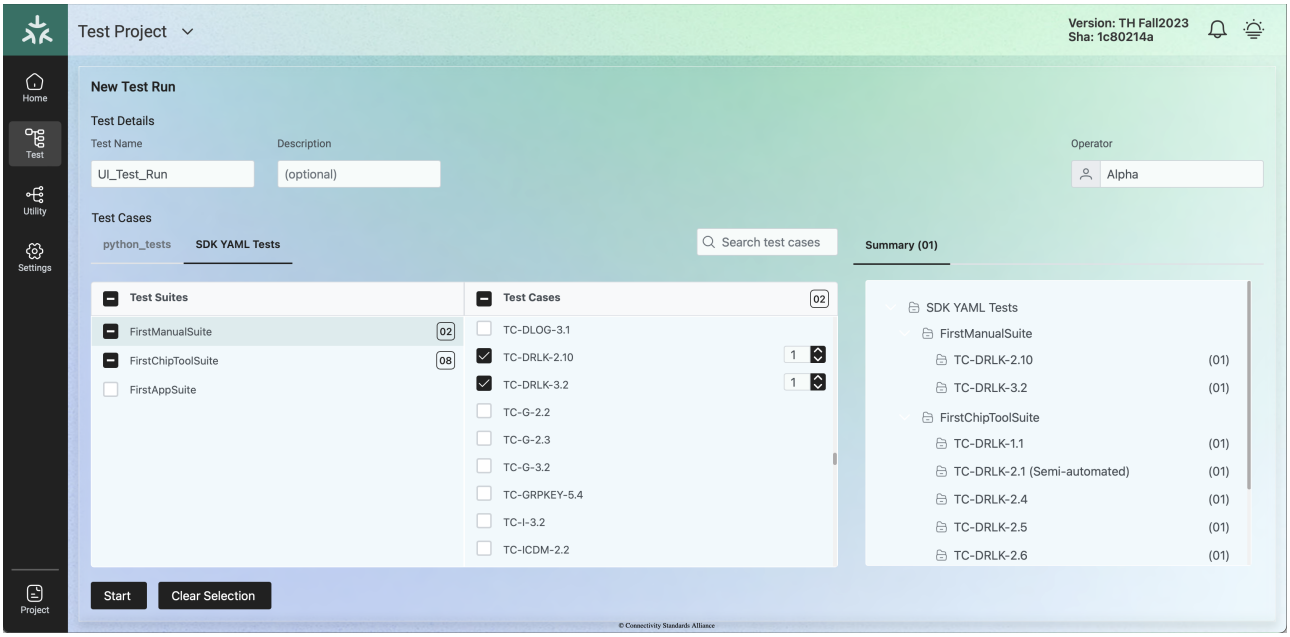
4. Load the XML file that is required for testing, e.g., Doorlock.xml.
5. Check the option for which the testing will be done for the DoorLock cluster. In the case of the Door Lock cluster to be tested in the Server mode, select the checkbox for DRLK.S. In case the cluster has to be tested in the Client mode, select the checkbox for DRLK.C.

6. Review all the attributes/commands that are supported by the DoorLock cluster and ensure the corresponding options are checked in the PICS tool.
7. Click on **Validate PICS** . Ensure that there are no warnings or errors. In case of any warnings or errors, revisit the options and check/uncheck the options as supported by the DUT.



8. Prior to the test execution, the user will have to load the relevant PICS file to list the required test cases. Depending on the PICS file loaded, the test suites list will be updated accordingly.



12.1. Automated and Semi Automated Tests

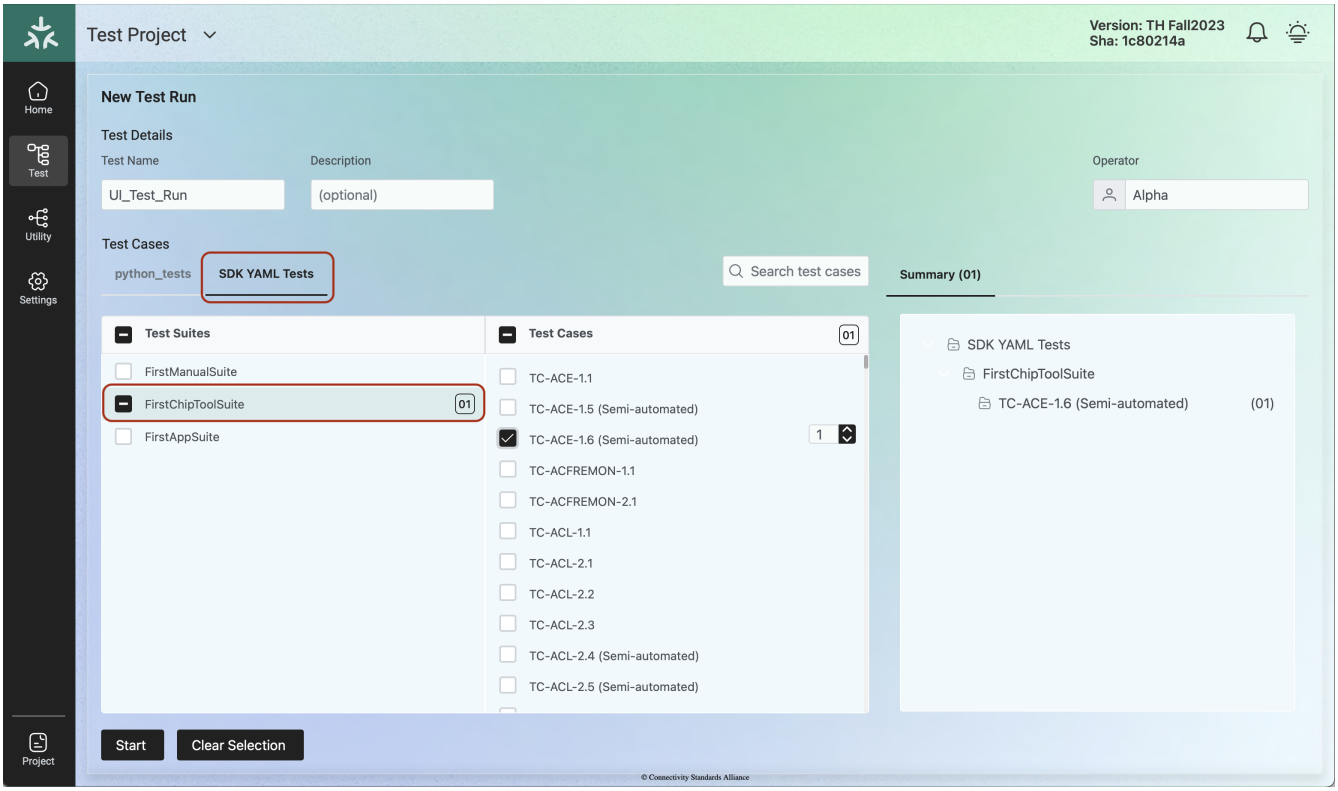
12.1.1. Automated Test Cases

Click on the **SDK YAML Tests** tab. The automated and semi automated test cases will be listed in **FirstChipToolSuite**. The Automated test cases will be listed as the TC-<Cluster>-XX without any suffix, e.g., TC-DRLK-1.1. Automated test case execution will not require any manual intervention.

12.1.2. Semi Automated Test Cases

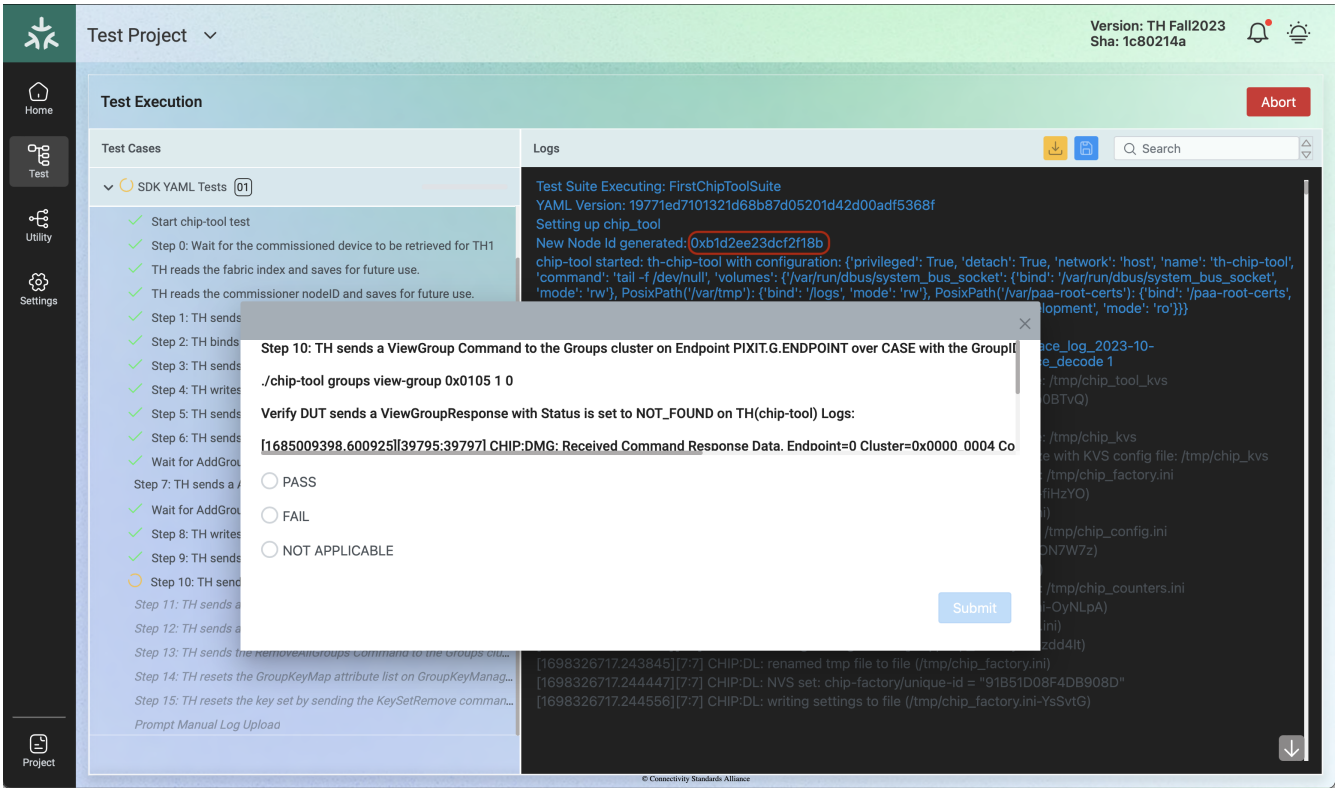
The Semi Automated test cases will be listed as TC-<Cluster>-XX(Semi-automated). During the Semi Automated test case execution, some of the steps will be executed automatically and the user will be prompted to perform a few steps as shown below in the screenshots. From the TH user interface, load the required PICS file to select the test cases, e.g., Doorlock Test Plan.xml.

Select the required Semi Automated test case to be executed and ensure other test cases are not selected. Take for example TC-ACE-1.6 as shown below:



Bring up the DUT (All Clusters as Server) by sending the following command `./chip-all-clusters-app` on the Raspberry Pi terminal and click on the **Start** button.

During the Test execution, as the log gets updated, copy the newly generated node ID.



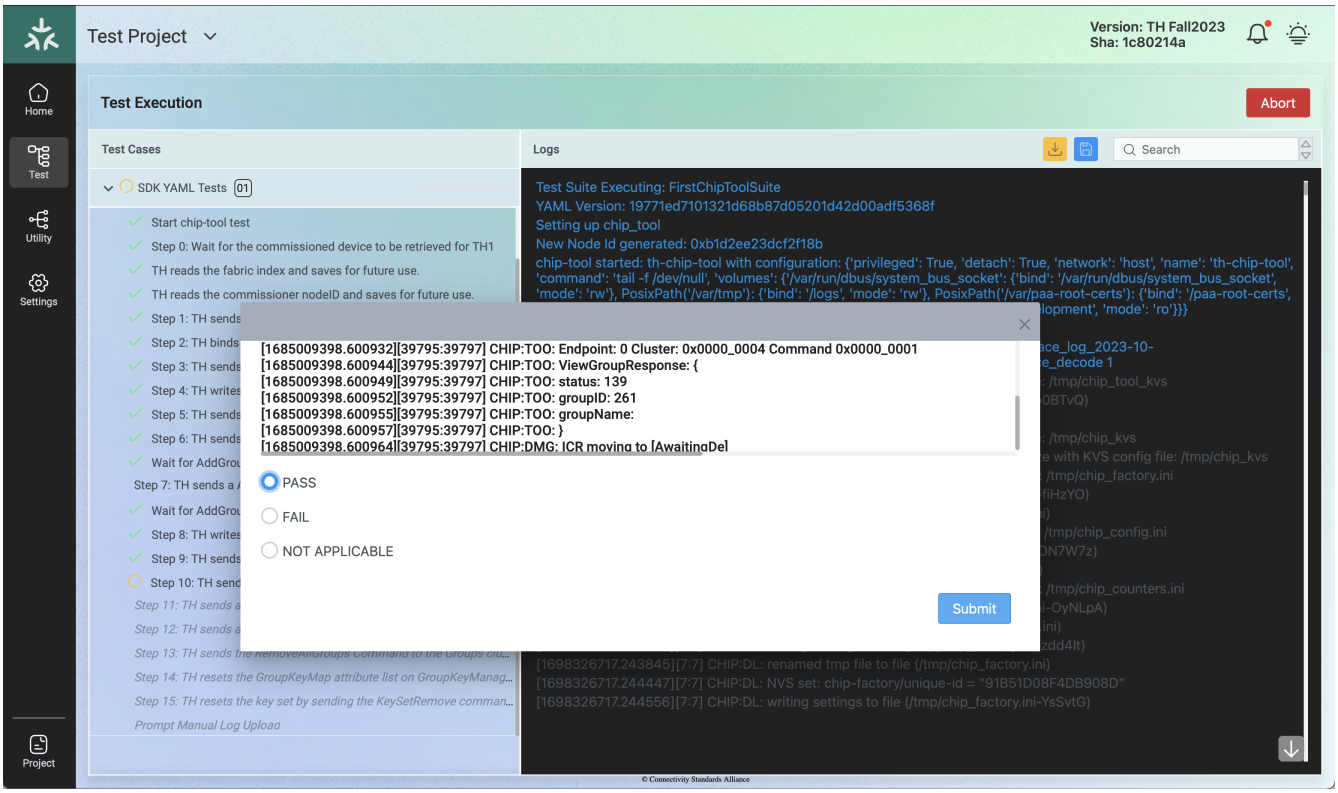
Form the Chip-tool, execute the above command with node ID listed in the TH log. Save the Chip-tool logs in a text file. Verify the result in the Chip-tool log and select the applicable choice from the user prompt in the TH tool and select the **Submit** button.

Example:

```
docker exec -it th-sdk <popup command> <newly generated nodeID> <end-point id>

cd apps
docker exec -it th-sdk ./chip-tool groups view-group 0x0105 0xb1d2ee23dcf2f18b 0
```

Check for the response of the command in the Chip-tool log and compare with the expected response from the TH user prompt as shown below. In case both the responses match, click on **PASS** followed by the **Submit** button.

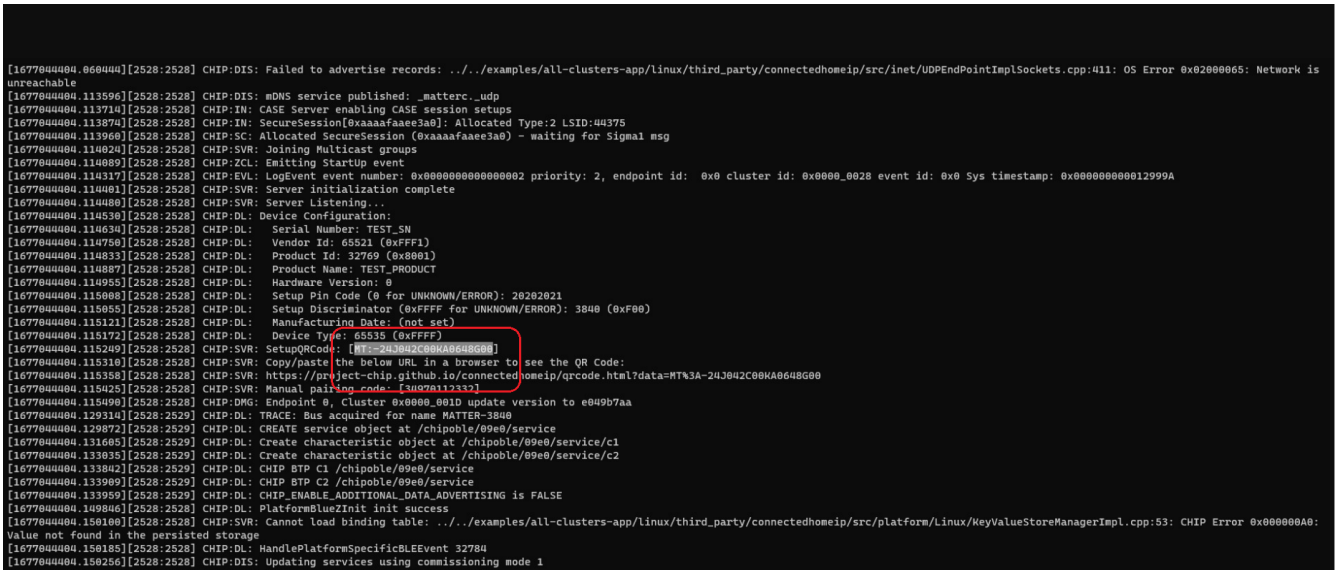


At the end of the test execution, the user will be prompted to upload the Chip-tool logs that were saved in the previous step.

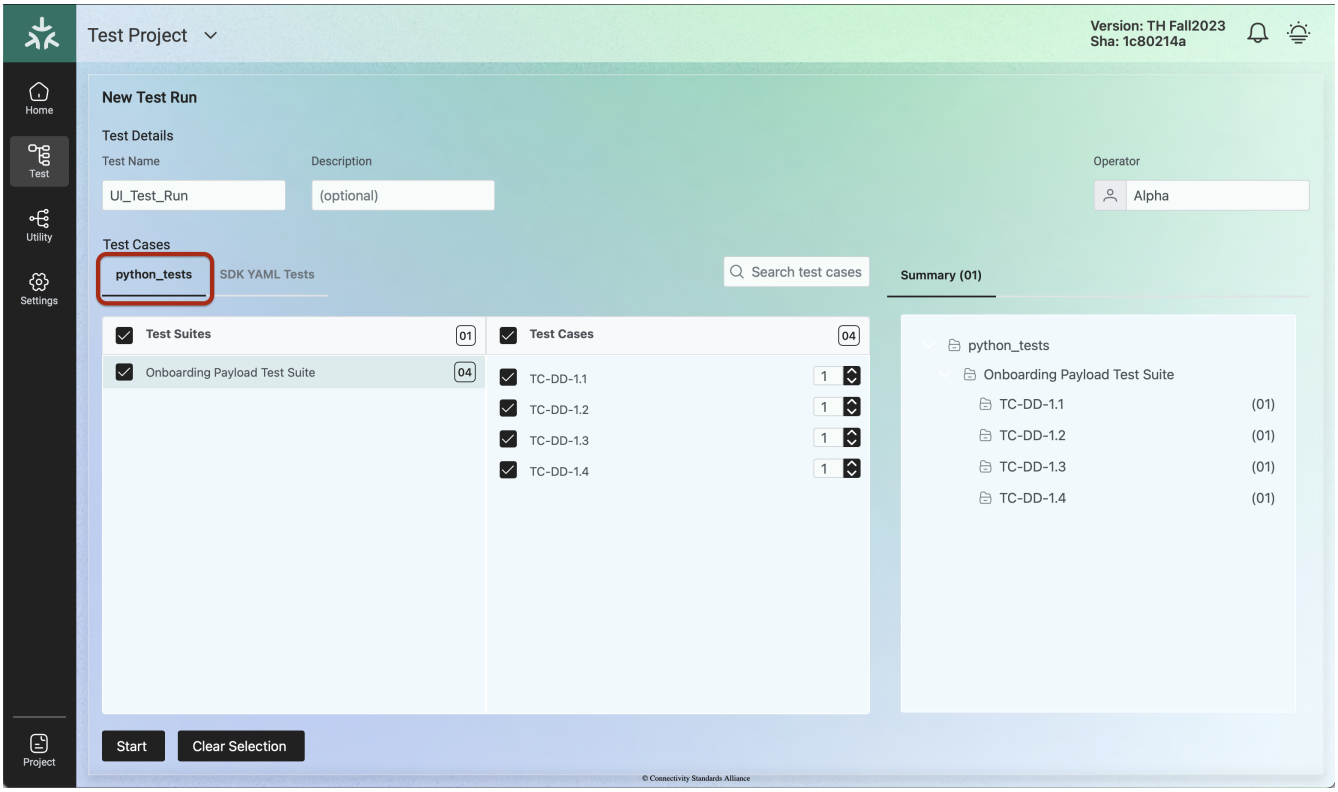
12.2. Python Tests

The Onboarding Payload Device Discovery test cases are listed under this option. Before executing the Python tests, bring up the DUT in the Chip-tool and save the discovery log. During the Python test execution, the user is prompted to input data such as QR code. Copy the data from the previously saved logs and provide the input. Follow the sequence below to execute the python_tests.

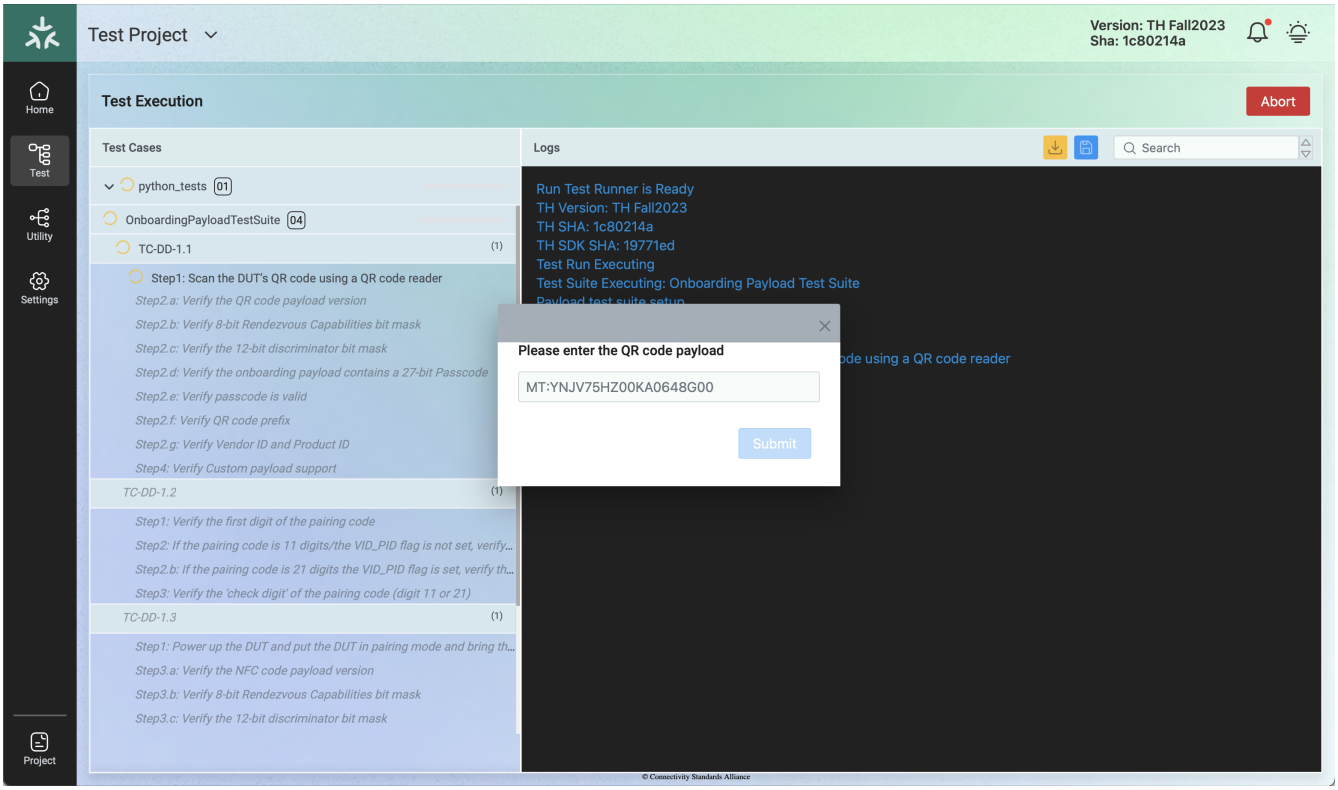
During the DUT bring-up, note down the QR code and save it for future use.



Select the python_tests tab for the test execution.

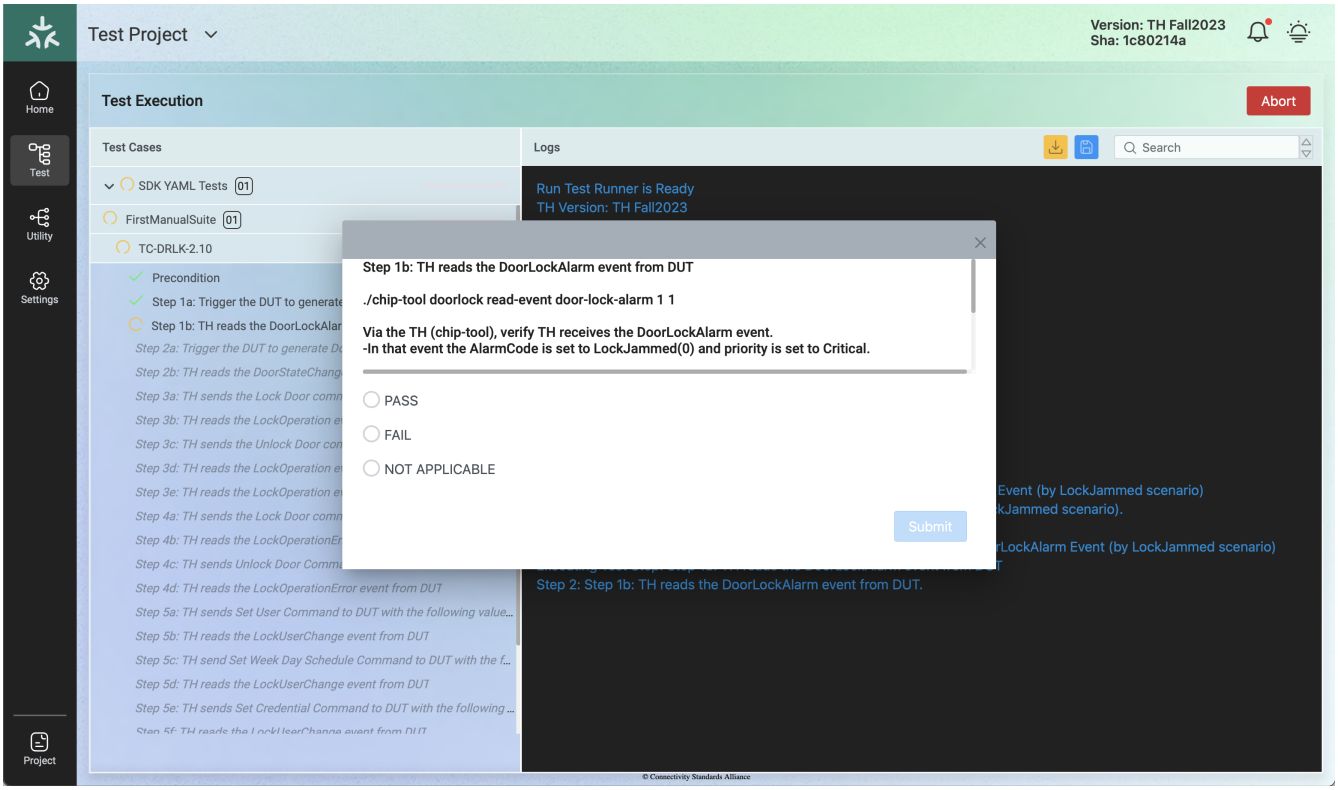


During the test execution the user is prompted for the QR code. Use the code that was saved earlier and proceed with the testing.



12.3. Manual Tests

During the manual test case execution, the user is prompted for an action for each test step as shown below.



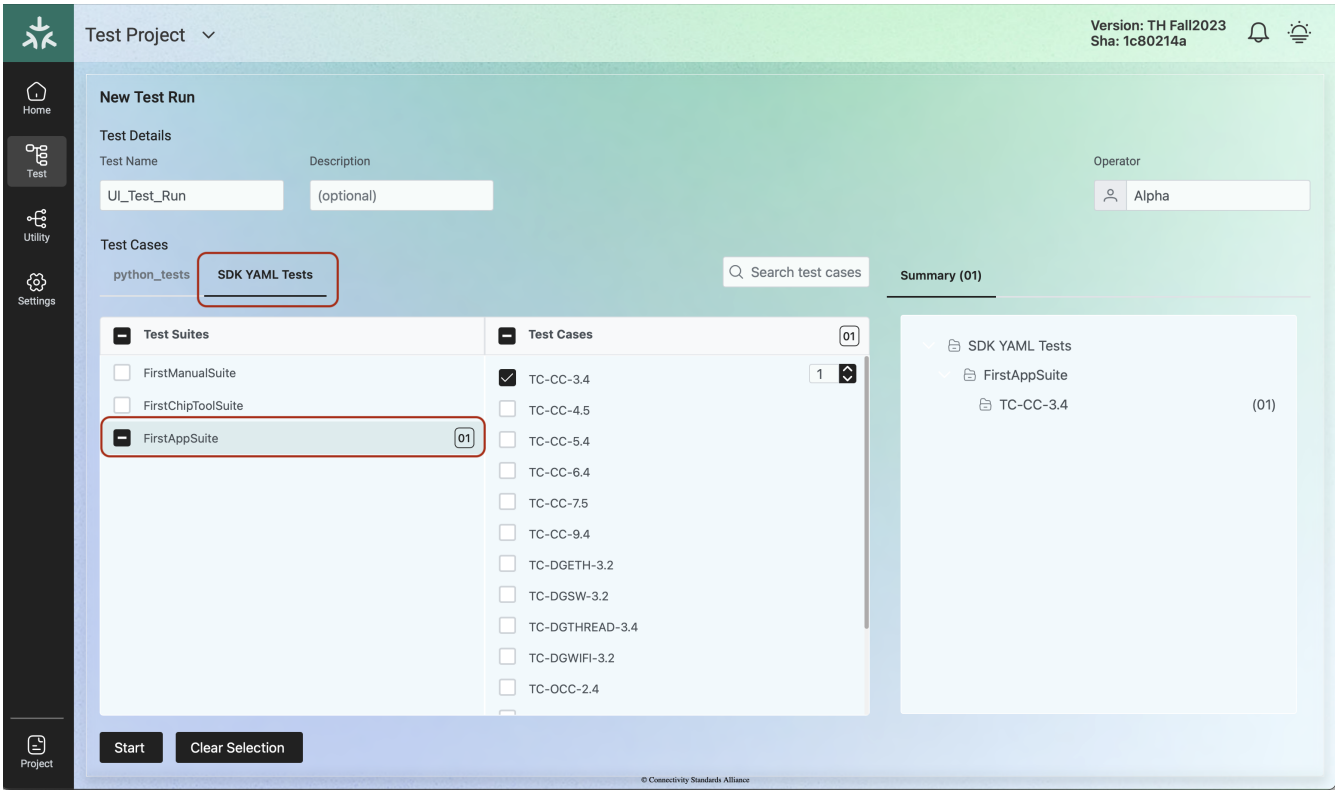
After the Manual pairing of the DUT, execute the command displayed on the prompt as shown below.

Example: `./apps/chip-tool doorlock read-event door_lock-alarm 1 1`

Save the Chip-tool logs in a text file. Validate the chip tool log and select the applicable choice from the user prompt in the TH tool and select the **Submit** button. At the end of the test execution, the user is prompted to upload the Chip-tool logs that were saved in the previous step.

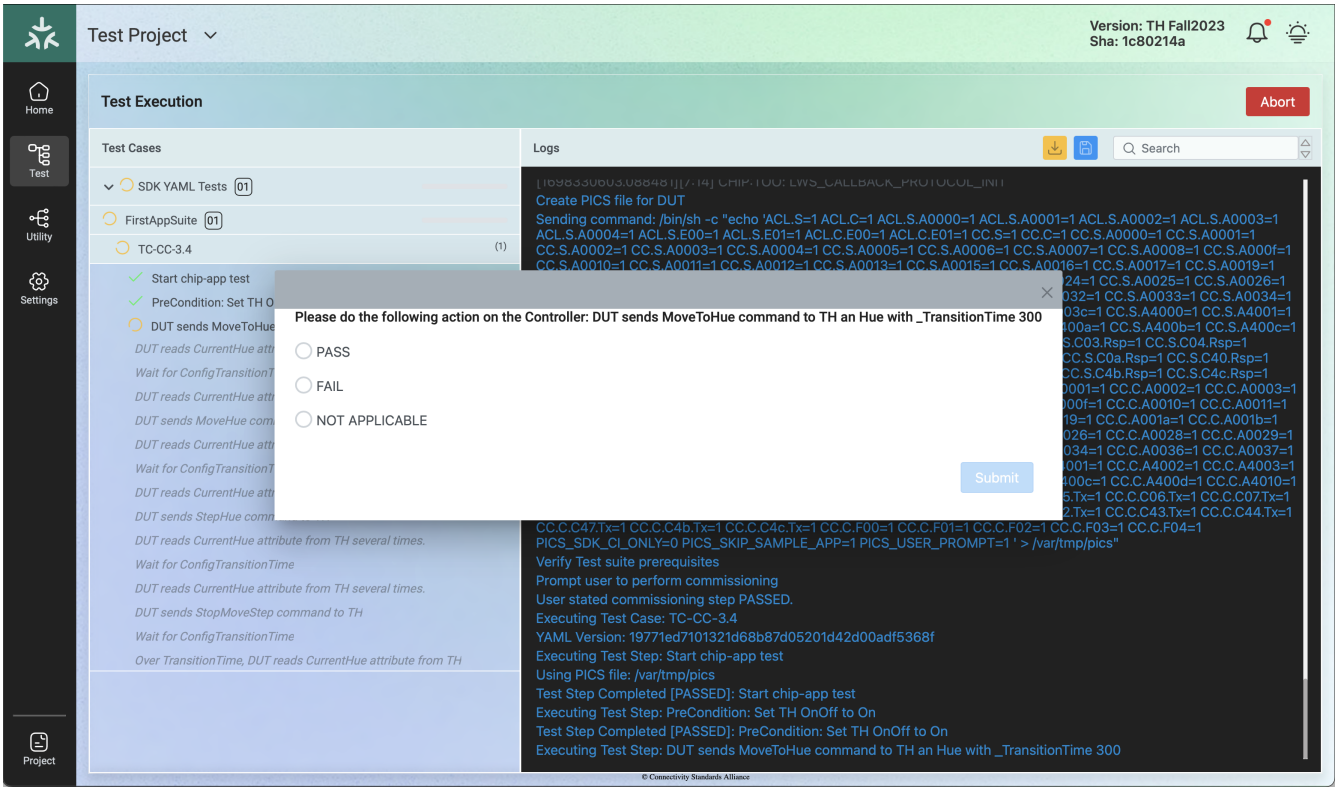
12.4. Simulated Tests

Simulated tests must be executed when the DUT is considered as a Client. The simulated test cases will be listed in **FirstAppSuite** under the **SDK YAML Tests** tab.



During the execution of these tests, the user is prompted for an action to be performed on the device as shown in the following screenshot.

Follow the instructions provided in the user prompt to complete the test execution.



IMPORTANT: Currently the selection will be done automatically by TH based on the test execution result. In the future the User Prompt will be updated to proper represent this behavior.

12.5. SDK Python Tests

12.5.1. Run Tests Inside SDK Docker Container

Some automated Python scripts are available inside the docker of the TH.

E.g.: TC_ACE_1_3.py, TC_ACE_1_4.py , TC_CGEN_2_4.py , TC_DA_1_7.py , TC_RR_1_1.py TC_SC_3_6.py

Follow the instructions below to execute the test cases.

12.5.1.1. Prerequisite

1. A directory containing the PAA (Product) roots that will be mounted as /paa_roots.
2. Run the following commands from the Raspberry Pi terminal.

```
cd certification-tool
./backend/test_collections/matter/scripts/update-paa-certs.sh
```

3. After execution of the above commands ensure that the PAA's are available locally at **/var/paa-root-certs** .

12.5.1.2. Mapped Volumes

The following host directories are mapped into the cert-bins container:

- `/root/python_testing → /home/ubuntu/certification-tool/backend/test_collections/matter/sdk_tests/sdk_checkout/python_testing`
- `/paa-root-certs → /var/paa-root-certs`
- `/credentials/development → /var/credentials/development`

12.5.1.3. Placeholders for Steps

Device-specific configuration is shown as shell variables. **PLEASE REPLACE THOSE WITH THE CORRECT VALUE** in the steps below.

- **$PATH_TO_PAA_ROOTS**: Path on host where PAA roots are located. Failure to provide a correct path will cause early failure during commissioning (e.g., /var/paa-root-certs/)
- **$DISCRIMINATOR**: Long discriminator for DUT (e.g., 3840 for Linux examples)
- **$SETUP_PASSCODE**: Setup passcode for DUT (e.g., 20202021 for Linux examples)
- **$WIFI_SSID**: SSID of Wi-Fi AP to which to attempt connection

Currently, WIFI_SSID with special characters or empty spaces is not supported.

- **$WIFI_PASSPHRASE**: Passphrase of Wi-Fi AP to which to attempt connection
- **$BLE_INTERFACE_ID**: Interface ID for BLE interface (e.g., 0 for default, which usually works)
- **$THREAD_DATASET_HEX**: Thread operational dataset as a hex string (e.g., output of dataset active -x in OpenThread CLI on an existing end-device

12.5.1.4. Common Steps

Factory-reset the DUT

```
docker run -v $PATH_TO_PAA_ROOTS:/paa_roots -v
/var/run/dbus/system_bus_socket:/var/run/dbus/system_bus_socket -v /home/ubuntu/certification-
tool/backend/test_collections/matter/sdk_tests/sdk_checkout/python_testing:/root/python_testin
g -v $(pwd):/launch_dir --privileged --network host -it connectedhomeip/chip-cert-bins:<SDK
SHA RECOMMENDED>
```

This downloads a Docker image with the test environment, and runs the environment including mounting the PAA trust store in **/paa_roots** and mounts the local Avahi socket so that Avahi in the VM can run against its host.

- You will be shown a # root prompt

The first time running docker will be SLOW (around 5 minutes) due to the need to download data. Every other run after that will be instant.

12.5.1.5. For NFC Pairing

The Test Harness implementation relies on a PC/SC (Personal Computer/Smart Card) driver. In our setup, the NFC device used for commissioning is exposed and managed as a contactless smart card.

A PC/SC reader must be connected to the Raspberry Pi.

Once the Docker container is running, start the PC/SC smartcard daemon with 'Polkit authorization disabled' using the following command:

```
pcscd --disable-polkit
```

The tests for NFC-based commissioning and NFC onboarding can then be executed.

12.5.1.6. For On-Network Pairing

Execute the following command:

```
rm -f admin_storage.json && python3 python_testing/scripts/sdk/TC_RR_1_1.py --discriminator
$DISCRIMINATOR --passcode $SETUP_PASSCODE --commissioning-method on-network --paa-trust-store
-path /paa_roots --storage-path admin_storage.json
```

To test this against a Linux target running on the same network as the host:

```
clear && rm -f kvs1 && ./chip-all-clusters-app --discriminator 3842 --KVS kvs1 --trace_decode
1
```

- The $DISCRIMINATOR to be used will be 3842 in this example.
- The **rm -f kvs1** is a factory reset.

12.5.1.7. For BLE+Wi-Fi Pairing

Execute the following command in the docker for the BLE+Wi-Fi pairing:

```
rm -f admin_storage.json && python3 python_testing/scripts/sdk/TC_RR_1_1.py --discriminator
$DISCRIMINATOR --passcode $SETUP_PASSCODE --commissioning-method ble-wifi --paa-trust-store
-path /paa_roots --storage-path admin_storage.json --wifi-ssid $WIFI_SSID --wifi-passphrase
$WIFI_PASSPHRASE --ble-interface-id $BLE_INTERFACE_ID
```

12.5.1.8. For BLE+Thread Pairing

Execute the below command in the docker for the BLE+Thread pairing:

```
rm -f admin_storage.json && python3 python_testing/scripts/sdk/TC_RR_1_1.py --discriminator
$DISCRIMINATOR --passcode $SETUP_PASSCODE --commissioning-method ble-thread --paa-trust-store
-path /paa_roots --storage-path admin_storage.json --thread-dataset-hex $THREAD_DATASET_HEX
--ble-interface-id $BLE_INTERFACE_ID
```

12.5.1.9. For CNET Tests

CNET (Network Commissioning cluster) test cases manage Wi-Fi networks directly through the host's `wpa_supplicant` control interface, so the `chip-cert-bins` container needs an additional volume mapping for `/var/run/wpa_supplicant` on top of the Common Steps command:

```
docker run -v $PATH_TO_PAA_ROOTS:/paa_roots -v
/var/run/dbus/system_bus_socket:/var/run/dbus/system_bus_socket -v /home/ubuntu/certification-
tool/backend/test_collections/matter/sdk_tests/sdk_checkout/python_testing:/root/python_testin
g -v $(pwd):/launch_dir -v /var/run/wpa_supplicant:/var/run/wpa_supplicant --privileged
--network host -it connectedhomeip/chip-cert-bins:<SDK SHA RECOMMENDED>
```

Without this mapping, CNET test cases will fail to reach `wpa_supplicant` and cannot add, update, or remove Wi-Fi networks.

12.5.1.10. Post-Test Steps

Factory reset the DUT again → The test fills tons of stuff and the device will be in an odd state of ACL's. This will be fixed once there is ample time to clean up after the test is completed by sending commands to, for example, remove the fabrics joined.

12.5.1.11. Possible Issues

- Failing at Step 9 during execution of TC_RR_1_1:
  a. Some DUT's have an incorrectly-configured UserLabel cluster where the backend is not implemented due to SDK example issues where some examples have the backend and others do not. This will fail at the last step ("Step 9: Fill UserLabel clusters on each endpoint"), with FAILURE writes. To override the test not to run this step, you can add "**--bool-arg skip_user_label_cluster_steps:true**" to the command line of **TC_RR_1_1.py**, at the end.
  b. Not having the **$PATH_TO_PAA_ROOTS** set properly when starting the docker or not having PAA roots certificates at that path.
  c. Follow the instructions for item 2 in Section 9.5.1.1, Prerequisite.

Common Test Failures

The documents in this link are intended to be used to help root-cause common test failures, especially in cases where the underlying cause of the failure may not be immediately obvious from the test step or expected outcomes.

12.5.2. Run Tests on the TH User Interface

Some automated Python scripts are available in TH User Interface.

To execute the tests, the parameters **discriminator**, **setup_code** and **pairing_mode** need to be filled in the device configuration parameters (**dut_config**).

To configure specific/custom parameters, please edit the project configuration to include the parameters in the session (**test_parameters**).

Project configuration example:

```
{
  ...
  "dut_config": {
    "discriminator": "3840",
    "setup_code": "20202021",
    "pairing_mode": "onnetwork",
    "chip_tool_timeout": null,
    "chip_tool_use_paa_certs": false
  },
  "test_parameters": {
    "paa-trust-store-path": "/credentials/development/paa_roots",
    "storage-path": "admin_storage.json"
  }
  ...
}
```

12.5.2.1. Test Parameters for SDK Python Tests

PIXIT Support

PIXIT type parameters must be filled in the **test_parameters** section. The following example will be used to define the following parameters:

```
PIXIT.ACE.APPENDPOINT:1
PIXIT.ACE.APPDEVTYPEID:256
PIXIT.ACE.APPCLUSTER:OnOff
PIXIT.ACE.APPATTRIBUTE:OnOff
```

Project configuration example:

```
{
  ...
  "test_parameters": {
    "paa-trust-store-path": "/credentials/development/paa_roots",
    "storage-path": "admin_storage.json",
    "int-arg": "PIXIT.ACE.APPENDPOINT:1 PIXIT.ACE.APPDEVTYPEID:256",
    "string-arg": "PIXIT.ACE.APPCLUSTER:OnOff PIXIT.ACE.APPATTRIBUTE:OnOff"
  }
  ...
}
```

The above example will be used to define the following arguments when running the test:

```
--int-arg PIXIT.ACE.APPENDPOINT:1 PIXIT.ACE.APPDEVTYPEID:256 --string-arg
PIXIT.ACE.APPCLUSTER:OnOff PIXIT.ACE.APPATTRIBUTE:OnOff
```

Test Parameters Examples

Access the spreadsheet via the Verification Steps Document and review the information provided. Based on this data, create the parameters set as requested.

Below are some specific examples assembled from data obtained from the spreadsheet.

1. TC-ACE-1.4

   Sample command to run manually inside docker

   ```
   python3 TC_ACE_1_4.py --discriminator 3840 --passcode 20202021 --commissioning
   -method on-network --storage-path admin_storage.json  --int-arg
   PIXIT.ACE.APPENDPOINT:1 PIXIT.ACE.APPDEVTYPEID:256 --string-arg
   PIXIT.ACE.APPCLUSTER:OnOff PIXIT.ACE.APPATTRIBUTE:OnOff  --paa-trust-store-path
   /credentials/development/paa-root-certs/
   ```

   Arguments to be used while executing using UI (use product specific values)

   ```
   "test_parameters": {
     "int-arg" : "PIXIT.ACE.APPENDPOINT:1 PIXIT.ACE.APPDEVTYPEID:256",
     "string-arg": "PIXIT.ACE.APPCLUSTER:OnOff PIXIT.ACE.APPATTRIBUTE:OnOff"
   }
   ```

2. TC-SC-7.1

   Sample command to run manually inside docker

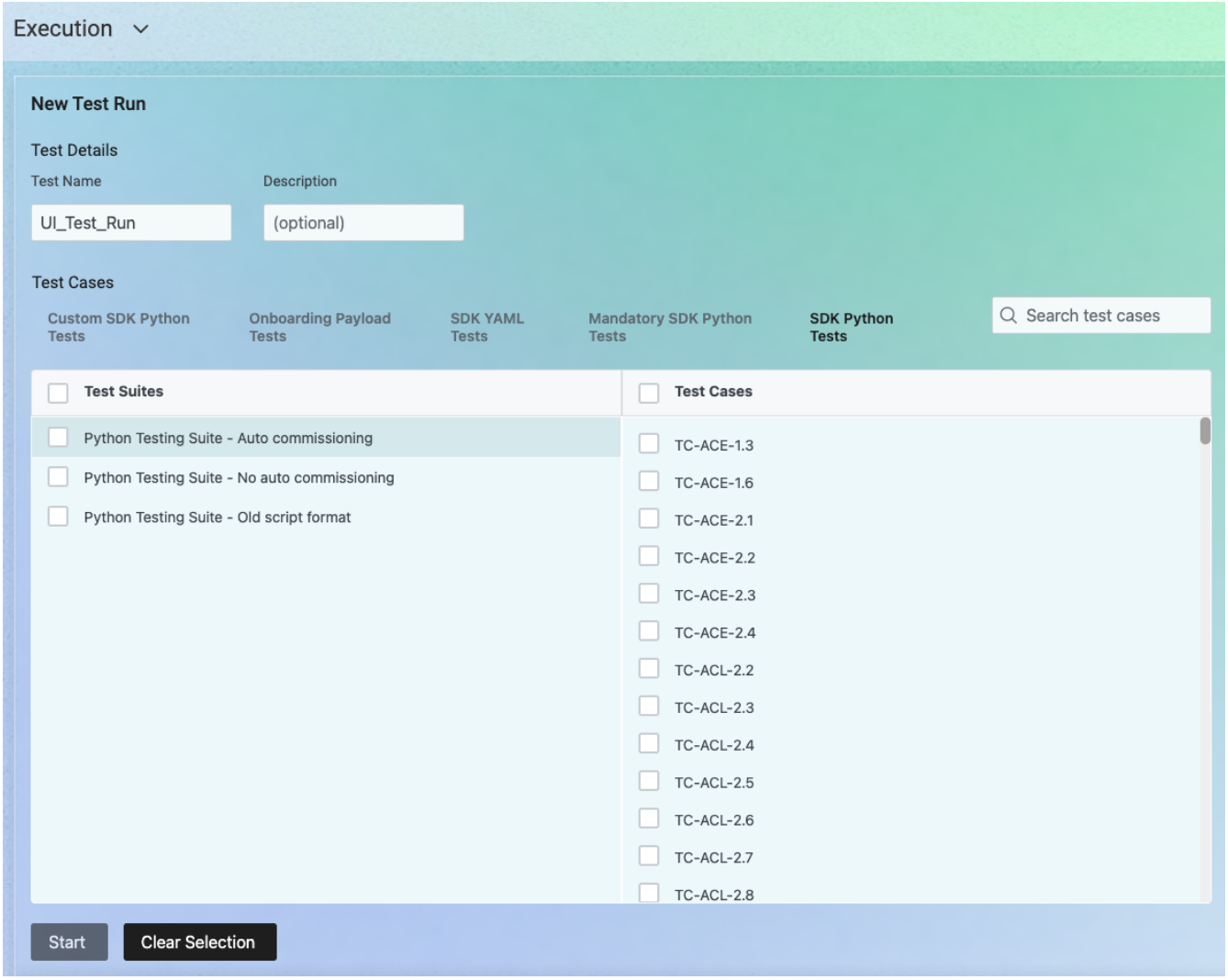
   ```
   python3 TC_SC_7_1.py --bool-arg post_cert_test:true  --qr-code MT:-
   <24J0CEK01KA0648G00> --storage-path admin_storage.json --paa-trust-store-path
   ../../credentials/development/paa-root-certs/
   ```

   Arguments to be used while executing using UI (use product specific values)

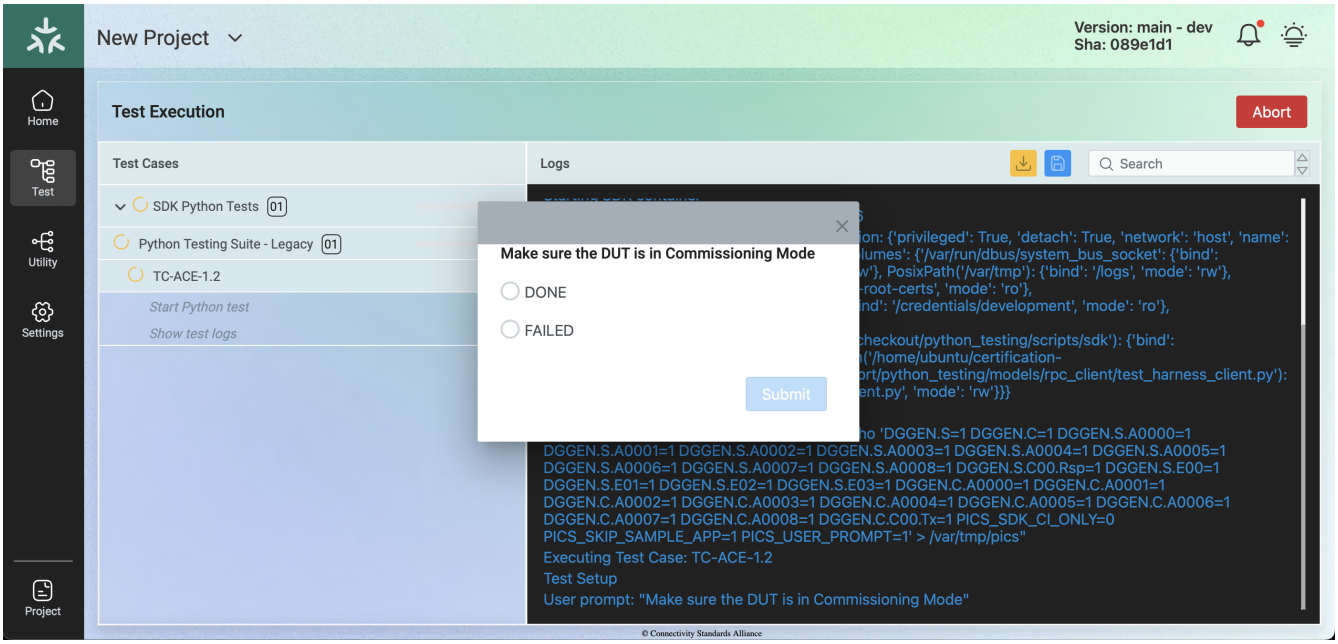
   ```
   "test_parameters": {
     "bool-arg": "post_cert_test:true",
     "qr-code": " MT-24J042C00KA0648G00"
   }
   ```

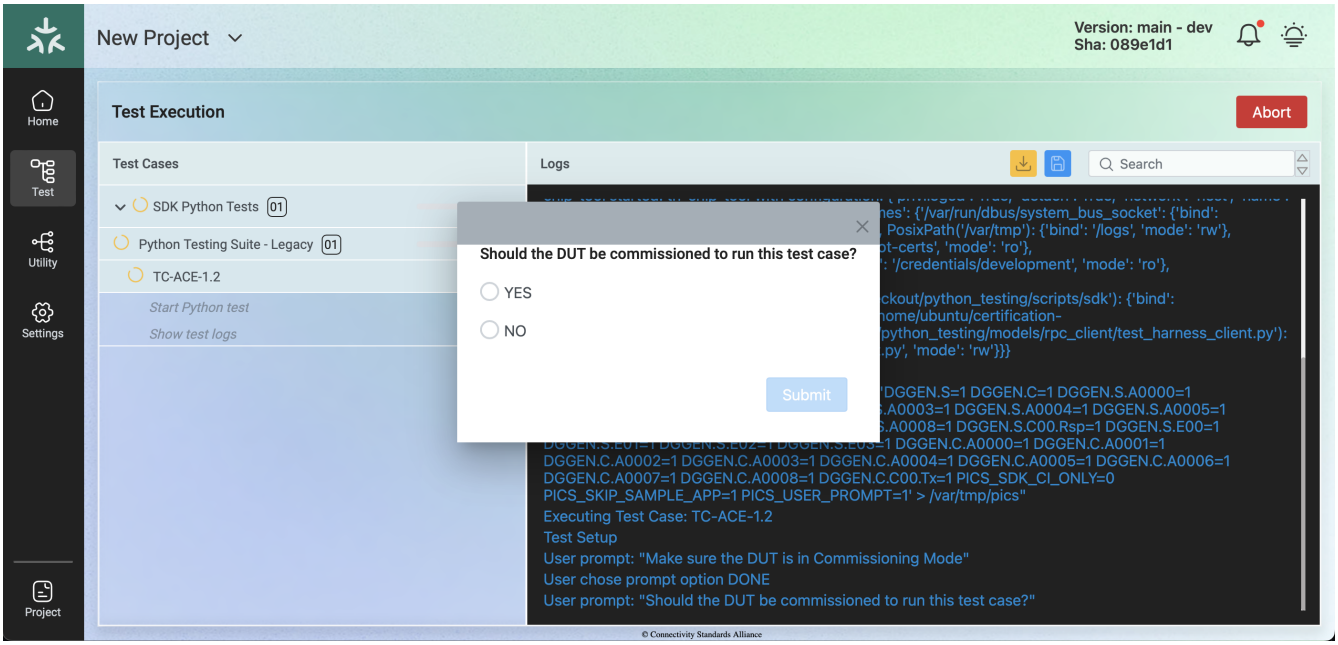
12.5.2.2. Test suites

TH expects the SDK Python Tests to follow a certain template. New tests are being written with this template and the old tests are being updated to conform to it. The tests are divided in 3 test suites:



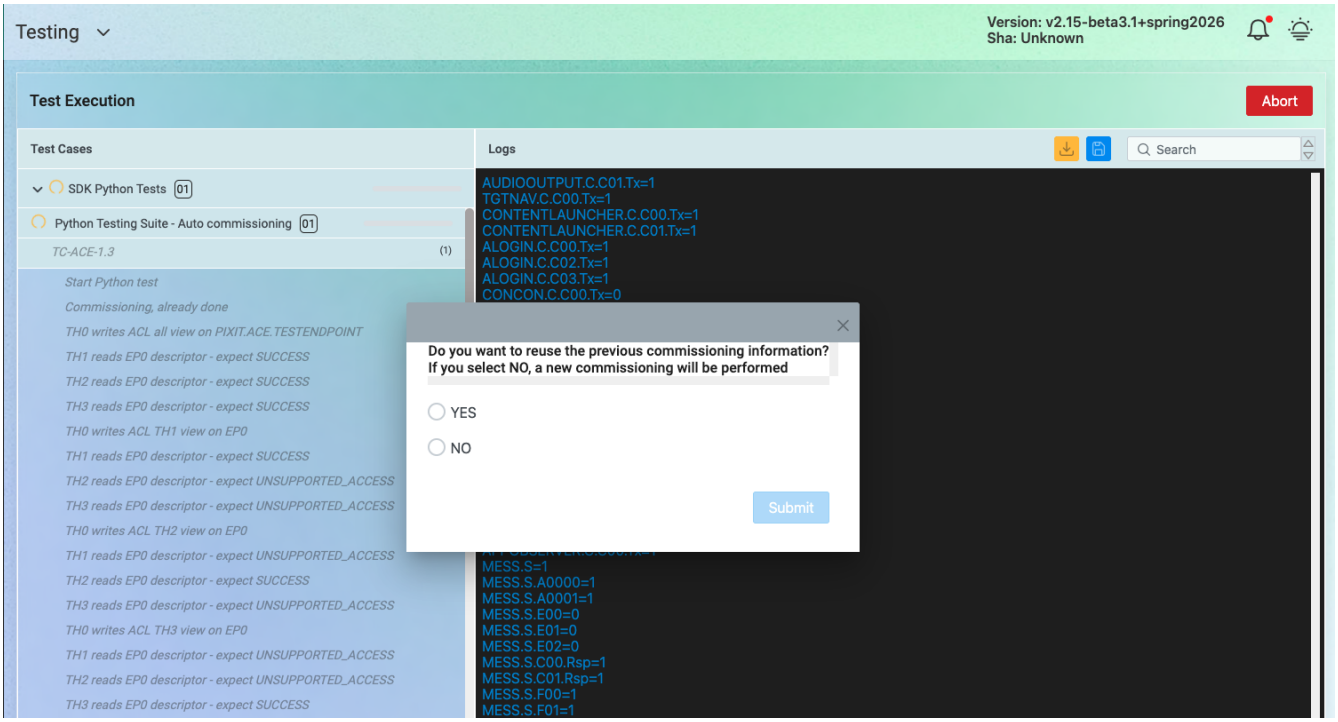
1. Python Testing Suite - Auto commissioning
    - For test cases that follow the expected template and have a commissioning first step.
    - The user will be asked to make sure that the DUT is in Commissioning Mode at the start of the test suite setup and then the DUT will be commissioned.
    - The commissioning will be kept throughout the execution of all its tests.
2. Python Testing Suite - No auto commissioning
    - For test cases that follow the expected template but don't have a commissioning first step.
    - The selected tests will be executed without commissioning the DUT.
    - The user will be asked to make sure that the DUT is in Commissioning Mode at the start of each test.
3. Python Testing Suite - Old script format
    - For test cases that don't follow the expected template yet.
    - The user will be asked to make sure that the DUT is in Commissioning Mode at the start of each test.
    - The user will also be asked if the DUT should be commissioned at the start of each test. The DUT will be commissioned depending on the user's answer.





12.5.2.3. Reuse commissioning information

This allows users to perform multiple test run executions without the need to perform the commissioning step in every test run execution. The TH is now storing the last commissioning information, so a prompt will presented asking user to reuse those previous commissioning information or if he wants to perform a new commissioning procedure.



12.5.2.4. Python Test Logging Configuration

The TH supports two modes for displaying Python test logs:

- **Real-time logging (default)**: Logs are displayed incrementally as each test step completes
- **Batch logging**: All logs are displayed at once after test execution completes

The logging mode is configured per project with `th_config.enable_realtime_python_test_logs` in the project's configuration (see User Prompt Timeout Configuration for where `th_config` lives in the project JSON). This takes effect immediately on the next test run — no container restart needed:

```
"th_config": {
  "enable_realtime_python_test_logs": false
}
```

Set it to `false` to use batch logging for that project. Set it to `true`, or leave it `null`/omit the key, to use real-time logging (default).

12.5.2.5. Container Logging Configuration

The TH supports optional logging of container operations for debugging and troubleshooting purposes:

- **Container logging disabled (default)**: No container operation logs are displayed
- **Container logging enabled**: Displays detailed Docker container operations including creation, destruction, and file copy operations

The logging mode is configured per project with `th_config.enable_container_logs` in the project's configuration (see User Prompt Timeout Configuration for where `th_config` lives in the project JSON). This takes effect immediately on the next test run — no container restart needed:

```
"th_config": {
  "enable_container_logs": true
}
```

Set it to `true` to enable container logging for that project. Set it to `false`, or leave it `null`/omit the key, to keep it disabled (default).

When enabled, the following container information will be logged:

- Container creation and docker run commands
- Container destruction (kill and remove commands)
- File copy operations between host and container
- Container execution commands

12.5.2.6. Helper App Binaries in the SDK Container

Some tests launch helper apps from inside the SDK container, for example the OTA Software Update tests TC-SU-2.2 and TC-SU-2.5, which run `chip-ota-provider-app` from the `provider_app_path` test parameter. The binary must exist *inside* the SDK container.

Set `th_config.sdk_container_apps_dir` to a folder on the TH host to mount it read-only at `/launch_dir/apps` in the SDK container:

```
"th_config": {
  "sdk_container_apps_dir": "/home/ubuntu/apps"
}
```

With the binary at `/home/ubuntu/apps/chip-ota-provider-app`, pass `provider_app_path:/launch_dir/apps/chip-ota-provider-app`, for example `th-cli run-tests -t TC-SU-2.2 --config my-config.json --string-arg provider_app_path:/launch_dir/apps/chip-ota-provider-app`.

- The path is resolved on the Docker host, so the TH cannot check that it exists. Docker creates an empty folder if it does not, and the test then fails with `Invalid provider_app_path`. The mount is logged when the SDK container starts.
- If `sdk_container_apps_dir` is `null` or not set, the TH falls back to the `SDK_CONTAINER_APPS_DIR` environment variable in `certification-tool/.env`. If neither is set, nothing is mounted.
- The SDK container is recreated for each test suite run, so a changed setting applies on the next run.

Without this setting, copy the binary to `/var/tmp` on the TH host, which is mapped to `/logs` in the SDK container, and use `provider_app_path:/logs/chip-ota-provider-app`.

12.5.2.7. User Prompt Timeout Configuration

During test execution, the TH sometimes prompts the operator for input or confirmation (e.g. to pair the DUT, enter a setup code, or confirm a commissioning step). By default, the operator has 60 seconds to respond before the prompt times out.

This timeout is configurable per project via a `th_config` object in the project's configuration JSON — the same file edited via the **Project Config** field described earlier in this section, and used with `th-cli project create/update --config` and `th-cli run-tests --config`. `th_config` sits alongside `network`, `dut_config`, and `test_parameters`:

```
"th_config": {
  "prompt_timeout_seconds": 60,
  "enable_realtime_python_test_logs": null,
  "enable_container_logs": null,
  "sdk_container_apps_dir": null
}
```

- `prompt_timeout_seconds`: how long, in seconds, the operator has to respond to a user prompt before it times out. Increase this if a test step requires the operator to read output and take a manual action that takes longer than the default 60 seconds.
- `enable_realtime_python_test_logs`: the per-project override for Python test log display described in Python Test Logging Configuration above.
- `enable_container_logs`: the per-project override for Docker container operation logging described in Container Logging Configuration above.
- `sdk_container_apps_dir`: a folder on the TH host with helper app binaries (e.g. `chip-ota-provider-app`), mounted read-only at `/launch_dir/apps` in the SDK container. See Helper App Binaries in the SDK Container below.

`th_config.prompt_timeout_seconds` controls the interactive user-prompt timeout. It is unrelated to `test_parameters.timeout`, which controls the chip-tool/Python test **execution** timeout (see the "Overwrite the default timeout" step under Project Configuration, above) — the two settings sit close together in the same JSON file but govern different things.

When using the CLI, the prompt timeout can also be overridden for a single run without editing the project configuration:

```
th-cli run-tests --tests-list TC-ACE-1.1 --prompt-timeout 300
```

13. Matter Test-Harness Command Line Interface (CLI)

13.1. Overview

The Matter Test-Harness Command Line Interface (CLI) provides a powerful command-line tool for automated test execution, project management, and test monitoring. The CLI has been significantly enhanced with improved usability features including colorized output, auto-completion, camera test support, and real-time execution monitoring.

Key Features:

- Enhanced output formatting with JSON/YAML options
- Automatic Manual Pairing Code generation and display
- Colorized terminal output for better readability
- Command auto-completion support for faster workflow
- Real-time test execution log monitoring
- Camera test support for video verification tests
- Node ID display in output for commissioning
- Advanced project management capabilities
- Improved PICS parser with better error handling
- Abort command to stop running tests

13.2. Quick-Start Guide

This quick guide shows you how to start testing in 5 minutes:

1. Make sure the CLI is installed:

   The CLI is installed automatically by the Test-Harness `auto-install.sh` and `auto-update.sh` scripts (see Section 4.1.2, TH Installation on Raspberry Pi and Section 4.4, Update Existing TH), so on the machine that runs the Test-Harness there is nothing else to install. To confirm, open a new terminal and run:

   ```
   th-cli --version
   ```

   If the command is not found, or if you want to install or update the CLI on its own (for example, on a machine other than the TH), run the CLI install script from a full clone of the Test-Harness repository:

   ```
   cd ~/certification-tool/cli
   ```

```
./scripts/th_cli_install.sh
```

The script needs the whole repository (not only the `cli` folder), because it copies a shared constants file from the `backend` folder. It also needs Poetry, which the Test-Harness install scripts provide, and installs `pipx` when it is missing. If `th-cli` is not found right after the installation, open a new terminal so the updated `PATH` is loaded.

2. Configure connection to Test-Harness:

   Edit `~/certification-tool/cli/config.json`:

   ```
   {
       "hostname": "192.168.1.100"
   }
   ```

   The `hostname` field in `config.json` tells the CLI where to find the Test Harness (TH) backend — it is **not** the address of the DUT.

   If `th-cli` is installed on a **separate machine from the TH** (e.g. a laptop connecting to a TH running on a lab Raspberry Pi or server), set `hostname` to the TH host's actual IP address, as shown in the above example (`192.168.1.100`).

   If TH and `th-cli` are running on the **same machine**, since everything is local, this step is not needed - the CLI defaults to `hostname: "localhost"` automatically.

   It is advisable to run TH and `th-cli` on the same machine.

3. View available tests:

   ```
   th-cli available-tests
   ```

4. Run your first test:

   ```
   th-cli run-tests --tests-list TC-ACE-1.1
   ```

5. View test results:

   The output will display real-time logs with color-coded test results.

13.3. Essential Commands

13.3.1. Check Version and Status

```
# Display CLI and Test-Harness versions
th-cli --version

# Check Test-Harness runner status
th-cli test-runner-status

# Check Test-Harness runner status in JSON format
th-cli test-runner-status --json
```

`th-cli --version` prints the CLI's own version and git SHA, followed by a Backend/SDK version breakdown fetched from the server: the backend version and SHA, the SDK SHA and Docker tag, and the database revision. If the server is unreachable, the CLI still prints its own version and reports that server versions could not be retrieved.

13.3.2. View Available Tests

```
# List all tests (YAML format - default)
th-cli available-tests

# List tests with compact format
th-cli available-tests --compact

# List tests in JSON format
th-cli available-tests --json

# Filter by cluster
th-cli available-tests --cluster ACE

# Group test cases by cluster, 4 per line
th-cli available-tests --group-by-cluster

# Filter by test case
th-cli available-tests | grep "TC-ACE"
```

13.3.3. Rescan Test Collections

Re-run test collection discovery on the backend, without restarting it. This picks up newly added or edited side-loaded custom test scripts (see Rescanning Side-Loaded Test Cases).

```
th-cli rescan-tests
```

The output lists the test cases that were added, modified or removed, and any scripts that could not be parsed. See Rescanning Side-Loaded Test Cases for an example.

This fails with an error if a test run is currently executing. Wait for it to finish, or run `th-cli abort-testing`, then try again.

13.3.4. Running Tests

Basic test execution:

```
# Run single test
th-cli run-tests --tests-list TC-ACE-1.1

# Run multiple tests
th-cli run-tests --tests-list TC-ACE-1.1,TC-ACE-1.2,TC-DRLK-1.1

# Run with custom title
th-cli run-tests --tests-list TC-ACE-1.1 --title "Door Lock Test Run"
```

With configuration files:

```
# Use custom project configuration file
th-cli run-tests --tests-list TC-ACE-1.1 --config my_config.json

# Use PICS configuration
th-cli run-tests --tests-list TC-ACE-1.1 --pics-config-folder ./pics_files/

# Override the user-prompt timeout for this run only (seconds)
th-cli run-tests --tests-list TC-ACE-1.1 --prompt-timeout 300

# Assign to existing project
th-cli run-tests --tests-list TC-ACE-1.1 --project-id 5

# Disable the real-time web log viewer for this run
th-cli run-tests --tests-list TC-ACE-1.1 --no-streaming
```

`--no-streaming` disables the browser-based real-time log viewer (enabled by default, normally reachable at a URL printed at the start of the run); logs are still written to the local log file either way.

If the **project-id** is not specified, a default project called **CLI Project Execution** will be used (**or** created if not presented) for the test run. Please make sure to use the appropriate project-id if you want to assign the test run to a specific project.
You can verify to the existing projects using the `th-cli project list` command, referred at Section 11.3.4, Project Management.

`--prompt-timeout` is execution-only — it overrides `th_config.prompt_timeout_seconds` (see User Prompt Timeout Configuration) for this run without persisting the change to the project's configuration.

13.3.5. TC Parameters Mapping File (`--tc-params-file` / `-m`)

When running multiple test cases that each require different PIXIT parameters (e.g. `endpoint`, `int-`

`arg`, `string-arg`), you can consolidate all parameter definitions into a single JSON mapping file instead of manually updating the project configuration before each run.

The mapping file is a JSON object whose keys are TC IDs and whose values are `test_parameters` dicts, using the same format accepted by the project configuration.

Mapping file format:

```
{
  "TC-ACE-1.1":      { "int-arg": "PIXIT.ACE.APPENDPOINT:1", "timeout": "300" },
  "TC-ACE-1.3":      { "endpoint": "1" },
  "TC-MCORE-FS-1.3": { "string-arg": "PIXIT.MCORE.ENDPOINT:3" },
  "TC-AVSM-2.10":    { "int-arg": "PIXIT.AVSM.ENDPOINT:1 PIXIT.AVSM.STREAMTYPE:1" }
}
```

Usage:

```
# Run a single test, supplying its parameters from the mapping file
th-cli run-tests --tests-list TC-ACE-1.3 --project-id 10 -m mapping_tc.json

# Run multiple tests; only the entries present in the mapping file are applied
th-cli run-tests --tests-list TC-ACE-1.1,TC-ACE-1.3,TC-MCORE-FS-1.3 -m mapping_tc.json
```

Parameter priority (lowest → highest):

Source	Description
Project configuration	Values stored in the TH project (persistent baseline)
`--tc-params-file`	Parameters from the mapping file — override the project config for matched TC IDs
`--config` file	Execution-only config file passed via `--config` — overrides the mapping file
`-- inline args`	Arguments passed after `--` on the command line — always take precedence

This means the mapping file values **win over the project config** but are still overridden by an explicit `--config` file or inline `--` arguments, giving you full flexibility to fine-tune individual runs without touching the project.

13.3.6. Project Management

Create project:

```
# Create new project with name
th-cli project create --name "My DUT Tests"
```

```
# Create with project configuration file
th-cli project create --name "Door Lock Project" --config project_config.json
```

Tip:
The project configuration structure from an existing project may be verified using the following CLI command `th-cli project list --id <PROJECT-ID> --json`.
That way it's possible to create new projects and update based on existing configurations.

List projects:

```
# List all projects
th-cli project list

# List with JSON output
th-cli project list --json

# View specific project details
th-cli project list --id 5
```

Update and delete:

```
# Update project configuration
th-cli project update --id 5 --config updated_config.json

# Delete project
th-cli project delete --id 5
```

Edit project config interactively:

```
# Open the project's current config in your local editor and save to persist changes
th-cli project edit --id 5
```

Use this to tweak one or two fields (e.g. a `th_config` or `dut_config` value) without exporting, hand-editing, and re-importing a full config file. The editor opens via `$EDITOR`/`$VISUAL` (falling back to `vim`/`nano`/`vi`); on save, the CLI validates your changes, warns before persisting any field it doesn't recognize, and reopens the editor with your edit preserved if something's wrong. Every configuration field, and how this command compares with editing the configuration in the web UI, is described in Section 8.3, Project Configuration Reference.

Tip:
For the full behavior — retry handling on invalid input, the new-field confirmation prompt, and how to pick a different editor via `$VISUAL`/`$EDITOR` — run `th-cli project edit --help` or see the CLI README.

List the test cases applicable to a project:

```
# List the test cases that apply to project 5 (one ID per line, sorted)
th-cli project applicable-tests --id 5

# Print the raw JSON response instead
th-cli project applicable-tests --id 5 --json
```

The list is filtered by the project's PICS configuration and its `dmp_test_skip` setting. This differs from `th-cli available-tests`, which lists every test case known to the Test Harness regardless of project: a test that requires a PICS item disabled in the project appears in `available-tests` but not in `project applicable-tests`. A header line (`Applicable test case(s) for project <id>:`) is printed to stderr, so the IDs on stdout can be piped (e.g. `th-cli project applicable-tests --id 5 | grep TC-ACL`).

Download project logs:

```
# Download all logs for a project as a flat zip archive (one .log file per test run
execution)
th-cli project logs --id 5

# Download logs grouped by test case state (one inner .zip per execution)
th-cli project logs --id 5 --grouped

# Save to a specific output file instead of the default '<project-name>-logs.zip'
th-cli project logs --id 5 --output-file project5-logs.zip
```

This command downloads a single zip archive containing the logs for **every** test run execution belonging to the project, which can take a while for projects with many or large executions — the CLI prints a notice while the download is in progress.
If the project has no test run executions yet, the command reports that there is nothing to download instead of writing an empty zip file.

Export and import a project:

The `project export` and `project import` subcommands export a project's configuration to a JSON file and import it again, letting you copy a project's setup (including PICS) between Test-Harness instances or reuse it as a template for a new project.

```
# Export project 5 to '<project-name>-project-config.json'
th-cli project export --id 5

# Export to a specific output file
th-cli project export --id 5 --output-file door-lock-project.json

# Import a previously exported project config, creating a new project
```

```
th-cli project import --file door-lock-project.json
```

13.3.7. View Test Execution History

```
# List recent test runs
th-cli test-run-execution

# List all test runs
th-cli test-run-execution --all

# Sort by ascending order
th-cli test-run-execution --sort asc

# Filter by project
th-cli test-run-execution --project-id 5

# Skip the first 10 (ordered by ID) and fetch the next 20
th-cli test-run-execution --skip 10 --limit 20

# Print JSON output for more details
th-cli test-run-execution --json
```

To print the full report of a single test run execution (the file saved by the **Download Report** button in the web UI), add `--id`: `th-cli test-run-execution --id 17 --json > report-17.json`. See Collecting Files for ATL Submission.

13.3.8. Repeat a Test Run Execution

The `test-run-execution repeat` subcommand creates a new test run execution with the same selected tests and config as an existing one, then starts it and attaches to it the same way `run-tests` does (and the frontend's "Repeat" action does): streaming live test progress and forwarding any user prompts to this terminal.

```
# Repeat execution 87 and immediately start/stream it
th-cli test-run-execution repeat --id 87

# Override the generated title (the backend always appends an updated timestamp
regardless)
th-cli test-run-execution repeat --id 87 --title my-rerun

# Disable colored output for this execution
th-cli test-run-execution repeat --id 87 --no-color

# Disable the real-time web log viewer for this execution
th-cli test-run-execution repeat --id 87 --no-streaming
```

`--no-streaming` disables the browser-based real-time log viewer (enabled by default, normally

reachable at a URL printed at the start of the run); logs are still written to the local log file either way.

13.3.9. Export a Test Run Execution

The `test-run-execution export` subcommand exports a test run execution's config and results to a JSON file, which can later be imported into another project with `test-run-execution import`.

```
# Export execution 87 to '<execution-title>-execution.json'
th-cli test-run-execution export --id 87

# Export to a specific output file
th-cli test-run-execution export --id 87 --output-file execution-87.json
```

13.3.10. Import a Test Run Execution

The `test-run-execution import` subcommand imports a test run execution previously written by `test-run-execution export` into a given project.

```
# Import a previously exported execution into project 5
th-cli test-run-execution import --file execution-87.json --project-id 5
```

13.3.11. Export PICS Used by a Test Run Execution

The `test-run-execution pics-export` subcommand exports the PICS actually used by a specific test run execution to a ZIP file.

```
# Export the PICS used by execution 87 to '<execution-title>-pics.zip'
th-cli test-run-execution pics-export --id 87

# Export to a specific output file
th-cli test-run-execution pics-export --id 87 --output-file execution-87-pics.zip
```

13.3.12. Rename a Test Run Execution

The `test-run-execution rename` subcommand renames an existing test run execution.

```
# Rename execution 87
th-cli test-run-execution rename --id 87 --name "My Renamed Execution"
```

13.3.13. Delete a Test Run Execution

The `test-run-execution delete` subcommand permanently deletes a test run execution.

```
# Delete execution 87 (prompts for confirmation)
th-cli test-run-execution delete --id 87

# Delete execution 87 without a confirmation prompt
th-cli test-run-execution delete --id 87 --yes
```

Deletion is permanent and cannot be undone.

13.3.14. Archive / Unarchive a Test Run Execution

The `test-run-execution archive` and `test-run-execution unarchive` subcommands mark a test run execution as archived or restore it, without deleting it. Archived executions are hidden from the default `test-run-execution` list; pass `--archived` to view them instead.

```
# Archive execution 87
th-cli test-run-execution archive --id 87

# Unarchive execution 87
th-cli test-run-execution unarchive --id 87

# List only archived test run executions
th-cli test-run-execution --archived

# List archived executions for a specific project
th-cli test-run-execution --archived --project-id 5
```

`--archived` is not applicable when fetching a specific execution via `--id`.

13.3.15. Download Test Execution Logs

The `test-run-execution log` subcommand fetches logs for a specific test run execution.

```
# Print the flattened JSON log to stdout
th-cli test-run-execution log --id 123

# Save the flattened JSON log to a file
th-cli test-run-execution log --id 123 --output-file run_123.log

# Download a grouped log ZIP archive (logs grouped by test case state)
th-cli test-run-execution log --id 123 --grouped

# Download grouped logs to a specific file
th-cli test-run-execution log --id 123 --grouped --output-file run_123_logs.zip
```

When `--output-file` is omitted with `--grouped`, the filename is derived automatically from the test run execution title (e.g. `MyTestRun.zip`).

The `--log` flag on the base `test-run-execution` command is deprecated. Use the `log` subcommand instead. The following syntax still works but will emit a deprecation warning:

```
# Deprecated: use `th-cli test-run-execution log --id 123` instead
th-cli test-run-execution --log --id 123
```

The `--grouped` option downloads a `.zip` archive containing a list of the test cases for each state, the logs of the executed test suites, and one log file per state with the logs of all test cases that finished in that state. For large runs the server may take several minutes to generate the archive.

13.3.16. Collecting Files for ATL Submission

The web UI offers three download buttons for each test run execution in the test history. The CLI provides the same files:

Web UI button	File	CLI command
Download Report	`.json`	`th-cli test-run-execution --id 17 --json > report-17.json`
Download Logs	`.log`	`th-cli test-run-execution log --id 17 --output-file run-17.log`
Download Grouped Logs	`.zip`	`th-cli test-run-execution log --id 17 --grouped --output-file run-17.zip`

Replace `17` with the ID of the test run execution, which is listed by `th-cli test-run-execution`. The report command has no `--output-file` option, so the output is redirected to a file. The file names above are examples; the web UI names the files after the test run title.

`th-cli test-run-execution export` is not the report to submit to an ATL. It writes a different file, meant to be imported again with `test-run-execution import` (see Export a Test Run Execution).

To submit the results to an ATL:

- **Individual test run upload:** only the `.log` file is necessary.
- **Bulk upload:** each test run consists of one `.json`, one `.log` and one `.zip` file. Include the files of all test runs in a single outer `.zip` file.

To download the logs of every test run execution of a project at once, use `th-cli project logs` (see Download project logs).

13.3.17. Abort Running Tests

```
# Stop current test execution
```

```
th-cli abort-testing
```

13.4. Advanced Features

13.4.1. Camera Test Support

The CLI now supports video verification tests for camera-enabled devices. When running camera tests, the CLI automatically:

- Captures live video stream from the DUT camera
- Hosts a web interface for video verification on port 8999
- Saves video recordings in MP4 format
- Provides interactive prompts for test verification

Running Camera Tests:

```
# Run a camera test case
th-cli run-tests --tests-list TC-WEBRTC-1.1
```

During Test Execution:

1. The CLI will display a URL to access the video verification interface
2. Open the URL in a web browser (e.g., http://<your-ip>:8999)
3. View the live video stream from the camera
4. Respond to verification prompts through the web interface
5. Video recordings are automatically saved to `./videos/` directory

Camera Test Requirements:

- GStreamer must be installed (see Section 12, Matter Test Harness Cameras)
- Chrome browser with "Insecure origins treated as secure" flag enabled for the TH IP
- Network connectivity between CLI machine and Test-Harness

For detailed camera setup instructions, refer to Section 12, Matter Test Harness Cameras in this guide.

13.4.2. Auto-Completion

Enable command auto-completion for faster typing:

```
# The installation script sets this up automatically
# Press TAB to auto-complete commands and options
```

```
th-cli run-<TAB>  # Shows: run-tests
```

13.4.3. Colorized Output

The CLI automatically colorizes output for better readability:

- Green: Success/Pass
- Red: Errors/Failures
- Yellow: Warnings/Running
- Blue: Information

Disable colors if needed:

```
th-cli run-tests --tests-list TC-ACE-1.1 --no-color

# Or use environment variable
export TH_CLI_NO_COLOR=1
th-cli run-tests --tests-list TC-ACE-1.1
```

`--no-color` is also available on `test-run-execution repeat` (see Repeat a Test Run Execution).

13.4.4. Manual Test Cases

When commissioning devices, the CLI now automatically displays the **Manual Pairing Code** along with the **Node ID** for easier device setup while executing Simulated Test Cases.

13.5. Troubleshooting

Connection Issues:

```
# Verify Test-Harness is reachable
ping <raspberry-pi-ip>

# Check th-cli configuration
cat ~/certification-tool/cli/config.json
```

Command Not Found:

```
# Reinstall CLI
cd ~/certification-tool/cli
./scripts/th_cli_install.sh
```

Test Execution Fails:

```
# Check Test-Harness status
th-cli test-runner-status

# View execution logs
th-cli test-run-execution log --id <execution-id>
```

Camera Test Issues:

```
# Verify GStreamer is installed
gst-inspect-1.0 --version

# Check if port 8999 is available
netstat -an | grep 8999

# Verify camera stream connectivity
# Access http://<your-ip>:8999 in browser
```

14. Matter Test Harness Cameras

Please follow the instructions below to use the camera feature in the Test Harness for certification purposes. The camera feature allows users to run test cases that require a camera stream, such as those related to video streaming or image capture.

14.1. Prerequisites

Before using the camera feature for the certification, please make sure there is a functional Raspberry Pi TH environment setup. Refer to the Section 4.1.2, TH Installation on Raspberry Pi for the installation of the TH environment.

14.1.1. Install GStreamer

The camera feature requires GStreamer to be installed on the Raspberry Pi. Use the following linux command to install into the system:

```
sudo apt-get install \
    libgstreamer1.0-dev \
    libgstreamer-plugins-base1.0-dev \
    libgstreamer-plugins-bad1.0-dev \
    gstreamer1.0-plugins-base \
    gstreamer1.0-plugins-good \
    gstreamer1.0-plugins-bad \
    gstreamer1.0-plugins-ugly \
    gstreamer1.0-libav \
    gstreamer1.0-tools \
    gstreamer1.0-x \
    gstreamer1.0-alsa \
    gstreamer1.0-gl \
    gstreamer1.0-gtk3 \
    gstreamer1.0-qt5 \
    gstreamer1.0-pulseaudio
```

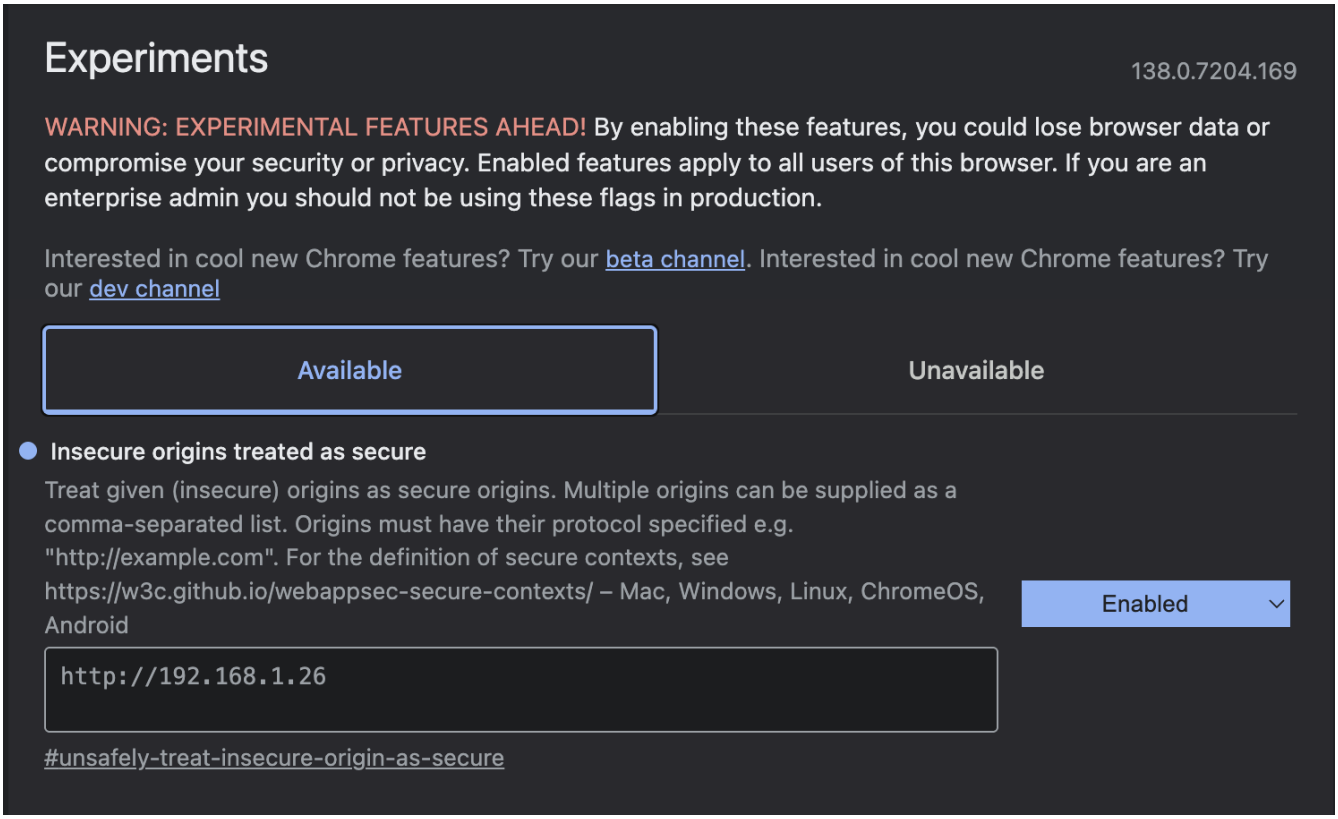
14.1.2. Chrome Browser Configuration

The camera feature requires the Chrome browser to be configured to allow a camera stream be shown in the video popup of the Test Harness. For that, it's necessary to enable the configuration flag "Insecure origins treated as secure" with an existing http address running the Test Harness application.

Follow the following steps to enable this required flag:

1. Open the Chrome browser and navigate to `chrome://flags/` in a new tab
2. Search for `Insecure origins treated as secure` flag
3. Enter the IP of the Test Harness into the field and enable the flag as shown in the image below

4. Relaunch the Browser



14.2. Setup Environment

Auto-update and start Test Harness with the camera feature (`v2.14+fall2025` or later) with the following linux commands:

- `cd ~/certification-tool`
- `./scripts/ubuntu/auto-update.sh v2.14+fall2025`
- `./scripts/start.sh`

14.3. Running Camera Test Cases

The Test Harness UI can be launched from a browser on any system on the same network by using the TH IP address. You can then use the camera application to run a camera-related Test Case.

14.3.1. Running chip-camera-app with USB Camera

The chip-camera-app located in the ~/apps directory can be executed using the following linux commands:

1. `rm -rf /tmp/chip_*`
2. `cd ~/apps`
3. `./chip-camera-app` (use `./chip-camera-app --camera-deferred-offer` for the **TC_WEBRTC_1_3** test case)

15. Platform Certification Configuration

Platform certification is the process of validating that a hardware or software platform meets specific technical and compliance standards. Certifying the platform allows device manufacturers to build products using a pre-approved foundation, reducing development time and simplifying the certification process for their final products.

15.1. Selecting Test Cases Rules

The Test Harness applies different rules to pre-select test cases depending on the type of certification being performed. The selection process takes into account the presence of specific flags in the PICS file and, in some cases, the content of additional configuration files such as the `platform-test.json` and `dmp-test-skip.xml` files.

The `dmp-test-skip.xml` file is provided by the PICS Tool, while the `platform-test.json` file is a static file that the Test Harness automatically downloads in the background from the SDK repository at the `connectedhomeip` project.

The three selection modes are:

- **Platform Certification**
- **Derived Product Certification**
- **Full Product Certification**

Each mode is designed to ensure that only the relevant test cases are executed based on the scope and purpose of the certification.

15.1.1. Platform Certification

In this mode, the product is being certified as a platform. The behavior is defined as follows:

- The PICS file must contain the `PICS_PLAT_CERT` flag set to `True`.
- The TH will pre-select test cases that:
  - Are explicitly listed in the new `platform-test.json` file.
  - Comply with the standard PICS rules.

15.1.2. Derived Product Certification

This mode applies to products built on a previously certified platform. The behavior is:

- The PICS file must contain the `PICS_PLAT_CERT_DONE` flag set to `True`, along with any product-specific PICS entries.
- The TH will pre-select test cases that:
  - Comply with the standard PICS rules.
  - Are **not** listed in the `dmp-test-skip.xml` (DMP file).

15.1.3. Full Product Certification

This is the default mode when the product does not fall under the previous two categories:

- The PICS file may contain any PICS entries, except `PICS_PLAT_CERT` and `PICS_PLAT_CERT_DONE`.
- The TH will pre-select test cases according to standard PICS rules only.

15.2. Configuration Input

On the project configuration screen, users can upload both the PICS file and the optional `dmp-test-skip.xml` file. Based on the content of the PICS file, the TH determines the certification mode by checking the values of the following flags:

- `MCORE.PLAT_CERT = True` → Platform Certification
- `MCORE.PLAT_CERT_DONE = True` → Derived Product Certification
- Neither flag set → Full Product Certification

This logic enables the TH to filter and pre-select the appropriate test cases based on the selected certification type.

The `dmp-test-skip.xml` file can be uploaded either by dragging and dropping the file into the upload area or by manually selecting it. Note that only one DMP file is accepted at a time. If a second DMP file is uploaded, the previous configuration will be discarded.

The file must be named exactly `dmp-test-skip.xml`. Renaming the file will result in it being ignored by the Test Harness.

Once the DMP file is uploaded, its content can be viewed in the panel on the left under the "dmp_test_skip" item, as shown in the figure.